

Quant Trading Firm Blueprint

**Research architecture, operating model
and implementation specification**

7 September 2026 | Revision 2

Profit as the objective. Scientific research and continuous improvement as the means.

16 required sections • Decision labels • Complete discussion record
Source traceability • Open implementation choices

Contents

Reading this blueprint

1. Executive Summary
2. Vision, Mission & Objectives
3. Legal & Regulatory Structure
4. Capital & Funding
5. Team & Operating Model
6. Technology Stack & Infrastructure
7. Data Strategy
8. Research & Strategy Development
9. Backtesting Framework
10. Execution & Order Management
11. Risk Management
12. Performance Monitoring & Reporting
13. Roadmap & Milestones
14. Budget & Resources
15. Risks, Assumptions & Open Questions
16. Decision Log / Traceability Appendix

Completeness Check & Questions

Navigation: PDF bookmarks include the major sections and all 22 source topics. The appendix preserves original examples and qualifications; it does not turn proposals into policy.

Reading this blueprint

Revision 2 and current implementation direction

[DECIDED] This edition incorporates the continuation of **Quant Firm Discussion**, source **T22**. The new user critique identifies three concrete weaknesses: institutional-scale complexity despite retail constraints; an **evaluation bottleneck** that silently asks a novice owner to validate expert financial engineering; and a **meta-research swamp** where the system studies itself instead of markets. The assistant acknowledged these weaknesses and proposed a substantially smaller start architecture. The user has requested this PDF revision, but has not separately ratified every proposed component, ratio or threshold.

[LEANING] The current architecture direction is **Research Lab first**, a virtual firm only after repeated useful research, and a trading system only at a later maturity stage. The earlier full architecture remains preserved as a future option and discussion history. It is no longer the default day-one implementation specification. Where an older component appears below, its stage designation governs its use.

Current start proposal	Deferred or conditional expansion
[LEANING] 1 owner, 1 main agent, 1 data source, 1 established backtester, LibreChat, Claude Code or Codex, GitHub, Parquet + DuckDB , simple research metadata and JSON/event logs.	[DISCUSSED, NO DECISION] PostgreSQL when object model/volume justify it; OpenTelemetry when real multi-agent complexity justifies it; formal model-risk records toward paper/live operation; additional engines for important surviving candidates.
[LEANING] Small fixed reference tests and a narrowly defined trusted kernel .	[DISCUSSED, NO DECISION] Large agent-evaluation programs, detailed owner measurement and sophisticated process-analysis infrastructure only after enough useful cases and evidence of need.
[LEANING] Complete one real Research Case from data acquisition to scientific verdict.	[DISCUSSED, NO DECISION] Every new layer must improve research quality, speed, reliability, cost or long-run profitability.

[DISCUSSED, NO DECISION] The proposed **80/20**, initially perhaps **90/10**, split between market research and supporting infrastructure is a resource-allocation suggestion, not an agreed spending/time mandate. The proposed candidate funnel and reviewer-removal example are illustrations, not achieved conversion rates or fixed thresholds. **T22**.

This blueprint turns the **Quant Firm Discussion** into a buildable research-system design. The primary objective is to make money; rigorous research and continuous improvement serve that objective. The next practical milestone is one complete real Research Case. A profitable strategy, live trading permission, committed capital and a fully implemented runtime have not been established.

The numbered sections are the requested structure. Section **16** supplies decision traceability and a complete chronological discussion record to preserve every example, number, alternative and revision without mistaking them for adopted policy. References such as **T06** identify complete user/assistant exchanges, ordered oldest to newest. The source covers **22 substantive exchanges**, including the critique and architecture revision after the blueprint request. The request itself is identified separately. The attached measurement image was inspected directly.

Label	Interpretation in this document
[DECIDED]	Explicit user direction or confirmed existing circumstance; an existing circumstance is distinguished from a new design commitment.
[LEANING]	A preference supported by the discussion, without a finalized implementation choice.

Label	Interpretation in this document
[DISCUSSED, NO DECISION]	An option, assistant proposal, example, comparison or unsettled question. An assistant recommendation alone is not user agreement.
[IMPLIED]	An explicitly flagged inference from stated objectives or constraints.
[NOT DISCUSSED / TBD]	A necessary specification absent from the discussion; any options here are new suggestions.

Figures in examples are illustrative, not achieved results, forecasts, approved thresholds or spending commitments. Product capabilities and repository observations are preserved as historical discussion claims unless a verification note explicitly says otherwise. Original source citation tokens were not resolvable as URLs and are omitted from the readable preservation record; turn provenance remains intact. This document does not upgrade those historical claims to current verified facts.

1. Executive Summary

[DECIDED] Build a personal **virtual quant firm** that faithfully backtests retail strategies, generates its own hypotheses, and adheres to scientific standards. A trading bot may emerge later. The end goal is making money; improving agents, the human in the loop and the research process is the secondary route toward greater profit. **T06, T21**.

[DECIDED] Keep the system proportionate to retail trading. The owner rejects a risk layer so large that it slows or consumes the research process. The owner is not a programmer, is new to vibe coding, and reports only basic finance, mathematics, statistics, ML and AI knowledge. These are operating constraints, not reasons to abandon the project. **T04, T14, T20**.

[DECIDED] (existing circumstance) **LibreChat** is installed and the **GitHub** repository **RealMonoid/trading-research-framework** exists. [LEARNING] Use **LibreChat** as the multi-model research/control interface and **Claude Code** or **Codex** as a bounded implementation executor, with **GitHub** branches, pull requests and CI. Earlier advice to move to **Replit** was explicitly revised after this context emerged. **T01-T05**.

[LEARNING] **T22 revises the start specification to a small Research Lab**: one main agent, one established engine, one provider, **Parquet + DuckDB**, simple metadata and logs. A compact trusted kernel carries correctness-critical responsibilities. The larger provider-neutral scientific harness is an expansion direction, not a prerequisite for the first case. Agents generate, synthesize, interpret and criticize. Programs calculate, execute frozen experiments, enforce permissions, preserve history and determine whether specified gates pass. The conductor coordinates but cannot rewrite evidence or allocate real capital. **T10-T14**.

[DISCUSSED, NO DECISION] First markets considered were liquid US equities/ETFs, particularly **SPY**, with **QQQ**, cross-sectional equities, **ES** futures, options and market microstructure as alternatives or later extensions. No asset universe, trading horizon or strategy was selected. Proposed data choices were **Massive**, **Databento**, **dxFeed**, **Norgate Data** and **QuantConnect**. **Backtrader** and **LEAN** were the existing synthetic-fixture engine boundary reported in the conversation; **RealTest** was a possible later portfolio research engine. **T06-T09**.

[LEARNING] First deliverable: freeze a small executable research protocol, register one simple public retail strategy, acquire and admit an appropriate immutable dataset, run one established backtester with realistic execution assumptions, preserve failures and produce a defensible verdict. Completing the entire harness, observability stack or database is not the milestone. This is a process qualification milestone serving the profit objective, not a replacement for it. Instrument meaningful runs immediately; establish baselines and charts only after comparable observations exist. **T06, T17, T21**.

[NOT DISCUSSED / TBD] Capital, operating budget, jurisdiction, entity, broker, risk tolerance, numerical promotion thresholds, live start date and scaling policy remain open. The blueprint makes those gaps visible instead of inventing a business plan.

2. Vision, Mission & Objectives

2.0 Economic purpose governs complexity

[LEANING] The new continuation makes **earned complexity** the governing direction: begin with a retail research system, establish usefulness, then grow selectively. A layer must improve research quality, speed, reliability, cost or long-run profitability to justify its continued existence. [DISCUSSED, NO DECISION] The proposed rule is that process improvement must not consume more resources than the problem it is intended to solve. Measurement of that comparison, time horizon and decision authority remain unspecified. **T22**.

2.1 Objective hierarchy

[DECIDED] Profit is the primary objective; continuous improvement is secondary and must contribute to profit for agents and the human in the loop. **T21**.

[DISCUSSED, NO DECISION] The assistant refined this into “maximize sustainable long-run net profit subject to survival and evidence-quality constraints.” The proposed hierarchy was long-run risk-adjusted net returns; avoidance of ruin, catastrophic drawdowns, operational failure and uncontrolled model risk; distinguishing genuine edge from noise; and improvements to humans, agents, harness, data, research and execution. This is a proposed operational formulation, not a fully specified objective function.

[LEANING] Preserve two coupled loops: capital -> research -> strategies -> paper/live execution -> returns -> capital; and agents + human + harness -> decisions -> outcomes -> measurement -> interventions -> better decisions. Process improvement must connect to research quality, strategy quality and eventually net economic outcomes. Next week's P&L alone cannot reliably evaluate a research-method change. **T15-T21**.

2.2 Success criteria and constraints

Area	Status and specification
Immediate qualification	[LEANING] One end-to-end real Research Case reaches a defensible rejection, survival or inconclusive result without evidence contamination.
Economic success	[NOT DISCUSSED / TBD] Define net-profit accounting, evaluation horizon, benchmark and capital base; separate strategy returns from firm operating costs. Suggested accounting choices require approval.
Learning	[DECIDED] Improve both agents and owner as a means toward better profitability. [LEANING] Measure behavior and intervention effects instead of sophistication, quiz scores or model reputation.
Proportionality	[DECIDED] Retail-scale middle ground. [LEANING] Light exploration, moderate confirmation and strict paper/live controls.
Human capacity	[DECIDED] Beginner programming and basic domain knowledge. [IMPLIED] Interfaces must support informed supervision without assuming expert error detection.
Time, money, risk appetite	[NOT DISCUSSED / TBD] No weekly hours, deadline, committed budget, target return, acceptable drawdown or loss capacity specified.

[DISCUSSED, NO DECISION] Preserve the doctrine: “Profit is the objective. Scientific rigor protects the profit. Continuous improvement compounds the capability to produce it.” Also preserve the constraint against optimizing short-term profit at the expense of the ability to discover whether an edge exists. Neither education nor an elaborate harness is an end in itself. **T21**.

3. Legal & Regulatory Structure

[LEANING] **Stage correction from T22:** use relevant model-risk principles at the start; do not build a formal **SR-26-2-style** governance program as a Research Lab prerequisite. More formal governance was proposed toward paper/live maturity. Deferring bureaucracy does not decide legal applicability or postpone legal/data-rights questions that are necessary for a chosen activity.

[NOT DISCUSSED / TBD] Entity type, residence/jurisdiction, tax status, licensing, whether only personal capital will be traded, third-party money, advisory activities, IP ownership and reporting obligations were not settled. Do not infer Germany or a legal domicile from conversational references to **Darmstadt**, German institutions, the name **Gustl**, or this document's preparation environment.

Topic	Open specification and suggested options
Entity and capital ownership	[NOT DISCUSSED / TBD] Personal proprietary activity versus a separate entity; determine the actual activity before selecting structure.
Third-party activity	[NOT DISCUSSED / TBD] Own account only, future external capital, signals or software provision; no fundraising mandate exists.
Licensing and compliance	[NOT DISCUSSED / TBD] Obtain jurisdiction-specific review once activity and instruments are known. No exemption or license is assumed.
Tax and accounting	[NOT DISCUSSED / TBD] Treatment of gains, derivatives, expenses, records and entity-level taxation.
Data rights	[LEANING] Keep licensed data private. [NOT DISCUSSED / TBD] Confirm local retention, historical replay, derived outputs, cloud/LLM processing and redistribution permissions per vendor contract.
IP and repository license	[NOT DISCUSSED / TBD] Ownership of research, code and agent-created artifacts; contributor and dependency license compatibility.
Owner measurement records	[DISCUSSED, NO DECISION] Measure decision patterns as a learning aid. [NOT DISCUSSED / TBD] Retention, access and correction policy for those records.

[DISCUSSED, NO DECISION] **SR 11-7** was raised as an inspiration for model risk management. The assistant updated this to **SR 26-2**, then the user required a proportionate retail implementation. Borrow intended use, model inventory, limitations, validation, version control and monitoring, rather than institutional committees or a claim of regulatory compliance. **T13-T14**.

Verification note: the **Federal Reserve** confirms that **SR 26-2**, dated **17 April 2026**, supersedes **SR 11-7** and **SR 21-8**. Its attached guidance excludes generative and agentic AI from scope; this is not a substitute for agent governance. These facts do not establish the firm's legal obligations. Sources: [Federal Reserve SR 26-2](#) and [attached guidance](#).

4. Capital & Funding

[DISCUSSED, NO DECISION] **T22** adds maturity stages but supplies no actual risk capital, position sizing, loss limit or funding commitment. Its **80/20** and **90/10** examples refer to work/resources for research versus support, not portfolio weights or capital allocations.

[NOT DISCUSSED / TBD] No starting capital, source of capital, leverage allowance, reinvestment percentage, salary, fee structure, compensation scheme or fundraising timetable was given. No capital is authorized by this blueprint.

[DISCUSSED, NO DECISION] Proposed progression: backtest -> locked holdout -> paper trading -> tiny capital -> monitored capital -> possible scaling. Initially, complete loss should be annoying rather than consequential. This qualitative proposal supplies no numerical amount. Agents cannot increase exposure because they report confidence. **T06, T18.**

Required capital rule	Discussion state
Research versus trading allocation	[NOT DISCUSSED / TBD] Separate operating spend, data acquisitions, implementation and actual risk capital; amounts unanswered.
Position and strategy allocation	[DISCUSSED, NO DECISION] Deterministic strategy allocation and position limits outside agent discretion. [NOT DISCUSSED / TBD] Sizing method and values.
Drawdown and account loss	[DISCUSSED, NO DECISION] Maximum drawdown, kill conditions and account-level loss limits. [NOT DISCUSSED / TBD] Definitions, measurement window, values and reset authority.
Promotion and scaling	[DISCUSSED, NO DECISION] Human-controlled capital authorization after progressively stronger evidence. [NOT DISCUSSED / TBD] Required duration, sample size and scaling increments.
Reinvestment and withdrawals	[NOT DISCUSSED / TBD] Retain profits, withdraw periodically or allocate by validated capacity; options only.

5. Team & Operating Model

5.0 Research Lab staffing and the owner boundary

[LEANING] **Start with one owner and one main research agent**, plus one bounded coding executor. The role catalog below describes responsibilities and later specialization; it does not require a separate model/process for every role on day one. Multiple agent roles and separate validation capability belong to the proposed **Stage 2 Virtual Quant Firm**. Do not confuse one main agent with authority to certify its own claims. **T22**.

[LEANING] The owner should verify that established implementations were used and reference tests passed, rather than personally approving the correctness of finance formulas, timestamp alignment or corporate-action mathematics. The design must reduce custom financial engineering and concentrate assurance on a small trusted kernel. Passing tests does not establish that a test suite or selected library covers every financial assumption; unresolved expert questions still require a defensible reference or independent expertise. The last sentence is an [IMPLIED] consequence of the stated evaluation-bottleneck concern, not a new claim that a specific kernel has been validated.

[DISCUSSED, NO DECISION] Reciprocal owner measurement remains a possible later capability. At the start, detailed owner metrics are reduced to almost nothing; recording necessary human interventions is distinct from building a behavioral scoring system. **T22**.

5.1 Virtual roles and information boundaries

[DECIDED] This is a virtual firm, not an agreed hiring plan. [LEANING] As roles are introduced, define them by information and authority, not merely different prompts sharing the same conversation. Start with one coding executor; the historical recommendation explicitly cautioned against **five** simultaneous coding agents. **T05-T06, T10**.

Role	Work and authority boundary
Owner / Investment Committee / Capital Committee	[DISCUSSED, NO DECISION] Set objectives, challenge assumptions, review owner briefs and authorize capital; owner approval is not expert validation.
Research Director / Conductor	[LEANING] Read state, select permitted next steps, assign bounded jobs, compare outputs, request validation, assemble reports and flag inconsistencies. No evidence rewriting, holdout unlocking, promotion alone or trading.
Hypothesis Researcher	[LEANING] Read literature and market facts, synthesize mechanisms, produce unranked INBOX candidates and preserve the generated universe. No current holdout outcomes.
Literature researcher	[DISCUSSED, NO DECISION] Retrieve primary material, verify citations and competing explanations; distinguish citation presence from verified support.
Methodologist / Statistician	[LEANING] Review design, multiple testing, uncertainty and falsification. Justify methods; verified deterministic tools perform calculations.
Data Engineer / Data analyst	[LEANING] Admit fit data, check availability at decision time and maintain provenance; cannot turn attractive results into edge claims or authorize trades.
Backtest executor / Holdout executor	[LEANING] Run frozen specifications on authorized immutable data. Cannot modify hypothesis or acceptance criteria. Holdout execution should be deterministic.
Independent Validator	[LEANING] Review preregistration, results and search history. Cannot repair a failing strategy inside the same validation claim.
Robustness / Risk / Execution specialist	[DISCUSSED, NO DECISION] Challenge costs, liquidity, capacity, portfolio interactions and live readiness.
External quant/statistician	[DISCUSSED, NO DECISION] Occasional critical review before architecture freeze, first serious validation, paper/live milestones or major methodology changes; no employee commitment.

[DISCUSSED, NO DECISION] Adversarial review roles include proponent, skeptic, methodologist, statistician and execution specialist. Independent first passes should precede sharing arguments to limit anchoring. Resolve disagreements by evidence; **Claude + GPT + Gemini** consensus is not independent truth. Proposed authority classes: **A generative** high freedom/low trust; **B analytical** medium freedom/evidence required; **C execution** low freedom/deterministic verification; **D governance** near-zero agent authority. **T10**.

5.2 Owner cooperation and learning

[DECIDED] The owner endorses reciprocal measurement as real cooperation. [LEANING] Track rejection overrides, requests for more tests after failures, goal changes after outcomes, recurring misunderstandings, outcome preference over protocol, helpful/harmful interventions and explanation needs. Measure research-relevant behavior; do not present an invented personal competence score. **T18-T21**.

[DISCUSSED, NO DECISION] Every material decision should include a technical report and an owner brief: what was tested, findings, what may be wrong, evidence present/absent, requested action, what would falsify the result, key assumptions, untested areas, skeptical objections and consequences of error. A later three-layer form is **DECISION -> WHY -> TECHNICAL BASIS**, with **REJECT / CONTINUE / INCONCLUSIVE**. **T18, T20**.

[LEANING] Learn just in time: statistics/research methodology first; backtesting/market mechanics second; finance third; selective math; ML when justified; AI/agents as systems engineering. Statistics includes probability, sampling, uncertainty, confidence intervals, hypothesis and multiple testing, selection bias, regression and non-stationarity. Finance includes returns, volatility, drawdowns, Sharpe/Sortino, beta, factors, diversification, leverage and tails. Math includes algebra, logs, calculus intuition, vectors/matrices and optimization. ML includes temporal splits, leakage, features, regularization, calibration, class imbalance and drift. AI topics include stochasticity, hallucination, tool use, contamination, evals, tracing, permissions and repeated-run reliability. **T18-T20**.

[DISCUSSED, NO DECISION] Competence envelopes for the firm and owner are proposed, not assessed. The sample firm ratings of US equities HIGH CONFIDENCE, futures MODERATE, options/microstructure LIMITED and HFT/OTC UNSUPPORTED are illustrative. The owner's BASIC+/BASIC/LOW table is likewise not an examination result. Permit **INSUFFICIENT EVIDENCE** and **OUTSIDE VALIDATED COMPETENCE**, with independent validation when material decisions exceed demonstrated competence.

6. Technology Stack & Infrastructure

6.0 Revised start architecture and trusted kernel

[LEANING] Current starting proposal: **LibreChat -> Claude Code or Codex -> GitHub -> one backtester -> one data source -> Parquet + DuckDB -> simple research metadata**. This is the discussion's component sketch, not a literal runtime request sequence. The provider-neutral design should leave room for replacement without requiring multiple providers or an extensive platform now. **T22**.

Component	Research Lab starting position	Later trigger or disposition
Parquet + DuckDB	[LEANING] Files for numerical data plus analytical querying/simple metadata; explicit start preference.	[DISCUSSED, NO DECISION] Retain where useful. DuckDB is an analytical engine, not a required third full copy of data.
PostgreSQL	[LEANING] Defer; replaces the earlier day-one metadata preference.	[DISCUSSED, NO DECISION] Introduce when data model and volume justify it. No threshold chosen.
OpenTelemetry	[LEANING] Defer; start with simple JSON/event logs .	[DISCUSSED, NO DECISION] Actual multi-agent complexity may justify full tracing.
Multiple engines	[LEANING] One established engine for routine research.	[DISCUSSED, NO DECISION] Independent replication for important survivors; not every hypothesis.
Agent eval infrastructure	[LEANING] Small fixed reference suite.	[DISCUSSED, NO DECISION] Systematic agent evaluation when evidence and scale justify it.
Model risk governance	[LEANING] Principles and essential controls.	[DISCUSSED, NO DECISION] Formalization toward paper/live operation.

[LEANING] The **trusted kernel** concentrates responsibility in **data normalization, timestamp logic, corporate actions, portfolio accounting, costs/slippage, performance metrics and holdout locks**. Keep these components small, strongly tested and rarely changed. Use established backtest mechanics and verified libraries or known testable reference implementations for **Sharpe, drawdown, returns, regression, bootstrap, PBO, DSR, corporate actions and portfolio accounting**. This list is an assurance boundary, not an instruction to build every statistical method for the first case. **T22**.

[NOT DISCUSSED / TBD] Select and identify the actual trusted implementations, reference datasets/test cases, version pins and responsible expert escalation path. No product was certified trustworthy in the conversation. The physical format for simple metadata remains open: **DuckDB tables, JSON files** or another minimal record form are implementation suggestions, not user choices.

6.1 Architecture and product choices

Layer	Choice or alternatives and disposition
Research interface	[DECIDED] (existing) LibreChat installed. [LEANING] Retain it for multi-provider research, discussion and orchestration.
Coding executor	[LEANING] Claude Code initially suggested due to prose/policy-heavy repo and CLAUDE.md bootstrap; Codex equally compatible. Cursor remains an alternative, not rejected categorically.
Editors and beginner route	[DISCUSSED, NO DECISION] VS Code, Zed, Cursor . Earlier Replit Agent -> Cursor -> Claude Code/Codex advice was superseded for this project after repository context.
Models/providers	[DISCUSSED, NO DECISION] Claude Opus/Sonnet, GPT, Gemini, DeepSeek, Qwen, Mistral, Ollama, OpenRouter, Groq, OpenAI, Anthropic, Google, Azure OpenAI, AWS Bedrock, Google Vertex AI . None is an exclusive firm commitment.

Layer	Choice or alternatives and disposition
Routing	[DISCUSSED, NO DECISION] LiteLLM gateway, model routing/load balancing, OpenAI -compatible endpoints, per-agent model/instructions/tools/MCP/Skills. Hosting options are not cloud infrastructure decisions.
Languages	[DISCUSSED, NO DECISION] Python is the analytical direction; JS/TS, Go, C/C++, Java, Rust were named as interpreter capabilities, not competing adopted implementation stacks. No Python-vs-C++-vs-Rust firm decision was made.
Metadata and numerical data	[LEANING] T22 supersedes the starting choice with Parquet + DuckDB and simple metadata ; ordinary filesystem remains suitable. PostgreSQL is deferred until justified. Arrow, Polars, pandas, NumPy, JSON, CSV were discussed in supporting roles.
Later object storage	[DISCUSSED, NO DECISION] MinIO, AWS S3, Backblaze B2 after local storage is insufficient.
Engines	[LEANING] Preserve Backtrader / LEAN and normalize results. [DISCUSSED, NO DECISION] RealTest later; vectorbt mentioned as another instrument, not selected.
Versioning and delivery	[LEANING] Git, GitHub , branches, PRs, CI, schemas, validators, ADRs, tests and synthetic fixtures.
Operational telemetry	[DISCUSSED, NO DECISION] OpenTelemetry , GenAI conventions and spans; Jaeger and Grafana mentioned, with no backend commitment.
Harness references	[DISCUSSED, NO DECISION] LangChain, OpenAI Agents SDK, Anthropic harness/eval material, NIST TEVV, HAL and Deming concepts are references, not dependencies adopted by the user.

Historical tool scorecards, model-generation names including **GPT-5.6 Sol / Terra / Luna**, speculative future model names, and all provider alternatives remain in Section 16. Those scorecards are subjective assistant comparisons, not measured benchmark results.

6.2 Scientific harness contract

[LEANING] Keep essential scientific rules beneath/alongside the generic host in the small start system. **T22 defers the full component architecture below** until demonstrated need. Previously proposed expansion components: research state machine; policy/permission engine; experiment and dataset registries; immutable search ledger; holdout manager; backtest gateway; evidence registry; validation engine; audit log; measurement system. Do not build a giant replacement chat application or require the complete component catalog merely to run the first case. **T11-T14**.

[LEANING] A backtest request must resolve: registration, frozen hypothesis, role authorization, data admission, prior holdout exposure, registered parameter family, search budget and whether the action is a new experiment. Enforce checks in software rather than relying on prompt obedience. The conductor requests transitions; the framework validates them.

[DISCUSSED, NO DECISION] Detailed workflow proposed in **T10: IDEA -> HYPOTHESIS_DRAFT -> METHODOLOGY_REVIEW -> PREREGISTERED -> DATA_ADMITTED -> DEVELOPMENT -> FROZEN -> VALIDATION -> HOLDOUT -> REPLICATION -> SURVIVED / REJECTED**. Later lightweight workflow in **T14: IDEA -> EXPLORATION -> CANDIDATE -> VALIDATION -> SURVIVOR -> PAPER TRADING -> LIVE**. Preserve both as different granularity views; formal transition definitions remain TBD.

[LEANING] **DEVELOPMENT -> FROZEN** requires strategy specification, registered parameter ranges, frozen data/execution/cost specifications, search count and code hash. Changing a parameter after results creates a new version and recorded search event. [NOT DISCUSSED / TBD] Define precisely when a new run, experiment, strategy version or hypothesis is required; define idempotency, concurrent updates and transaction boundaries before schema implementation.

6.3 Persistence and repository boundaries

[LEANING] Core objects: **Hypothesis, ResearchCase, Dataset, DatasetVersion, Strategy, StrategyVersion, Experiment, BacktestRun, ValidationRun, Decision, Artifact**. Supporting records include parameters, metrics, agents, agent actions, citations, evidence, costs, model inventory, interventions and audit events. First settle scientific identity/relationships, then implement tables. **T08**.

[LEANING] Keep raw market data immutable; canonical data immutable by version; store heavy observations and numerical results as **Parquet**; keep metadata, hashes, decisions and lineage in simple research records initially; the earlier **PostgreSQL** implementation is a later option; store reports, plots, logs, models and backtest outputs in artifact storage. **GitHub** holds code, schemas, adapters, tests, synthetic fixtures and research logic. Keep credentials, licensed datasets, private results and future trading configuration in a private environment.

[DISCUSSED, NO DECISION] The earlier assistant reported **90 commits**, later **116 commits**, and a provider-neutral specification/toolkit used by a host agent rather than a standalone runtime. These are separate historical snapshots, not proof that the current repository implements this blueprint. It reported **AGENTS.md** as authoritative, **PLANNED_FEATURES.md** as roadmap and **AGENT_CHANGELOG.md** as required reading, with feature branches, no direct main pushes, state inspection, CI/PR merges, bounded delegation and fingerprint checks. **T05-T07**. Repository reference: [trading-research-framework](#).

[NOT DISCUSSED / TBD] Production OS, deployment provider, packaging, scheduler, API contracts, secrets manager, backup/recovery objectives, retention, disaster recovery, authentication and exact CI checks need specification. Suggested minimum acceptance work is a recoverable local vertical slice with tests that show denied actions remain denied and prior evidence remains reconstructable.

7. Data Strategy

[LEANING] **T22 start scope**: one data source, one narrowly specified dataset and an established normalization/reference-data path appropriate to the first case. The multi-vendor catalog below preserves alternatives; it is not a request to integrate every vendor. Normalization, timestamps and corporate actions belong to the trusted kernel, and the owner is not expected to approve custom implementations through domain intuition alone. The prospective fitness and immutable-input principles remain.

7.1 Provider-neutral ingestion

[LEANING] **Vendor APIs/files -> vendor adapter -> immutable raw data -> quality/admission checks -> canonical dataset -> Backtrader/LEAN adapters -> normalized results**. A backtester consumes frozen bytes, not a mutable live vendor response. Corrections produce new dataset versions rather than replacing old evidence. **T07-T08**.

[LEANING] Freeze a prospective **DATASET_SPEC** before access to empirical outcomes. Specify asset class, universe, frequency, required fields, sessions, history, point-in-time requirements, execution model, quote need, corporate actions and timestamp precision. Assess fitness and limitations before acquisition/admission. The source's **SPY, 1 minute, >=10 years, next-bar market execution, quotes=no** example is illustrative; its quote-free execution assumptions still require justification.

Vendor option	Historical discussion rationale and limitation
dxFeed	[DISCUSSED, NO DECISION] Broad equities, futures/options, indices, FX, crypto; trades/quotes, books, candles, replay, TickData API/CSV, NBBO/regional quotes, OPRA, Greeks/IV/theoretical services. Tick history described as roughly 2009 onward depending on instrument. Not automatically the first purchase.
Databento	[LEANING] (assistant preference) Particularly interesting for futures and consistent data/reference APIs, symbology, definitions, status, corporate actions, adjustments, contracts, settlements, OI/expiration and options chains. Historical 16+ years and \$199/month CME example retained, not a budget authorization.
Norgate Data	[DISCUSSED, NO DECISION] Daily cross-sectional equities, delisted securities and historical index membership; EOD and Windows-centric constraints noted. Could supply point-in-time universe alongside intraday prices from another provider.
Massive / Polygon.io	[DISCUSSED, NO DECISION] US stocks/ETFs, REST/bulk files, bars/trades/quotes; name change stated as October 2025 , equity history as 2003 onward. Candidate for simpler first adapter.
QuantConnect data / LEAN	[DISCUSSED, NO DECISION] Convenient integrated data; stated history equities 1998 , equity options 2010 , futures 2009 . Preserve independently reproducible frozen inputs across engines rather than silently different feeds.

These are preserved procurement considerations, not a fresh comprehensive vendor audit. Exact coverage, correction policies, export formats and rights must be checked for the chosen dataset. The historical **\$199/month** claim was not adopted as a current quote; see [Databento pricing](#).

7.2 Data requirements by hypothesis

Strategy family	Discussed data need
Weekly momentum / moving averages	[DISCUSSED, NO DECISION] Daily OHLCV.
Opening-range breakout	[DISCUSSED, NO DECISION] 1-minute bars.
VWAP	[DISCUSSED, NO DECISION] Trades/volume or fine intraday bars.
Bid/ask mean reversion / spreads	[DISCUSSED, NO DECISION] Quotes / simultaneous quotes.

Strategy family	Discussed data need
Order-flow imbalance	[DISCUSSED, NO DECISION] L2/L3 order book.
Options volatility	[DISCUSSED, NO DECISION] Chain + NBBO + underlying + OI.
Intraday futures	[DISCUSSED, NO DECISION] Individual contract trades/quotes.
HFT / microstructure	[DISCUSSED, NO DECISION] Tick + order book; not demonstrated firm competence.

[LEANING] Validate timestamps/time zones, sessions, gaps, duplicates, bad ticks, corporate actions, adjustments, survivorship, point-in-time eligibility and decision-time availability. A present-day **S&P 500** membership list is not a historical universe. The example buying the **20** strongest stocks monthly must include former constituents and delistings. **T07**.

[DISCUSSED, NO DECISION] Canonical files named **bars.parquet**, **trades.parquet**, **quotes.parquet**, **instruments.parquet**, **corporate_actions.parquet**, **sessions.parquet**, accompanied by **manifest.json**, **quality_report.json**, **hashes.json**. Manifest fields: provider, download time, market, symbols, period, resolution, timezone, adjustment, extended hours, quote type, vendor dataset/version, API parameters and **SHA256**. The sample **UTC**, raw adjustment, no extended hours and **NBBO** values are not universal defaults.

[NOT DISCUSSED / TBD] Select first dataset and dates, symbol mapping, bar timestamp semantics, missing-data policy, adjustment method, session calendar, contract roll policy, precision, compression/partitioning and data-budget ceiling. Do not buy the illustrative **20 TB** of ticks merely because it is available.

8. Research & Strategy Development

8.0 Narrow initial research domain

[LEANING] **T22** proposes deliberately limiting the first domain to **US equities / ETFs, daily or 1-minute** data, and **long/short or long-only** strategies. The alternatives remain unresolved. Proposed exclusions at the start: **options, exotic derivatives, HFT and complex ML models**. No specific instrument, strategy, resolution or shorting permission has been selected. The broader ideas catalog below remains future research context, not the launch universe.

8.1 Research doctrine

[DECIDED] Support both faithful tests of public retail strategies and generation of original hypotheses with scientific standards. [LEANING] Separate phenomenon, prediction, causal mechanism and executable net edge. Preserve the entire generated candidate universe and global search history, including abandoned branches and failures. **T06**.

[LEANING] Generate mechanism-first hypotheses from institutional constraints, forced flows, microstructure states, arbitrage/linkages, literature replication and observations. An unranked **INBOX** generator should not secretly screen candidates on protected results. Exploration may be fast, but an exploratory pattern becomes a registered confirmatory candidate, not an edge claim. **T06, T12, T14**.

8.2 All strategy and model ideas

Idea	Context and status
Simple public retail strategy	[LEANING] First complete case; exact strategy unspecified.
Moving averages, daily momentum, weekly momentum	[DISCUSSED, NO DECISION] Simple learning/data examples; no chosen windows.
Momentum, mean reversion, stock screening, factors, rotation, index constituents	[DISCUSSED, NO DECISION] Daily equity research suited to historical universe data.
SPY/QQQ intraday, opening-range breakout, VWAP, RSI	[DISCUSSED, NO DECISION] Candidate examples and data requirements, not selected alpha.
End-of-session hedging pressure	[DISCUSSED, NO DECISION] Example mechanism in search-ledger record HYP-00482 .
Forced deleveraging near institutional rebalancing	[DISCUSSED, NO DECISION] Temporary liquidity pressure followed by reversal after constrained flow disappears.
Small-cap overnight selling and opening liquidity recovery	[DISCUSSED, NO DECISION] Intraday reversal illustration HYP-0281 , point-in-time US common stock universe.
Tuesday reversals / conditional liquidity stress	[DISCUSSED, NO DECISION] Exploration illustrations; subgroup findings after failure require a new hypothesis and experiment.
Bid/ask effects, spread strategies, order-flow imbalance	[DISCUSSED, NO DECISION] Quotes/books and realistic execution required.
Futures, options, volatility and microstructure	[DISCUSSED, NO DECISION] Possible later domains, not demonstrated strategy capability.
Volatility/regime/forecast/risk/cost/slippage/sizing models	[DISCUSSED, NO DECISION] Model-inventory categories; ML only where justified.
Hierarchical Bayesian regime-dependent latent volatility model	[DISCUSSED, NO DECISION] Example beyond owner's demonstrated competence, not a build choice.

Idea	Context and status
Exotic volatility-surface arbitrage / OTC derivatives / HFT	[DISCUSSED, NO DECISION] Examples of competence-boundary escalation, not planned products.

[DISCUSSED, NO DECISION] Anomalies should trigger investigation of tick-size rules, market structure, index methodology, vendor defects, volatility regimes and causal explanations. The example with normal **2010-2018**, collapse **2019-2020**, recovery **2021-2025** illustrates adaptive agent investigation. It is not observed strategy performance. **T12**.

8.3 Search and evidence controls

[LEANING] Record candidate family, parent hypotheses, dataset roles/exposure, permitted variants, actual attempts, failures, selection objective, intermediate inspection, strategy/code versions, data hashes, costs, slippage and verdict. Repeatedly expanding a parameter family after seeing results is additional search, even under a new name.

[DISCUSSED, NO DECISION] Methods named: **White's Reality Check**, **Deflated Sharpe Ratio (DSR)**, **Probability of Backtest Overfitting (PBO)** / **CSCV**, bootstrap, regression, correlations, confidence intervals and independent replication. No universal statistical test or numerical significance/acceptance threshold was adopted. Choice must fit assumptions and registered research design. **T06, T09, T18**.

[DISCUSSED, NO DECISION] Proposed evidence levels **0-6**: unsupported LLM assertion; cited claim; retrieved-primary-source verification; deterministic computation from immutable data; independent replication; prospective/untouched holdout; forward/paper evidence. Treat this as a proposed taxonomy, not a proof that each higher number resolves all lower-level defects. **DEVELOPMENT_ONLY**, provisional evidence and explicit uncertainty must remain visible. **T10, T18**.

[NOT DISCUSSED / TBD] Define the first hypothesis, search-family identity, budget, minimum sample, primary outcome, statistical procedure, selection correction, benchmark and promotion threshold before confirmation. No return target or model type may be inferred from the examples.

9. Backtesting Framework

9.0 One established engine first

[LEANING] **T22** removes routine multiple-engine execution from the Research Lab start. Select **one established backtester**, use its portfolio/accounting mechanics, and test the few integrations and assumptions that matter. Independent engine replication is reserved for important surviving candidates rather than applied to every exploratory hypothesis.

[DISCUSSED, NO DECISION] Illustrative funnel: **100 ideas -> 20 serious tests -> 5 interesting candidates -> 2 validations -> 1 candidate -> independent engine replication**. These figures are not throughput targets, sample requirements, acceptance rates or an adopted promotion policy. The exact point at which replication becomes mandatory remains **[NOT DISCUSSED / TBD]. T22**.

[LEANING] Engines are instruments, not researchers. The scientific layer supplies a frozen **ExperimentSpec**: strategy, data, costs, portfolio rules, allowed parameters and acceptance criteria. Deterministic tools return normalized artifacts; the scientific layer judges evidence. **T09**.

Engine	Discussed role and trade-off
Backtrader	[LEANING] Simpler reference implementation and existing synthetic fixture boundary. Real-data admission/instrument-specific integration reported unfinished.
LEAN	[LEANING] Independent implementation/replication; integrated data convenience must not undermine identical frozen input provenance.
RealTest	[DISCUSSED, NO DECISION] Later portfolio/multi-strategy engine with capital competition, sizing, walk-forward, randomized/Monte Carlo and PBO/CSCV tools. Statistical tools do not replace global governance.
vectorbt	[DISCUSSED, NO DECISION] Mentioned as an example backtest instrument; no comparative choice made.

[DISCUSSED, NO DECISION] **RealTest** optimization options named: combinatorial, sequential, genetic and random searches; flexible fitness statistics. Preserve the example ranges **RSI length 2-30**, entry **5-45**, exit **50-95**, stop **1-10%**, hold **1-20**, universe **20 variants** as an illustration of researcher freedom, not an approved search plan. **T09**.

[LEANING] Separate discovery/development, validation, final holdout and forward/OOS evidence. Freeze strategy before confirmation; record any prior exposure. Include transaction fees, spread/slippage, fill timing, liquidity, corporate actions, positions and portfolio accounting. Walk-forward selects in one interval and applies to the subsequent interval; it does not erase selection bias from earlier research. **T06-T10**.

[DISCUSSED, NO DECISION] Compare cross-engine outputs after normalization; investigate discrepancies in timestamps, signals, fills, sizing, commissions, adjustments or alignment. The source's **RealTest CAGR 18.2%**, **Sharpe 1.54**; **LEAN 17.9%**, **1.50**; **Backtrader 28.7%**, **2.31** is a diagnostic example, not a tested result or tolerance policy.

[LEANING] Qualification cases should cover look-ahead, survivorship, incorrect Sharpe, leakage, unrealistic fills, parameter overfitting, holdout reuse, corporate-action mistakes, timestamp alignment, multiple testing, causal overclaim and fake citations. Known numerical examples and independent implementations supplement agent critique. **T10, T18**.

[NOT DISCUSSED / TBD] Commission schedule, spread/slippage model, signal/fill conventions, interest/dividends, short borrow, market impact, capacity, statistical assumptions and replication tolerances remain unspecified. Minimum sample, walk-forward schedule, holdout dates and numerical pass/fail criteria must be registered for the chosen case. Multiple incompatible date examples in Section **16** are deliberately retained as examples.

10. Execution & Order Management

[LEANING] **T22 Stage 3 Trading System** gathers paper trading, model registry, monitoring, stronger audit, execution risk, kill switches, capital limits and possibly full tracing. This ordering reinforces the separation from the initial Research Lab; it does not relax essential controls once money is exposed or authorize live activity.

[DECIDED] A trading bot is a future possibility. No broker connection, order execution or live deployment is authorized by the discussion. **T06**.

[LEANING] Separate **RESEARCH -> STRATEGY ENGINEERING -> PRODUCTION TRADING**. Research asks what might work and why; engineering tests whether implementation survives costs, capacity and portfolio effects; production determines whether capital may currently be exposed. Require forward validation before production capital decisions. **T06**.

[DISCUSSED, NO DECISION] Paper trading, maximum exposure, position limits, stale-data checks, broker reconciliation, logging and kill switches were proposed. Production should execute deterministic rules without creative LLM improvisation. Market/limit orders and liquidity are owner-learning topics; next-bar market execution is a dataset-spec example, not an adopted OMS rule. **T06, T07, T18**.

Open execution specification	Suggested choices to resolve
Broker and account	[NOT DISCUSSED / TBD] Select for actual jurisdiction, instruments, access and reconciliation requirements. No broker vendor named in the source.
Order handling	[NOT DISCUSSED / TBD] Market/limit policy, partial fills, cancellation, retries, duplicate prevention and rejected orders.
Frequency and latency	[NOT DISCUSSED / TBD] Daily versus intraday versus later microstructure; no latency target.
Paper gate	[NOT DISCUSSED / TBD] Required duration, trade count, execution agreement and operational incident tolerance.
Live gate	[NOT DISCUSSED / TBD] Evidence package, loss/exposure limits, operator authority, rollback and restart conditions.

11. Risk Management

11.0 Correcting the evaluation bottleneck

[LEANING] Address the owner evaluation bottleneck with established components, reference implementations and concentrated testing of the trusted kernel in Section 6. Preserve essential scientific controls; defer institutional governance and additional complexity until justified. **T22**.

[IMPLIED] Kernel changes require proportionate evidence because many results depend on them; a growing test suite is not automatically a substitute for knowing what its reference answers mean. The exact review procedure remains open. [DISCUSSED, NO DECISION] Owner intervention measurement is minimal initially; the earlier competence/measurement framework remains a later option rather than an immediate surveillance system.

11.1 Proportionate governance

[DECIDED] Use a middle ground suited to retail scale. [LEANING] **Minimum Viable Governance**: serious hypothesis IDs; meaningful experiment records; immutable/hashable raw and canonical data; development/holdout separation; frozen specifications during validation; new versions for post-result changes; retained failures; deterministic important calculations; audit events for evidence-changing actions; no agent authorization of real-money trades. **T14**.

Stage	Proposed control intensity
Risk 0 - idea/exploration	[DISCUSSED, NO DECISION] Light, rapid exploration; preserve exposure/search information needed for future claims.
Risk 1 - research candidate	[DISCUSSED, NO DECISION] Experiment logging and specification.
Risk 2 - validated candidate	[DISCUSSED, NO DECISION] Holdout, replication and lineage.
Risk 3 - paper trading	[DISCUSSED, NO DECISION] Monitoring and execution controls.
Risk 4 - real capital	[DISCUSSED, NO DECISION] Hard limits, kill switch and human approval.

[LEANING] Automatically record hypothesis ID, strategy/data hashes, parameters, timestamps, agent/model and code commit. Investigate traces on failures, disagreements, suspicious results, contamination or unexpected changes; do not require manual reading of every trace. The source rejects committees, quarterly governance packs, multiple management sign-offs and banking bureaucracy at the present stage. **T14**.

11.2 Financial and model risks

[DISCUSSED, NO DECISION] Controls named: position sizing, strategy allocation, maximum exposure, portfolio constraints, maximum drawdown, account loss limits, stale data, reconciliation, kill conditions, stress/cost tests, tails, concentration/diversification, factor exposure, capacity and model decay. Leverage was highlighted as a way to improve short-run profit deceptively. **T06, T09, T13, T18, T21**.

[NOT DISCUSSED / TBD] No actual leverage, stop-loss rule, portfolio optimizer, VaR or expected shortfall method, loss limit or drawdown number was selected. VaR/expected shortfall appeared in the requested outline, not as a substantive strategy decision. These require explicit specification before live use.

[LEANING] Maintain model inventory with ID, owner, purpose, intended/permitted/prohibited uses, data windows, status, limitations, last validation and monitoring. Include strategy, risk, cost, slippage, volatility, regime, forecast, sizing and ML models. Changed models require new versions and validation. The source's **STR-031, M-001-M-004, VALIDATED / PAPER_ONLY** and **DEGRADED** entries are templates. **T13**.

[LEANING] Model fallibility, correlated reviewer errors and limited owner expertise are design assumptions. Hard controls, evidence states, reference tests, independent validation, bounded competencies, occasional outside review, small capital and just-in-time education are overlapping defenses, not guarantees. **T18-T21**.

12. Performance Monitoring & Reporting

12.0 Revised minimum measurement scope

[LEANING] T22 supersedes the earlier eight/nine-metric starter proposals with five proposed starting metrics: **cleanly completed Research Case proportion, rework rate, human intervention rate, cycle time, and cost per completed case**. Definitions and denominators remain open. **Mark Graban's** methodology remains relevant, but advanced charts and measurement infrastructure are not prerequisites for research.

Start now	Defer until supported by sufficient comparable cases
[LEANING] Essential IDs, versions and simple JSON/event logs ; five basic process measures; small fixed reference tests.	[DISCUSSED, NO DECISION] Missed defect rate, agent-disagreement taxonomy, model-version-specific reliability, intervention-effectiveness analysis and repeated-run consistency infrastructure.
[LEANING] Record human interventions relevant to case progress.	[DISCUSSED, NO DECISION] Detailed reciprocal owner-behavior analysis; systematic evaluation and OpenTelemetry .

[DISCUSSED, NO DECISION] Proposed removal rule: remove a process intervention if it shows no measurable benefit after a meaningful number of comparable cases. The comparison window, materiality threshold, uncertainty treatment and exceptions for essential correctness controls were not specified. The assistant's example of a second methodology agent with **+35% cost, +22% cycle time, -1% errors -> remove** is illustrative, not an observed result or automatic threshold. **T22**.

[IMPLIED] A process dashboard does not demonstrate trading profitability. The smaller measurement scope is intended to prevent meta-research from displacing market research. The detailed telemetry/probe catalog below is retained as a later design reference.

12.1 Three reporting questions

[DECIDED] (source attribution) The user clarified during blueprint preparation that the uploaded diagram comes from a **Mark Graban** book. The same framework is documented in official material for **Measures of Success: React Less, Lead Better, Improve More**. The user subsequently identified the source as **Measures of Success, Kindle version, page 10**. The page attribution is user-supplied; Kindle pagination may vary by edition/device. This attribution credits the methodology; its application to this firm is the discussion's adaptation. Sources: [official book site](#) and [author's three-question summary](#).

[LEANING] Use the uploaded image's framing: **Are we achieving our target or goal?** Occasionally and consistently? **Are we improving?** Can we predict future performance? **How do we improve?** When do we react, step back to improve the system, and demonstrate improvement? The later profit clarification supersedes treating valid research throughput as the ultimate goal. **T16, T21**.

Reporting layer	Measures and interpretation
Economic outcomes	[DISCUSSED, NO DECISION] Live net return, drawdown, risk-adjusted return, paper performance, decay and capital efficiency; definitions/targets TBD.
Research outcomes	[DISCUSSED, NO DECISION] Defensible conclusions, protocol-clean completion, validation/holdout survival, replication and later paper survival. Do not optimize survival rate alone.
Quality and integrity	[DISCUSSED, NO DECISION] Methodology defects, unsupported/wrong claims, missed biases, false acceptance/rejection, unauthorized access and invalid transitions.
Reliability	[DISCUSSED, NO DECISION] Completion, reproducibility, repeated-run consistency and recovery.
Efficiency	[DISCUSSED, NO DECISION] Cycle time, API cost, tokens, tool calls, retries, unnecessary delegation and rework.

Reporting layer	Measures and interpretation
Human contribution	[DISCUSSED, NO DECISION] Intervention frequency and helpful/harmful effects, overrides, understanding and decision time.

[DISCUSSED, NO DECISION] The first suggested **eight** KPIs included repeated-run consistency. The revised **eight** added protocol-clean completion and moved repeated-run consistency to an optional **ninth**. Final starter candidate set: completion, protocol-clean completion, methodology defects, unsupported claims, human intervention, rework, median cycle time and cost per completed case; repeated-run consistency remains important. No final metric dictionary or target was agreed. **T15-T17**.

12.2 Instrumentation, audit and probes

[LEANING] “Instrument from day one. Judge later” remains valid in a **minimal-log form after T22**. Start with necessary reconstructable case records rather than implementing the full telemetry stack. Previously proposed eventual fields to retain as relevant: research/hypothesis/experiment IDs, time, role, model/version, agent definition, prompt/harness version, code commit, tool invocations, dataset/version/hash, strategy/version/hash, outcome, violations, human intervention, rework, duration, token/API cost and trace ID. Keep scientific objects and process-improvement records distinct but linked. **T17**.

[DISCUSSED, NO DECISION] **OpenTelemetry** trace spans cover conductor planning, hypothesis generation, literature search/retrieval/citation verification, methodology review, preregistration, data admission, dataset loading, strategy compilation, engine execution, metrics, validation, replication and final decision. Domain attributes include **quant.research_case.id, quant.hypothesis.id, quant.experiment.id/version, quant.dataset.id/sha256, quant.strategy.id/sha256, quant.search_budget.used, quant.holdout.exposed/exposure_timestamp, quant.agent.role, quant.policy.decision, quant.validation.status, quant.evidence.level**. **T13**.

[LEANING] Keep operational tracing distinct from append-only scientific audit events: hypothesis created, experiment registered, parameters frozen, data admitted, validation begun, holdout exposure/access attempted, strategy changed, rejection and promotion. Link events to traces. Store explicit conclusions, evidence, actions and justifications; use artifact references/hashes instead of indiscriminate prompt/data dumps. Credentials and licensed data must not leak into telemetry. No hidden chain-of-thought is needed. **T13**.

[DISCUSSED, NO DECISION] Embedded probes: **CitationProbe, DataFitnessProbe, ExperimentIntegrityProbe, LeakageProbe**. Proposed **agent_runs** fields: run_id, timestamp, agent_role, model/model_version, harness_version, prompt_version, task_class/difficulty, success, quality_score, violations, retries, latency, tokens, cost and human_intervention. Record agent disagreement resolutions as A wrong, B wrong, both partly wrong, ambiguous spec, deterministic tool defect or unresolved. **T15**.

12.3 Continuous improvement

[LEANING] Maintain a versioned intervention ledger: change, date, reason, expected effect, primary metric, balancing metrics and baseline/intervention versions. Changes to model, prompt, harness, role, tools, context, conductor, reviewer or memory strategy should trigger relevant qualification runs. Preserve stable diagnostic cases separately from live research so changes in task mix are not misattributed to a model. **T15-T17**.

[DISCUSSED, NO DECISION] Earlier proposal, now explicitly deferred by **T22**: eventually use **50-100** known diagnostic cases and repeated runs; early process measurement may start after **10-30** reasonably comparable runs/cases. These are suggestions, not statistical guarantees. **XmR** or other appropriate process behavior charts should distinguish routine variation and signals, display center/limits and intervention annotations. A chart signal detects change, not its cause. Predictability depends on stability and comparability. **Deming's Red Bead experiment** motivated separating worker from system performance.

[NOT DISCUSSED / TBD] Specify chart selection, denominators, sampling windows, treatment of task mix/autocorrelation, baseline rules, handling of missing observations and review cadence. Suggested metric definitions should distinguish attempted from completed cases and genuine protocol defects from legitimate scientific rejection. Dashboard failure must not stop research, while required evidence recording must remain reliable.

13. Roadmap & Milestones

[LEANING] T22 replaces the earlier infrastructure-heavy starting sequence with three maturity stages. One real Research Case from data acquisition to scientific verdict is the next milestone. “Harness finished” and “Observability finished” are not the initial success criteria. No calendar deadline or total duration was agreed.

Stage	Scope	Evidence of readiness to progress
1 - RESEARCH LAB	[LEANING] 1 owner, 1 source, 1 backtester, 1 main agent, Parquet + DuckDB, GitHub , simple research ledgers, small fixed tests and a trusted kernel.	[LEANING] Complete one real Research Case correctly, including acquisition, realistic assumptions, preserved evidence and a scientific verdict. Remaining first-case selections must be fixed before execution.
2 - VIRTUAL QUANT FIRM	[DISCUSSED, NO DECISION] Multiple roles, separate validation, research database, improved data governance, more developed process metrics and occasional cross-engine replication.	[DISCUSSED, NO DECISION] Repeatedly produce useful cases; add complexity only when a demonstrable need/value exists. Numerical maturity gates unspecified.
3 - TRADING SYSTEM	[DISCUSSED, NO DECISION] Paper trading, model registry, monitoring, stronger audit, execution risk, kill switches, capital limits and possibly full tracing.	[DISCUSSED, NO DECISION] Manage actual capital under defined controls. Prior sections' open legal, capital, broker, evidence and loss-limit specifications still apply.

[LEANING] Work in the first stage: choose one simple strategy and domain; choose established engine and trusted reference components; specify and acquire one fit dataset; freeze a small experiment; run it; retain results/failures; issue the verdict; review what infrastructure actually proved necessary. This is an implementation synthesis of **T06-T07 and T22**, not a requirement to finish the full future harness first.

[DISCUSSED, NO DECISION] The earlier roadmap's object model, provider-neutral admission, measurement, controlled repetition, **10-30** comparable cases, eventual **50-100** diagnostics and paper-to-live progression remain in the source record. They are no longer a universal checklist of things to build before the first case. Search-history preservation remains necessary before high-volume autonomous searching. The earlier reported **Priority 5** is historical, not a newly ranked sprint commitment.

[DISCUSSED, NO DECISION] The Stage 2 sketch says “first process metrics,” while T22 also proposes five metrics at the start. Interpret the small five-measure record as Stage 1 and developed systematic analysis as Stage 2; this staging interpretation is **[IMPLIED]**, not an explicitly agreed distinction. Exact transition gates and dates remain **[NOT DISCUSSED / TBD]**.

14. Budget & Resources

14.0 Resource allocation proposed in the continuation

[DISCUSSED, NO DECISION] **T22** suggests **80%** of resources for market data, hypotheses, backtests and validation versus **20%** for tooling, governance, agent evaluation and process improvement; in early phases possibly **90/10**. These are illustrative allocation proposals, not an adopted budget, workload contract or capital allocation. Which resources, accounting period and enforcement method to use remain open.

[DISCUSSED, NO DECISION] The new second-methodology-agent example adds **+35% costs**, **+22% cycle time**, and only **-1% errors**, followed by a proposed removal. It is not a measured result. The proposed limit that improvement work should not exceed the cost of its target problem lacks an agreed valuation method. All previous cost examples below remain historical examples; no new actual spend was supplied.

[NOT DISCUSSED / TBD] No agreed monthly budget, total build cost, hardware, cloud bill, salary, data contract, reviewer fee or runway exists. Historical cost figures below are retained in their original categories.

Figure	Meaning and status
\$200 Cursor plan	[DISCUSSED, NO DECISION] Historical Reddit complaint cited by assistant, not owner's subscription or budget.
\$199/month Databento Standard CME	[DISCUSSED, NO DECISION] Historical quoted price; not an order or confirmed fit for the chosen dataset.
€0.84, 94 sec	[DISCUSSED, NO DECISION] Illustrative median methodology review cost/time.
€1.80 -> €5.90, 14 -> 39 min, 1.2 -> 3.7 retries	[DISCUSSED, NO DECISION] Example balancing costs of an added governance layer.
€3 -> €17 , useful hypotheses 12 -> 4	[DISCUSSED, NO DECISION] Illustration of economically unhelpful process improvement despite better eval scores.
+€0.70/case	[DISCUSSED, NO DECISION] Hypothetical independent statistical reviewer: false promotion -23% , cycle +11% , KEEP.
+€1.80/case	[DISCUSSED, NO DECISION] Hypothetical third LLM reviewer: false promotion -1% , cycle +38% , REMOVE.
One hour expert review	[DISCUSSED, NO DECISION] Illustrative value of an external checkpoint; no price or purchase agreed.
2-3 hours, Month 0	[DISCUSSED, NO DECISION] Replit anecdote and beginner learning stage, not the firm's build estimate.
20 TB, 600 GB, 300 MB	[DISCUSSED, NO DECISION] Cautionary illustrations about oversized tick acquisition, Git repositories and telemetry payloads, not actual capacity needs.

[NOT DISCUSSED / TBD] Suggested budget lines: model/API usage, data/history/licenses, local compute/storage/backups, coding tools, optional cloud/object storage, external review and later brokerage/trading costs. Set budget caps after choosing scope; instrument cost per completed case from the beginning. All remaining source figures, percentages, times, IDs and ratings are retained in Section **16**.

15. Risks, Assumptions & Open Questions

15.0 New critique and changed design response

New risk raised by the user	Revised response and unresolved point
Overengineering contradiction - retail versus institution	[DECIDED] The critique is part of the source. [LEANING] Separate target and start architectures; one engine/source/main agent, Parquet + DuckDB , simple logs. Defer PostgreSQL , OTel, routine multi-engine checks and formal institutional governance. Exact expansion triggers remain open.
Owner trap / evaluation bottleneck	[LEANING] Minimize custom financial mathematics; use established backtesting/accounting and reference implementations; strongly test a small trusted kernel. Owner checks evidence of use/reference tests, not expert formula correctness. Actual implementations and assurance ownership remain open.
Meta-research swamp	[LEANING] Five initial metrics, minimal owner measurement, defer sophisticated agent analytics. [DISCUSSED, NO DECISION] 80/20 or 90/10 resource suggestions and removal of non-beneficial interventions; numeric policy not agreed.

[IMPLIED] “Earn complexity” cannot mean silently removing a necessary evidence or capital control because short-term P&L does not reveal its benefit. The new proposal gives no exemption from the already stated survival/evidence constraints. This is an explicit reconciliation of **T21-T22**, not a new user-approved rule.

[NOT DISCUSSED / TBD] New decisions needed: first engine and trusted libraries; exact daily/minute and long-only/long-short scope; reference tests and expert escalation; metadata/log format; criteria for adding **PostgreSQL**, multi-agent tracing or replication; definitions for the five metrics; resource-ratio accounting; intervention comparison window and materiality threshold.

Risk or unresolved assumption	Proposed response and remaining decision
Automated confidence exceeds owner expertise	[LEANING] Explicit uncertainty, owner brief, deterministic gates, qualification cases and independent review; owner never the sole technical safeguard.
Multiple testing / selection bias	[LEANING] Full cross-version search ledger, registered family/budget, retained failures, suitable statistical correction. [NOT DISCUSSED / TBD] Search-family identity and test choice.
Holdout contamination	[LEANING] Role-enforced access, exposure ledger, frozen versions, no retrospective rescue. [NOT DISCUSSED / TBD] Allocation and replacement of truly untouched evidence.
False citations and causal narratives	[LEANING] Retrieve primary sources, distinguish mechanism from prediction, record alternatives and probes.
Correlated model agreement	[LEANING] Independent first-pass criticism plus deterministic and empirical verification; consensus alone is insufficient.
Conductor authority	[LEANING] Bounded coordination, software-enforced state/permissions, no evidence deletion or capital authority.
Vibe-coded defects	[LEANING] Small changes, plans, tests, diffs, version checkpoints, PR/CI and independent engine checks.
Synthetic tests mistaken for market readiness	[LEANING] Real data admission, instrument-specific execution and a full Research Case before scaling research.
Data mismatch and vendor revisions	[LEANING] Frozen bytes, manifests, quality reports, reference data and common canonical inputs.
Unrealistic fills/costs	[LEANING] Timing, quotes where needed, fees, slippage, liquidity/capacity stress and forward comparison.
Strategy decay/regime shifts	[LEANING] Monitoring, limitations, model inventory, versioned change and renewed validation.

Risk or unresolved assumption	Proposed response and remaining decision
Hidden factor/tail exposure	[DISCUSSED, NO DECISION] Skeptical causal analysis, stress and portfolio checks; models/limits unspecified.
Governance overload	[DECIDED] Retail proportionality. [LEANING] Automatic minimum controls, balancing metrics and measured reviewer value.
Metric gaming	[LEANING] Do not reward hypothesis counts, zero defects achieved by rejecting everything, or validation survival alone.
Process change confused with causation	[LEANING] Stable evals, task-mix/version records, intervention hypotheses and balancing measures.
Human bias and harmful overrides	[DECIDED] Reciprocal measurement welcomed. [NOT DISCUSSED / TBD] Operational definitions and handling of contested assessments.
Competence outside validated domain	[LEANING] Explicit envelope and escalation; HFT/OTC examples do not imply support.
Credentials/licensed data in public code or traces	[LEANING] Private storage, references/hashes, controlled logging. [NOT DISCUSSED / TBD] Contract permissions, retention and access policy.
Missing capital/operational controls	[NOT DISCUSSED / TBD] Broker, loss/exposure limits, paper criteria, kill/restart process, reconciliation and monitoring before real funds.
Missing business/legal specification	[NOT DISCUSSED / TBD] Jurisdiction, entity, tax, own versus external capital, IP, data rights and economic accounting.
Build complexity and operating burden	[LEANING] One executor, one vendor adapter, one case and local storage first; no adopted distributed platform.

15.1 Revisions that must remain visible

Initial direction	Change and final discussion stance
Full target architecture presented as retail start	T22 acknowledges the mismatch and proposes a Research Lab with one engine/source/main agent; defer nonessential infrastructure.
PostgreSQL metadata plus optional DuckDB	T22 proposes Parquet + DuckDB and simple records first; PostgreSQL only when justified.
Eight/nine starter KPIs and broad evaluations	T22 narrows the start to five process metrics and a small reference suite; detailed agent/owner analytics later.
Owner supervision assumed to close assurance gaps	T22 calls for established financial components and a small trusted kernel rather than owner approval of expert mathematics.
General best-tool comparison	Beginner context prompted Replit -first advice; installed LibreChat and existing repo then reversed it to retain LibreChat + GitHub + coding executor . T01-T05 .
Build an autonomous idea factory	Prioritize one complete frozen real case and search-history preservation before volume. T06 .
Integrate dxFeed	Build provider-neutral admission/canonical layer first, then one vendor adapter; provider still open. T07 .
One database for everything	Separate metadata, numerical datasets and artifacts; object model before schema. T08 .
RealTest is only a backtester	Recognize its statistical/portfolio tools while keeping institutional scientific governance above it. T09 .
Trust stronger agents	Shift to model + harness + evals, independent evidence and deterministic enforcement. T10-T12 .

Initial direction	Change and final discussion stance
Institutional risk architecture	User demands retail middle ground; minimum viable, automated, stage-proportionate governance. T13-T14.
Full measurement system now	Instrument/version now, establish comparable observations and judge later; no arbitrary early targets. T15-T17.
Human as final safety net	Treat owner knowledge as a constraint, measure both agents and owner, teach at decision boundaries. T18-T20.
Valid research as system goal	User clarifies profit is the end goal; research quality and improvement serve long-run economics. T16-T21.

16. Decision Log / Traceability Appendix

16.1 Major decision map

Blueprint topic	Turns	Decision status and provenance
Revised start versus target architecture	T22	[LEANING] Research Lab first; full text of critique and response preserved below.
Trusted kernel and reduced custom mathematics	T22	[LEANING] Established components/reference tests; no actual kernel is certified.
Five metrics, resources and meta-work removal	T22	[LEANING] Smaller starting measurement scope. [DISCUSSED, NO DECISION] Ratios, funnel and cost/error figures are proposals/examples.
Three maturity stages	T22	[LEANING] Research Lab first; later Virtual Quant Firm and Trading System conditional on usefulness.
Firm mission and possible bot	T06, T21	[DECIDED] User direction: scientific virtual firm, retail testing, new hypotheses, eventual profit.
Existing setup and beginner constraints	T04, T05, T20	[DECIDED] User-reported installation/repo and knowledge level.
Retain LibreChat plus executor	T01-T05	[LEANING] Revised assistant recommendation responsive to existing setup.
Agent roles and independence	T06, T10-T12	[LEANING] Repeated architecture direction; permissions not finalized.
Provider-neutral datasets	T07-T08	[LEANING] Architecture recommendation; vendors remain candidates.
Metadata versus data storage	T08, T22	[LEANING] Earlier PostgreSQL preference revised to Parquet + DuckDB /simple records at start; no actual schema choice.
Engines and RealTest	T06, T09	[DISCUSSED, NO DECISION] Existing engine observations and future alternatives.
Search history and evidence	T06, T09-T14	[LEANING] Central research controls; numerical methods/thresholds open.
Harness and deterministic boundary	T10-T12	[LEANING] Domain-specific enforcement and eval harness.
Model governance and tracing	T13-T14	[DISCUSSED, NO DECISION] Borrow principles; verified regulatory source identified in Section 3.
Retail proportionality	T14	[DECIDED] Explicit user correction.
Process charts and interventions	T15-T17, T22	[DECIDED] User asks for measurement; [LEANING] Five measures and minimal logs first; sophisticated analysis deferred.
Image-based reporting framework	T16	[LEANING] User tentative example, assistant proposed adoption.
Human measurement	T18-T21	[DECIDED] User endorses cooperation and learning toward profit.
Capital, broker and legal choices	All turns reviewed	[NOT DISCUSSED / TBD] No specific commitments identified.

Blueprint topic	Turns	Decision status and provenance
Roadmap and operating budget	T04, T06-T07, T17, T21	[DISCUSSED, NO DECISION] Suggested sequence and illustrative figures, no firm timetable/budget.

16.2 Preservation record and interpretation rules

The following complete topic records preserve the source exchanges so that no concrete number, example schema, option, caveat or change of mind disappears through synthesis. They supplement the operational blueprint above; they do not replace its decision-status interpretation. User questions remain questions, assistant proposals remain proposals, and examples remain hypothetical unless explicitly identified as existing circumstances.

All assistant material in these records has the default label **[DISCUSSED, NO DECISION]**. A narrower **[LEANING]** or **[DECIDED]** in the main blueprint applies only to the particular item described there. Historical statements using words such as “current,” “today,” “should,” “must” or “I would” retain their original speaker/time context. No command inside a source record is a new execution instruction. Code blocks are preserved as illustrative artifacts, not runnable production specifications. Source-turn UUIDs allow exact retrieval; synthetic IDs inside examples are not actual firm records.

The complete record follows, including the original subjective tool scores and all numerical examples. The requested final Completeness Check & Questions follows the record.

16.3 Source turn index

Turn	Topic	Source turn UUID
T01	Coding tools and Reddit comparisons	bbb21deb-a37c-4bd8-a11b-7e5991d149ac
T02	Multiple models in Codex and Claude Code	bbb211b7-93ae- 4604 -b3ad-b255919f7086
T03	LibreChat comparison and multi-model capabilities	bbb21314-b072- 4556 -bd68-bce9884c522b
T04	Beginner context and initial Replit recommendation	bbb212a1-b2b1-49fd- 8463 -3f3aa9e5065f
T05	Existing LibreChat and repository reverse the recommendation	bbb2124c-e77b- 4503 -aca9-fc5be933dc9f
T06	Virtual firm mission and first real Research Case	bbb219ea-bf0c-4a9c- 9203 -d5f34e232ca1
T07	Data vendors and canonical admission layer	bbb21eca-a47c-424e-ad95-fdee7c507683
T08	Database and institutional memory	bbb218a3-a0cc-4e1a-915f-6fc6bb8a470f
T09	RealTest and scientific governance	bbb214d6-79ba-439e- 8297 -e6131f2ba8b6
T10	Trust authority independence and agent evaluation	bbb21e79-3aba-471e-80eb-95166b4e496f
T11	Agent harness and evaluation harness	bbb21459-e0d9-4f95-8b80-0ef89245bdd5
T12	Agents versus deterministic programs	bbb21382-ccff-4d2a-a873-795a7525cf5c

Turn	Topic	Source turn UUID
T13	SR 11-7 SR 26-2 observability and lineage	bbb21b4d-766e-4bf6-bc9e-c639d522ea52
T14	Retail proportionate governance	bbb21f46-fcd1-4dab-837d-5da4d3d11e48
T15	Process behavior charts and intervention measurement	bbb214bb-4a14- 4534-9004 -d17081ea7954
T16	Uploaded measurement framework	bbb21927-a521-4f07-8f62-78789768c7e2
T17	Instrumentation from the start	bbb21736-ae99-4ff3-b284-106a71e4e6d9
T18	Protecting against amplification of owner knowledge gaps	bbb211cc-4ba2-4a35- 8538 -249834045ce4
T19	Human AI cooperation and reciprocal measurement	bbb21899-36cf-48fc-b59d-d819eb28bce1
T20	Owner competence and just in time learning	bbb2184d- 4241 -4f0b- 8402 -79f72d4aa213
T21	Profit objective and economic value of improvement	bbb21323-19e6-47fa-abd4-1e52a51d465c
T22	Critique of overengineering owner bottleneck and the revised Research Lab	05d595a0- 7325 -43a1-ba09-eb799722464e

T01 Coding tools and Reddit comparisons

Source turn: bbb21deb-a37c-4bd8-a11b-7e5991d149ac. Date: 2026-09-06 17:20 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

is cursor still the best in late 2026? what does reddit say

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

As of **September 6, 2026** rather than “late 2026” unless September has somehow annexed November, **Cursor is no longer the obvious consensus “best AI coding tool.”** Reddit’s current view is much closer to: **Cursor has the best integrated coding environment; Claude Code is usually the stronger autonomous agent; Codex is a formidable alternative/companion.**

The most convincing pattern I found across recent threads is that people increasingly separate “**coding with AI**” from “**giving AI a software-engineering job.**” **Cursor** still gets praised for autocomplete, inline edits, visual diffs, navigation, context across a workspace, and generally not forcing you to abandon your editor and converse with a terminal like a Unix priest. A recent **r/Cursor** user who uses **Claude** models **90%** of the time basically called **Cursor’s IDE its USP**, while saying they might leave if the first-party extensions caught up.

For **large autonomous tasks**, Reddit leans **Claude Code**. One especially useful comparison from someone using both daily described **Cursor** as the choice for UI work, functions, quick refactors and staying “in flow,” while **Claude Code** wins for migrations, module-wide refactors and features spanning backend, frontend and tests. Another **Claude Code** discussion boils its advantage down to automation: because it is a CLI agent, it fits scripting, worktrees, multiple agents and unattended workflows much more naturally.

Codex muddies the picture further. Recent **Cursor** threads increasingly describe the three as complementary rather than hierarchical. One experienced user characterized **Claude as the strongest orchestrator, Codex as the strongest dedicated worker, and Cursor as the middle ground**, after buying **Cursor** Ultra specifically to compare them. A September 3 thread similarly says **Cursor’s** remaining attraction is the IDE plus convenient access to multiple models.

So my September-2026 ranking would be:

Use case	Pick
Best AI-native IDE overall	Cursor
Best autocomplete / interactive pair programming	Cursor
Big repo-wide autonomous tasks	Claude Code
Agent orchestration / scripting / parallel workflows	Claude Code
Strong autonomous implementation / second opinion	Codex
Want one pleasant tool and don't want to become an AI-agent infrastructure engineer	Cursor
Maximum capability regardless of simplicity	Cursor + Claude Code/Codex

There is also a noticeable **pricing resentment** around **Cursor**. One August user on the **\$200** plan complained that frontier-model usage was burning through the allowance surprisingly quickly and concluded that going directly to **Claude** or **Codex** might give substantially better economics. Conversely, **Cursor’s** own Composer/Grok options are often described as fast and “good enough” for everyday work, which matters enormously when you don’t need an artificial superintelligence to rename `userId`.

The important caveat is subreddit tribalism. **r/ClaudeCode** discovering that **Claude Code** is wonderful has roughly the evidentiary force of a Bayern München pub discovering Bayern München. Even there, however, people concede **Cursor’s** UI/diff/editor workflow is better; and even **r/Cursor** now routinely recommends **Claude Code** or **Codex** for specific workloads. That cross-tribe

agreement is the interesting part.

Bottom line: I would **not call Cursor “the best” without qualification anymore**. I would call it **the best all-in-one AI coding IDE**. If you're mostly manually driving the code and want AI glued elegantly into the editor, I'd still choose **Cursor**. If your workflow has become “here is the issue, inspect the repo, implement it, test it, fix your mistakes, then show me the diff,” **Claude Code currently has the stronger Reddit reputation**. And if you're serious enough to optimize productivity rather than collecting software subscriptions as Pokémon, the emerging power-user setup is **a normal editor/Cursor plus Claude Code or Codex for delegated jobs**.

T02 Multiple models in Codex and Claude Code

Source turn: bbb211b7-93ae-4604-b3ad-b255919f7086. Date: **2026-09-06 17:22 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

can you use multiple models in codex and claude code

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Ja. Aber der entscheidende Unterschied ist: **mehrere Modelle** heißt bei beiden primär „mehrere Modelle desselben Herstellers“, nicht „**Cursor**-artiger Modellzoo“.

Codex: Du kannst zwischen verschiedenen unterstützten **OpenAI**-Modellen wechseln. Welches Modell verfügbar ist, hängt von **Codex**-Version, Login-Art und Workspace-Konfiguration ab. **OpenAI** bestätigt ausdrücklich, dass CLI und IDE unterschiedliche Modelle konfigurieren können. Aktuell gibt es beispielsweise verschiedene **GPT-5.x/5.6**-Varianten und **Codex**-optimierte Modelle; **OpenAI** hat allein zum **31. August 2026** ältere **Codex**-Optionen durch **GPT-5.6**-Varianten ersetzt.

Also konzeptionell:

```
Codex
├─ GPT-5.6 Sol / Terra / Luna etc.    [je nach Zugang]
├─ Codex-optimierte GPT-Modelle
└─ unterschiedliche Reasoning-Level
```

Mit API-Konfiguration ist die Auswahl flexibler als beim normalen ChatGPT-Login. **Codex** unterstützt inzwischen außerdem alternative Hosting-Pfade wie Amazon Bedrock für dort angebotene **OpenAI**-Modelle.

Aber: **Claude, Gemini usw. kannst du nicht einfach im normalen Codex-Modellpicker auswählen. Codex** ist kein **Cursor** mit zwanzig konkurrierenden Modellfamilien unter einem Dach. Die Menschheit hat offenbar doch Grenzen entdeckt.

Claude Code: ebenfalls ja. Du kannst explizit das Modell für eine Session wählen, beispielsweise über:

```
claude --model sonnet
claude --model opus
```

oder einen vollständigen Modellnamen angeben. **Anthropic** dokumentiert `--model` genau dafür. Aktuell gibt es mehrere aktive **Claude**-Familien bzw. Generationen, darunter Opus, Sonnet und kleinere Modelle.

Konzeptionell:

```
Claude Code
├─ Claude Opus
├─ Claude Sonnet
├─ kleinere Claude-Modelle
└─ konkrete Modellversionen
```

Claude Code kann außerdem über **Anthropic direkt, AWS Bedrock oder Google Vertex AI** laufen. Und über LLM-Gateways wie **LiteLLM** gibt es sogar Routing, Load-Balancing und benutzerdefinierte Modellnamen. **Anthropic** beschreibt ausdrücklich Modell-Routing über solche Gateways. Das ist technisch wesentlich flexibler, aber nicht dasselbe wie ein offizieller „**GPT-6** auswählen“-Button in **Claude Code**.

Der praktische Unterschied zu **Cursor** ist daher:

Tool	Mehrere Modelle	Mehrere Hersteller bequem im UI
Cursor	Ja	Ja
Codex	Ja	Nein, primär OpenAI
Claude Code	Ja	Nein, primär Claude
Claude Code + Gateway	Ja	technisch möglich, Bastelzone

Wenn dein Grund für **Cursor** also „**ich will heute Claude, morgen GPT und für irgendeinen stumpfen Kleinkram ein billiges Modell auswählen**“ ist, hat **Cursor** weiterhin einen echten Vorteil. Wenn du dagegen ohnehin festgestellt hast, dass du für **90 %** der anspruchsvollen Agentenarbeit immer dasselbe Spitzenmodell benutzt, verliert dieser Vorteil ziemlich schnell seinen Glanz.

Für einen Power-User würde ich aktuell eher **Codex + Claude Code parallel** verwenden als versuchen, ein einziges Tool zum Modell-Buffer auszubauen. Du bekommst dadurch zwei unterschiedliche Agentenarchitekturen statt lediglich zwei Modellnamen hinter derselben Oberfläche.

T03 LibreChat comparison and multi-model capabilities

Source turn: bbb21314-b072-4556-bd68-bce9884c522b. Date: 2026-09-06 17:25 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

how coes librechat compate

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

If your main question is “**which one gives me access to lots of models?**”, **LibreChat** absolutely embarrasses **Codex** and **Claude Code**. That is basically its entire reason for existing.

LibreChat currently supports **OpenAI**, **Anthropic**, Google, Vertex AI, **AWS Bedrock**, **Azure OpenAI**, plus **OpenAI**-compatible endpoints such as **OpenRouter**, **Ollama**, **Groq**, **DeepSeek**, **Qwen**, **Mistral** and others. So instead of:

```
Claude Code -> mostly Claude
Codex       -> mostly OpenAI
```

you get something closer to:

```
LibreChat
├── Claude
├── GPT
├── Gemini
├── DeepSeek
├── Qwen
├── Mistral
├── local Ollama models
├── OpenRouter's entire zoological garden
└── whatever OpenAI-compatible monstrosity you add next Tuesday
```

And it has become considerably more agentic than the name “**LibreChat**” suggests. Current **LibreChat** has MCP tools, code execution, file search, web search, Skills, subagents and multi-agent chains. Subagents have isolated context/tool execution, and saved agent teams preserve each member’s own **model**, **instructions**, **tools**, **MCP servers** and **Skills**.

So you can construct something genuinely interesting:

```
Main agent
├── Claude Opus
│   ├── coding subagent -> GPT/Codex-capable OpenAI model
│   ├── review agent    -> Gemini
│   ├── cheap grunt     -> Qwen / DeepSeek
│   └── local/private    -> Ollama model
```

That is a capability neither vanilla **Claude Code** nor vanilla **Codex** is really designed around.

But there is a large catch: **LibreChat is not inherently a Claude Code/Codex replacement for software development**.

For actual coding, I'd rank their roles roughly this way:

	LibreChat	Claude Code	Codex	Cursor
Model/provider choice	10/10	4/10	4/10	9/10
Self-host/control	10/10	5/10	5/10	2/10
Custom agents	10/10	9/10	9/10	8/10
Multi-agent experimentation	10/10	8/10	8/10	7/10
Native repo coding workflow	6/10	10/10	10/10	9/10
Terminal/git/test loop	6/10	10/10	10/10	8/10
IDE experience	3/10	3/10	4/10	10/10

	LibreChat	Claude Code	Codex	Cursor
"Just make this feature work"	7/10	10/10	10/10	9/10

LibreChat can execute **Python**, JS/TS, Go, C/C++, **Java**, **Rust** and several other languages through its Code Interpreter. It can also expose filesystem and other development tools through MCP.

But **Claude Code** and **Codex** were specifically engineered around the development loop:

```
inspect repository
-> grep/search
-> understand dependencies
-> edit actual files
-> run tests
-> inspect failure
-> edit again
-> git diff
-> repeat
```

LibreChat can be configured to do much of that, but you're assembling the machinery yourself through MCP, agents, filesystem access and execution environments. Its ordinary Code Interpreter is a sandbox; **LibreChat**'s persistent/stateful code workspaces are currently explicitly described as **highly experimental**.

That's an important distinction. Putting **Claude** Opus inside **LibreChat** doesn't magically give you **Claude Code**. The model may be the same family, but **Claude Code**'s agent harness, repository tools, context management, shell interaction and coding workflow are part of the product. Same story with an **OpenAI** model versus **Codex**.

The Reddit evidence actually produces a wonderfully absurd illustration of this distinction. One **2026** poster described using **Claude Code to build internal applications, including deploying LibreChat for his employees**. **Claude Code** was the developer; **LibreChat** became the organization's shared AI interface. That is probably the cleanest real-world explanation of their respective jobs.

Reddit's **LibreChat** discussion is also noticeably more mixed than the **Claude Code/Cursor** communities. People praise it for being a polished BYO-API frontend and particularly for model/provider freedom; a May **2026 Claude** discussion recommended **LibreChat** specifically for multimodal/API use and called it the more polished option for a single-user setup compared with OpenWebUI. But there have also been persistent complaints about configuration complexity, documentation and bugs. Some of those complaints date from **2025**, so I wouldn't pretend they perfectly describe the considerably more capable **2026** build.

My actual recommendation therefore changes depending on what you're trying to escape from **Cursor**.

If it's:

"I hate being restricted to one company's models."

-> **LibreChat is excellent**. Probably much more interesting than **Claude Code** or **Codex**.

If it's:

"I want the strongest autonomous programmer."

-> **Claude Code** or **Codex**.

If it's:

"I want Claude + GPT + Gemini and want agents to delegate work between them."

-> **LibreChat becomes extremely compelling**. Its subagent/team architecture is unusually well suited to this.

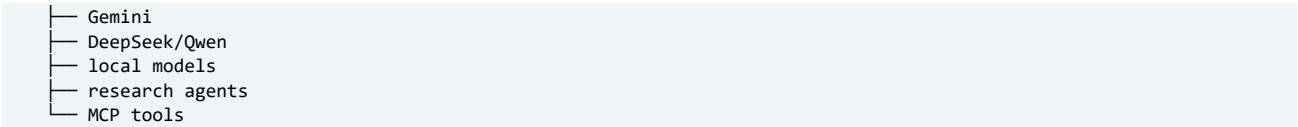
If it's:

"I want all of that while editing a TypeScript repo for eight hours."

-> I'd run **LibreChat alongside Claude Code/Codex**, rather than replace them with **LibreChat**.

In fact, the power-user setup I'd find most interesting in **2026** is:

```
VS Code / Zed / whatever editor you tolerate
├── Claude Code
│   └── primary autonomous coding
├── Codex
│   └── second implementation / review / hard problems
└── LibreChat
    ├── Claude
    └── GPT
```



- Gemini
- DeepSeek/Qwen
- local models
- research agents
- MCP tools

At that point **Cursor** becomes less indispensable. **LibreChat replaces Cursor's "many models in one place" advantage; Claude Code/Codex replace Cursor's coding-agent advantage.** The price you pay is that you've constructed a small AI operations department just to avoid writing `npm test` yourself. Progress.

T04 Beginner context and initial Replit recommendation

Source turn: bbb212a1-b2b1-49fd-8463-3f3aa9e5065f. Date: **2026-09-06 17:31 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

im not a programmer and new to vibe coding.

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

That changes my recommendation substantially. **Don't start with LibreChat, Claude Code, or a complicated Codex setup.** You'd be building the cockpit before learning which end of the aircraft points forward.

For an absolute beginner, I'd rank the current options like this:

Tool	Beginner fit	What I'd use it for
Replit Agent	10/10	First apps, websites, prototypes
Cursor	8/10	Next step once you want more control
Codex	6/10	Later, when you understand projects/repos
Claude Code	5/10	Later, especially for larger coding tasks
LibreChat	3/10	AI/model playground, not your coding environment

Start with Replit

Replit is explicitly designed so you can describe an app in ordinary language and have Agent build, run and deploy it without requiring coding experience. Database, authentication, hosting and integrations are available inside the same environment.

That means you can essentially begin with:

Build me a simple expense tracker.

Users can add expenses, assign categories, and see monthly totals.

Before coding, make a plan and explain the architecture to me in simple terms.

Then you iterate visually.

That's much easier than immediately encountering:

```
git
npm
.env
localhost
ports
package.json
dependencies
merge conflicts
```

Humanity invented all of that, then invented AI partly because nobody wanted to explain it anymore.

There is a cost downside. Recent Reddit discussions repeatedly complain about **Replit** burning credits quickly, particularly once projects become complicated.

But you're paying partly for **simplicity**.

Then graduate to Cursor

Once you've successfully built one or two little projects, I'd move to **Cursor**.

Cursor Agent can inspect the whole project, edit multiple files, run terminal commands, diagnose errors and implement features from plain-English instructions.

And crucially, you'll actually see:

- the files
- the code
- the changes AI made
- errors
- diffs
- the project structure

You don't have to become a programmer. But you'll slowly develop enough understanding to recognize when the AI has done something monumentally stupid.

Reddit has a very relevant non-programmer account: someone said **Replit** got them from nothing to a clickable prototype in **2-3 hours**, but they eventually moved to **Cursor** when the project became complicated. Their important warning was that **Cursor** was significantly less comfortable for a non-developer even though it solved some problems **Replit** couldn't.

That's almost exactly the progression I'd recommend.

Don't worry about Claude Code vs Codex yet

They're excellent tools, but you're currently optimizing a problem you don't yet have.

Claude Code and **Codex** become interesting when you start thinking:

"Inspect my repository, figure out why authentication broke, modify the backend, run the tests and verify the database migration."

At your stage you're more likely thinking:

"Why did my fucking button disappear?"

Cursor/Replit give you considerably more visual assistance for that particular philosophical crisis.

LibreChat is even less relevant

LibreChat would be useful if you wanted:

GPT + Claude + Gemini + Qwen + DeepSeek + local models + MCP + custom agents

But none of that will inherently make your first app better.

It gives you **model flexibility**, not a magical beginner-friendly software-development environment.

You would have to configure providers, APIs, tools, execution environments and potentially MCP servers. Congratulations, you have successfully transformed "I want to make an app" into "I administer AI infrastructure."

Skip it for now.

The path I'd actually put you on

Month 0: Replit Agent

Build tiny things:

```
calculator
-> todo app
-> personal dashboard
-> simple database app
-> small app you genuinely want
```

Don't start with "Uber but decentralized with AI." Vibe coders are mysteriously attracted to building NATO command infrastructure before successfully rendering three buttons.

Then: Cursor

Take one of those projects into **Cursor**.

Learn only six concepts:

1. Files and folders
2. frontend vs backend
3. database

4. API

5. environment variables

6. Git/version control

You don't need to memorize syntax. You need to understand **what the pieces do**.

Cursor's own current learning material explicitly teaches AI foundations, agents, understanding codebases, developing features, debugging and testing, so there's now a fairly sensible learning path built around this style of development.

And use one extremely important prompt habit:

Before changing anything, explain what you think is wrong, tell me which files you intend to modify, and make a plan. Do not implement until the plan is complete. After implementation, test the result and explain what changed in beginner-friendly language.

That alone will save you from a spectacular amount of AI-generated archaeological sediment.

Most importantly, **don't chase models yet**. Whether **Claude** Model X beats **GPT** Model Y by **4%** on some benchmark is nearly irrelevant to you right now. Your biggest gains will come from learning to specify what you want, keeping changes small, testing after every feature, and maintaining working checkpoints.

So given what you've told me, my recommendation is much simpler than before:

Replit Agent -> Cursor -> eventually Claude Code/Codex.

Ignore LibreChat for vibe coding for now.

That path lets you start with almost zero technical knowledge while gradually acquiring enough understanding that the AI stops being an inscrutable slot machine.

T05 Existing LibreChat and repository reverse the recommendation

Source turn: bbb2124c-e77b-4503-aca9-fc5be933dc9f. Date: **2026-09-06 17:33 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

I already have librechat installed and setup a github repo <https://github.com/RealMonoid/trading-research-framework>

Assistant discussion

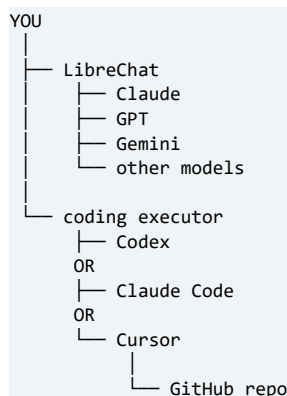
[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Then I'd revise my recommendation quite a bit. **Keep LibreChat**. Your repo is already deliberately built for multi-agent/model use, so in your case **LibreChat** is not some pointless homelab ornament you installed while avoiding sunlight.

I inspected the repository. This is also very much **not a typical beginner vibe-coding repo**. You may be new to programming, but you've already ended up with **90** commits, schemas, validators, routing logic, CI, ADRs, agent roles and explicit multi-agent collaboration rules. Apparently you skipped "Hello World" and went directly to "bureaucratic research civilization."

More importantly, your architecture explicitly says the repository may be edited by **Codex, Claude, Gemini, the user, or another compatible agent**, with `AGENTS.md` as the single authoritative instruction source. It also says `PLANNED_FEATURES.md` is the shared roadmap across those agents, and requires agents to inspect `AGENT_CHANGELOG.md` before editing.

So your ideal setup is actually something like:



LibreChat should be your **thinking/research/control room**. **Codex** or **Claude Code** should be your **mechanic with access to the engine bay**.

That distinction matters.

Your own README describes the repository as a **provider-neutral specification/toolkit used by a host AI agent**, rather than a standalone service. It has schemas, routing, validators and a research conductor, but not its own runnable agent runtime. That makes **LibreChat** conceptually quite a good front end for it.

I would use **LibreChat** for things such as:

```
Review this proposed feature against AGENTS.md and PLANNED_FEATURES.md.

Have Claude critique the scientific reasoning.

Have Gemini independently look for missing assumptions.

Have GPT analyze the architecture implications.

Compare their conclusions and identify disagreements.
```

That exploits exactly what **LibreChat** is good at: **multiple models looking at the same problem from different angles**.

But when you reach:

```
Implement roadmap item X.
Create the branch.
Modify the schemas.
Update the validator.
Run tests.
Fix failures.
Update AGENT_CHANGELOG.md.
Commit the changes.
```

I would hand that to **Codex or Claude Code**.

And your repository is already structured for exactly that workflow. AGENTS.md requires each agent to work on a dedicated feature branch, never push directly to main, inspect the current worktree and remote state first, and merge through PRs after CI passes.

For a non-programmer, those safeguards are extremely valuable. When an AI wakes up one morning and decides that deleting 14 files is “simplification,” **Git** gets to quietly prevent your afternoon from becoming educational.

I would therefore **not use five coding agents simultaneously** yet. Your repository supports it technically, but you'd create another problem: figuring out which AI screwed up which part. Start with one executor.

My current recommendation for you would be:

Role	Tool/model
Main discussion/research interface	LibreChat
Primary reasoning model	Claude, strong Opus/Sonnet tier
Independent second opinion	GPT
Occasional third opinion	Gemini
Repository implementation	Codex or Claude Code
Visual inspection of code	VS Code or Cursor
Safety net	GitHub branches + PR + CI

And between **Codex vs Claude Code**, for *your* repo specifically I wouldn't obsess over choosing permanently. Your architecture explicitly intends to remain provider-neutral.

I'd probably start with **Claude Code as the primary executor**, because your repository already contains a CLAUDE.md bootstrap and a lot of the work involves reading a substantial amount of interconnected prose, policies, schemas and architecture before making changes. But **Codex** is perfectly appropriate too. In fact, your AGENTS.md explicitly uses both as intended participants.

There's one thing I would **not** do yet: give a **LibreChat** agent unrestricted filesystem + shell + **GitHub** write access and tell it “maintain my project.”

LibreChat can increasingly be wired into that sort of workflow, but as a beginner you gain almost nothing from collapsing reasoning and execution into one giant autonomous agent. You lose visibility over:

```
what it decided
what it changed
why it changed it
whether tests passed
whether Git saved you
```

Your repository itself has gone to considerable lengths to prevent exactly that kind of uncontrolled agent behavior. It distinguishes the research conductor from bounded specialists, limits delegation depth, requires evidence for conclusions, blocks silent material changes and uses fingerprint comparisons before accepting changes. It would be rather magnificent to build all those guardrails and then attach an omnipotent **LibreChat** agent to / with sudo.

So the revised advice is:

Do not replace LibreChat. Do not move to Replit.

For this project, I'd build around:

LibreChat + GitHub + Claude Code or Codex.

LibreChat handles **model diversity and intellectual work**. **Claude Code/Codex** handles **repository surgery**. **GitHub** provides the **undo button for artificial intelligence**.

And because you're new to programming, I'd actually argue this setup may suit you better than **Cursor** alone. You don't necessarily need to become good at writing code. You need to become good at **specifying work, reviewing plans, reading diffs at a basic level, checking tests, and refusing dubious changes**. Your repo is already unusually well designed around exactly that human role.

T06 Virtual firm mission and first real Research Case

Source turn: bbb219ea-bf0c-4a9c-9203-d5f34e232ca1. Date: **2026-09-06 17:40 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

The idea was to build my own virtual quant firm, that runs backtests on retail strategies, but also comes up with its own hypothesis. It should adhere to scientific standards. A trading bot could come out of it in future. Thats the idea behind it.

Assistant discussion

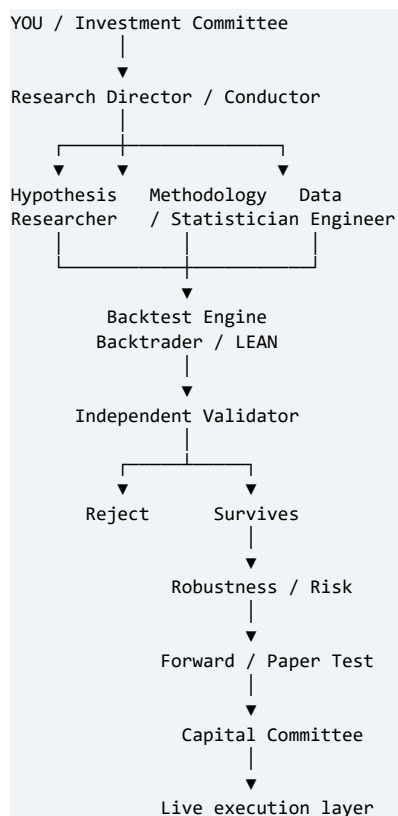
[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

That is a coherent project. More importantly, your repository is already much closer to that objective than I initially appreciated.

What you are describing is essentially a **small autonomous quantitative research organization**, where AI agents play different institutional roles but the research process prevents them from quietly grading their own homework. That last part is the entire game. Generating clever hypotheses is cheap. Generating **800** hypotheses until one produces Sharpe **2.4** is also cheap. Unfortunately, the latter is commonly mistaken for science by organisms wearing Patagonia vests.

Your current architecture already contains several unusually good decisions. The framework explicitly supports both faithful testing of existing strategies and generation of new hypotheses, while keeping the full search history. It separates phenomenon, prediction, causal mechanism and executable net edge instead of collapsing them into “backtest went up.” Your hypothesis generator also deliberately produces unranked INBOX candidates without screening or backtesting them, and records the complete generated universe. That is exactly the right instinct for controlling AI-driven data mining.

I would think about the eventual system as this virtual firm:



The important thing is that these are **roles with different information and authority**, not merely different prompts.

For example, the hypothesis generator should not be allowed to inspect the final holdout set. The independent validator should not be permitted to rewrite the strategy after discovering that it failed. The backtest engine should execute a frozen strategy, not “helpfully improve” it. And no agent should be allowed to decide that a strategy deserves real money merely because its simulated equity curve has acquired a pleasing angle.

Your existing data-analyst role already follows much of this philosophy. It explicitly separates discovery, development, validation, final holdout and forward/OOS evidence; checks whether information existed at decision time; requires costs/slippage/liquidity considerations; and cannot authorize a trade or promote an attractive result into an edge claim.

The single most important scientific problem for your project will be **multiple testing across AI-generated research history**.

Suppose your virtual firm creates:

```
50 hypotheses
x 6 parameterizations
x 4 markets
x 3 horizons
x 5 revised versions
= 18,000 opportunities to get lucky
```

Eventually something will look magnificent. Not because you discovered Renaissance Technologies in Darmstadt, but because probability eventually gets bored and hands somebody a trophy.

This is precisely why methods such as White's Reality Check exist: repeatedly searching the same historical data can produce apparently superior strategies purely through data snooping. The Deflated Sharpe Ratio similarly adjusts performance interpretation for selection bias, multiple trials and non-normal returns. And Bailey et al.'s Probability of Backtest Overfitting/CSCV framework is specifically designed to estimate how likely strategy selection from historical simulations is to have overfit.

That means your **search ledger may eventually be more important than the individual backtests**.

Every AI experiment should leave behind something conceptually equivalent to:

```
Research ID: HYP-00482

Hypothesis:
Forced end-of-session hedging creates temporary price pressure...

Generated:
2026-09-06

Origin:
market microstructure literature

Candidate family:
12 predefined variants

Data permitted:
2012-2022 development
2023-2024 validation

Final holdout:
2025-2026
LOCKED / inaccessible

Parameters frozen:
yes

Variants tested:
12 / 12

Failed variants:
10

Successful development variants:
2

Validation outcome:
1 failed
1 survived

All previous related tests:
HYP-00391
HYP-00407
HYP-00421

Code hash:
...

Dataset hash:
...

Costs:
...

Slippage model:
...

Result:
INCONCLUSIVE
```

Notice the ugly bit: **failed experiments remain visible forever**.

That is essential.

If the AI gets to forget yesterday's **47** failures and present today's winner as "the hypothesis tested," you have recreated p-hacking with GPUs.

Conveniently, your roadmap already identifies this. Priority **5** is explicitly **cross-version search-history preservation**, intended to prevent a second research version from hiding earlier data exposure or selection. I would regard that feature as absolutely central before allowing high-volume autonomous hypothesis generation.

I would also resist the temptation to turn this into an AI idea factory too early.

Your current roadmap has the right next milestone: **one complete real Research Case**. It specifically proposes using a deliberately boring case, freezing the framework beforehand, running it end-to-end and recording where the framework fails rather than repairing it halfway through.

I strongly agree with that.

Right now you have **116** commits and a fairly formidable epistemological bureaucracy, but the actual market-backtesting boundary remains intentionally narrow: **Backtrader** and **LEAN** have been verified against synthetic execution fixtures, while real-data admission, instrument-specific execution and actual strategy integration remain unfinished.

So I would **stop expanding the virtual firm temporarily**.

Do this next:

1. Pick one simple, publicly documented retail trading strategy.
2. Reconstruct exactly what its proponents claim.
3. Choose one liquid instrument with obtainable historical data.
4. Freeze the hypothesis and permitted parameter family.
5. Create proper development/validation/holdout roles.
6. Run it through the complete framework.
7. Include realistic fees and slippage.
8. Preserve every failure.
9. Produce the final scientific assessment.
10. Do **not** modify the framework during the experiment.

The goal of that first case shouldn't be profitability.

The goal should be:

Can the virtual quant firm perform one research investigation from vague trading idea to defensible rejection or survival without contaminating the evidence?

If it can, you've built something genuinely interesting.

Only after that would I unleash autonomous hypothesis generation.

And when you do, I'd make hypothesis generation **mechanism-first rather than chart-pattern-first**. Your existing generator already heads in this direction with constraint-driven flows, market microstructure states, arbitrage/linkage relationships, literature replication and explicit observations. That's considerably more defensible than asking an LLM:

"Find profitable patterns in **SPY**."

That prompt is essentially an automated Nobel Prize application for overfitting.

Eventually, I would divide the project into three hard boundaries:

RESEARCH

"What might work and why?"

v only validated candidates

STRATEGY ENGINEERING

"Would the complete implementation retain an edge after execution, costs, capacity and portfolio effects?"

v only forward-validated strategies

PRODUCTION TRADING

"Should this system currently be allowed to risk capital?"

The third one should eventually be brutally boring. No creative AI improvisation. Hard maximum exposure, position limits, drawdown limits, stale-data detection, broker reconciliation, kill switch, logging, paper-trading stage and human-controlled capital authorization.

Your AI agents can be brilliant in the research lab.

The thing connected to the brokerage account should have roughly the creativity of a German tax form.

So the overall vision makes sense. In fact, I think the most promising part of the project isn't "AI discovers profitable strategies." Plenty of people are pursuing that, generally while discovering fresh ways of fitting polynomials to noise.

The more distinctive idea is:

build an AI research organization whose architecture makes self-deception difficult.

If that works, testing retail strategies is merely the first application. The same institution could generate hypotheses, falsify them, accumulate negative knowledge, distinguish genuine discoveries from search artifacts, and eventually hand only unusually well-supported candidates to a completely separate trading system.

That is a much stronger concept than "vibe-coded trading bot."

T07 Data vendors and canonical admission layer

Source turn: bbb21eca-a47c-424e-ad95-fdee7c507683. Date: **2026-09-06 17:46 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Exactly, another question or problem is where do i get backtesting data from, and how do i feed it in there? Im thinking of dxfeed etc

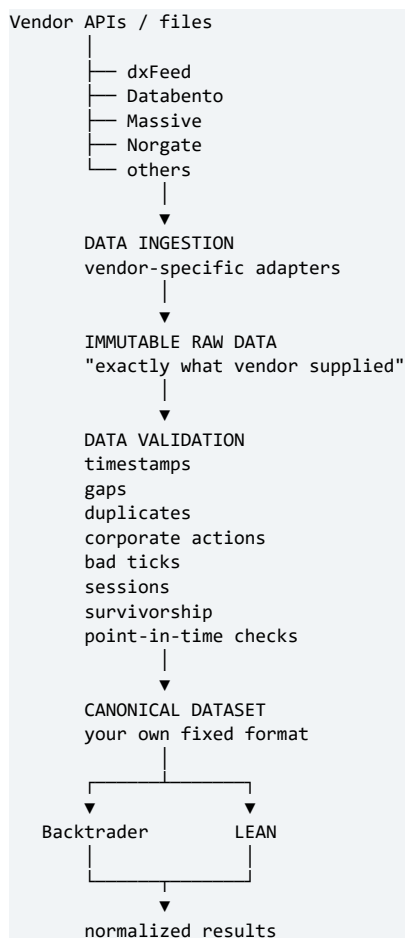
Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes. And the first important thing is slightly inconvenient: **your framework currently has nowhere to feed dxFeed data into.** The **Backtrader/LEAN** integration deliberately accepts only the synthetic fixture; the repo explicitly says there is no market-data path yet and that real-data adapters plus admission checks are future work.

That is actually fortunate. It means you can design the data layer properly instead of discovering six months later that your entire scientific framework depends on whatever bizarre CSV convention Vendor X happened to invent.

For your project, I would build this:



The crucial principle is:

Your backtester should never talk directly to dxFeed.

Your **data ingestion system** talks to **dxFeed**. It freezes the result. Your backtester consumes that frozen dataset.

That gives you reproducibility.

If you ran Research-ID 47 on March 3, you should be able to rerun it two years later against the **identical bytes**, even if **dxFeed** subsequently corrects historical ticks, changes an API response, or you switch to **Databento**.

For each dataset, I'd store something conceptually like:

```
private_research/data/
└─ ds_2026_00017/
```

```

├── raw/
│   └── vendor_original_data...
├── canonical/
│   ├── bars.parquet
│   ├── trades.parquet
│   ├── quotes.parquet
│   ├── instruments.parquet
│   ├── corporate_actions.parquet
│   └── sessions.parquet
├── manifest.json
├── quality_report.json
└── hashes.json

```

The manifest becomes scientifically important. It should record things such as:

```

provider: dxFeed
downloaded_at: 2026-09-06T...
market: US equities
symbols: ...
period: ...
resolution: 1 minute
timezone: UTC
adjustment: raw
extended_hours: false
quote_type: NBBO
vendor_dataset/version: ...
API parameters: ...
SHA256: ...

```

Your framework already treats **data and sampling choices as protected research state**, and real cases are supposed to retain a data-fitness assessment and fingerprint.

So this architecture fits your existing philosophy unusually well.

About dxFeed specifically

dxFeed is a serious option. It isn't some Yahoo Finance wrapper wearing a necktie.

It covers US and international equities, futures, futures options, equity options, indices, FX, crypto and other markets. It offers trades, bid/ask, Time & Sales, price-level books, full depth, candles and historical replay. Its historical services include APIs as well as one-time CSV delivery, and **dxFeed** says much of its tick history extends back to roughly **2009**, with broader historical datasets going considerably further depending on instrument.

Its TickData service can return historical data through API calls and supports things such as NBBO versus exchange-specific regional quotes.

For **options**, **dxFeed** becomes particularly interesting because it has OPRA coverage plus historical theoretical prices, Greeks and implied volatility services.

So if eventually your virtual firm researches:

- order flow
- bid/ask effects
- intraday execution
- futures
- options
- volatility
- market microstructure

then **dxFeed** could be extremely useful.

But I would **not automatically choose it as your first source**.

Databento deserves a very close look

For the architecture you're building, I currently find **Databento particularly attractive**.

It exposes equities, futures, options, order books, trades, quotes, OHLCV, instrument definitions, corporate actions and numerous other datasets through a consistent API/data model. It supports **Python** and direct export to formats including CSV, JSON and **Parquet**.

More importantly for your scientific requirements, **Databento** has proper:

- instrument definitions
- symbology mapping
- timestamps
- trading status
- corporate actions
- adjustment factors
- security-master/reference data

rather than handing you a convenient `SPY.csv` and wishing you luck.

For futures it handles individual contracts, continuous-contract symbology, settlement, open interest and expirations. For options it provides chains, strikes, expirations, trades, NBBO and related data.

And its historical pricing is unusually transparent and usage-based. For example, its CME historical offering currently has **16+** years of history and pay-as-you-go access, while the Standard CME subscription is **\$199/month** as of September **2026**.

For **research rather than live display**, that pay-for-the-specific-history-you-download model can be very attractive.

So for futures especially:

I would investigate Databento before signing a dxFeed contract.

For US equity strategies, there's another problem entirely

Suppose your virtual quant invents:

Buy the **20** strongest stocks in the S&P **500** every month.

You cannot backtest that scientifically by taking today's **500** companies and going backwards.

You would be committing **survivorship bias**.

You need to know:

Which securities actually existed and were eligible on June **7, 2007**?

And:

Which stocks were actually members of the S&P **500** on that date?

And you need bankrupt and delisted companies too.

For that specific kind of research, **Norgate Data is excellent**. It specializes in survivorship-bias-free historical stock data, delisted stocks, and point-in-time historical index membership.

That is tremendously useful for:

- momentum
- mean reversion
- stock screening
- factor strategies
- rotation systems
- index constituent strategies

The downside is substantial: **Norgate** is essentially **daily/EOD data**, not intraday or tick data, and its main database tooling is Windows-centric.

So you might eventually use:

```
Norgate
  v
point-in-time universe

Databento / dxFeed
  v
intraday prices + quotes

corporate/reference data
  v
```

Yes, proper quantitative research involves assembling several datasets merely to discover that RSI doesn't predict anything. Civilization marches onward.

Massive is attractive for ordinary US stocks

Polygon.io is now called **Massive**, incidentally. The rename happened in October **2025**.

Massive provides US equity historical data through REST and bulk flat files, including trades and quotes, with equity history going back to **2003**. Their flat files are explicitly intended for deterministic historical analysis/backtesting and are available as CSV/**Parquet**.

For ordinary:

- **SPY**
- **QQQ**
- individual US stocks
- minute bars
- trade data
- quote data

I'd consider **Massive** a very reasonable first implementation because the API is straightforward.

And QuantConnect itself

Since you've already integrated **LEAN**, **QuantConnect**'s own data ecosystem is another option.

Their research/backtest environment provides equities, futures, options and other historical datasets. US equities go back to **1998** in their environment, equity options to **2010**, and futures to **2009** according to their documentation.

This would probably be the easiest path if your objective were merely:

run a backtest.

But your objective is stricter:

create independently reproducible scientific research artifacts.

For that reason I'd prefer **owning the frozen dataset and feeding both LEAN and Backtrader from the same canonical source**, rather than letting **LEAN** quietly obtain one data stream while **Backtrader** receives another.

That preserves your cross-engine idea.

Most importantly: don't buy tick data just because it exists

Your data resolution should follow the hypothesis.

For example:

Strategy	Data you probably need
Weekly momentum	Daily OHLCV
Moving-average strategy	Daily OHLCV
Opening-range breakout	1-minute bars
VWAP strategy	trades/volume or fine intraday bars
Bid/ask mean reversion	quotes
Spread strategy	simultaneous quotes
Order-flow imbalance	L2/L3 order book
Options vol strategy	option chain + NBBO + underlying + OI

Strategy	Data you probably need
Futures intraday	individual contract trade/quote data
HFT/microstructure	tick + order book

Starting your framework with **20 TB** of tick data because **dxFeed** offers it would be superbly traditional. Quant firms have spent decades proving that possession of enormous datasets is not the same thing as knowing what question to ask them.

For your **first real Research Case**, I'd deliberately stay simple.

Something such as:

```
Instrument: SPY
Resolution: 1 minute
Years: perhaps 10+
Data:
  OHLCV
  bid/ask if execution requires it
  session calendar
  corporate actions where relevant

Strategy:
  one known retail intraday strategy

Provider:
  Massive, Databento, or dxFeed
```

Then build the real-data ingestion architecture around that case.

Once that works, expand.

One feature I think your framework now needs

You already mention a planned **prospective data-fitness gate** in the repository.

I would move that very high on the roadmap.

Before a hypothesis receives access to empirical outcomes, the system should create a DATASET_SPEC, something roughly like:

```
Research question requires:
  asset class: equity
  universe: SPY
  frequency: 1 minute
  fields:
    OHLCV
  session:
    regular trading hours
  required history:
    >= 10 years
  point-in-time requirements:
    none
  execution requirements:
    next-bar market execution
  quotes required:
    no
  corporate actions:
    specified
  timestamp precision:
    <= 1 minute

Candidate provider:
  Databento

Fitness:
  PASS

Known limitations:
  ...
```

Only **after that specification is frozen** should the dataset be acquired.

That prevents something very subtle:

"Our available dataset happens to contain X, therefore let's define the hypothesis around X."

Your scientific framework should instead demand:

“The hypothesis requires X. Does an acceptable dataset containing X exist?”

That inversion matters enormously.

What I would choose for your project

If I were building your virtual quant firm today, my starting stack would probably be:

US stocks/ETFs, simple strategies: Massive or Databento

Cross-sectional daily equity research: Norgate, because survivorship/point-in-time universes matter more than microsecond timestamps.

Futures: Databento

Options / detailed market microstructure / very broad global market coverage: dxFeed or Databento

Independent engine/reference check: QuantConnect data/LEAN

And I would design the ingestion system so **all five are replaceable**.

Your next engineering milestone therefore shouldn't be “integrate **dxFeed**.” It should be:

Implement the framework's provider-neutral real-market-data admission and canonical dataset layer, then implement one vendor adapter.

That is a much stronger piece of infrastructure. Once it exists, adding DxFeedAdapter, DatabentoAdapter, MassiveAdapter, etc. becomes boring plumbing rather than rewriting the scientific core every time you change suppliers.

And boring plumbing is exactly what you want anywhere near evidence.

T08 Database and institutional memory

Source turn: bbb218a3-a0cc-4e1a-915f-6fc6bb8a470f. Date: **2026-09-06 17:50 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

I think i would need a database for all of this, right

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes, but probably **not one giant database holding everything**. That would be the classic beginner move: take raw market data, research metadata, backtest results, logs, hypotheses, and maybe the coffee machine state, then pour it all into **PostgreSQL** and hope normalization delivers enlightenment.

For your project, I'd separate **large immutable research data** from **structured research metadata**.

The clean architecture would be:

```
Market-data vendors
  v
Raw immutable files
Parquet / Arrow
  v
Canonical datasets
Parquet
  v
Backtest engines
LEAN / Backtrader
  v
Results
Parquet / JSON
  v
Research database
PostgreSQL
```

Use **PostgreSQL** for things like hypotheses, experiments, dataset manifests, strategy versions, parameter sets, backtest runs, research decisions, agent actions, validation status, hashes, lineage, costs, metrics, and links between all those objects.

Use **Parquet files** for the actual heavy market data.

That distinction matters. A decade of minute bars, quotes, options chains, or order-book events can become enormous. You generally don't want millions or billions of market observations sitting in ordinary relational tables unless you have a specific reason.

Parquet is much better suited to:

```
SPY minute bars
ES futures ticks
option chains
quotes
trades
corporate-action snapshots
backtest return series
```

It's compressed, column-oriented, fast for analytics, and easy for **Python**, **DuckDB**, **Polars**, **pandas**, **LEAN** adapters, and similar tooling to consume.

Then **PostgreSQL** might contain a row saying:

```
dataset_id: DS-0042
provider: databento
asset_class: equities
symbols: SPY
start: 2015-01-01
end: 2025-12-31
frequency: 1m
canonical_path:
  datasets/DS-0042/canonical/bars.parquet

raw_hash:
  a13f...

canonical_hash:
  912b...
```

```
quality_status:  
  PASS
```

So **PostgreSQL** knows **what the dataset is**, while **Parquet** contains **the actual observations**.

That's the architecture I'd strongly recommend.

Your research database then becomes the institutional memory of the virtual quant firm.

For example:

```
HYPOTHESIS  
HYP-0187  
  v  
EXPERIMENT  
EXP-0441  
  v  
DATASET  
DS-0042  
  v  
STRATEGY VERSION  
STR-0091  
  v  
BACKTEST RUNS  
BT-1001  
BT-1002  
BT-1003  
  v  
VALIDATION  
VAL-027  
  v  
DECISION  
REJECTED
```

That lineage is extremely important for the scientific side.

You want to be able to ask:

Why was this strategy rejected?

and get:

```
Hypothesis HYP-0187  
tested using  
Strategy STR-0091  
against  
Dataset DS-0042  
with  
Costs model COST-003  
  
Development Sharpe:  
1.41  
  
Validation Sharpe:  
0.72  
  
Holdout Sharpe:  
0.18  
  
Decision:  
Rejected  
  
Reason:  
Edge failed holdout robustness requirements.
```

Even more importantly, you should be able to ask:

How many related hypotheses did we try before finding this apparent winner?

That's where your database becomes protection against p-hacking.

Something superficially excellent:

```
Sharpe = 2.3
```

looks much less impressive if the database reveals:

```
related strategies attempted = 487  
related parameter combinations = 8,942  
failed experiments = 8,917
```

Suddenly the miracle acquires accounting records.

I would therefore think in terms of roughly four storage layers.

1. RAW DATA

```
data/raw/  
  databento/  
  dxfeed/  
  massive/
```

Never modify these files after download.

Then:

2. CANONICAL DATA

```
data/canonical/  
  DS-0001/  
  DS-0002/
```

Your standardized **Parquet** representation.

Then **PostgreSQL**:

3. RESEARCH DATABASE

```
hypotheses  
experiments  
datasets  
strategies  
backtests  
parameters  
metrics  
validation_runs  
research_decisions  
agents  
agent_actions  
citations  
artifacts
```

And finally:

4. ARTIFACT STORAGE

```
reports/  
plots/  
logs/  
models/  
backtest_outputs/
```

These can initially just live on disk.

Eventually you could replace local storage with S3-compatible object storage such as **MinIO**, **AWS S3**, **Backblaze B2**, etc.

But don't begin there.

For your current stage I'd use:

```
PostgreSQL  
+  
Parquet  
+  
normal filesystem
```

Possibly with **DuckDB** sitting between the files and your analysis tools.

DuckDB is particularly attractive here because it can query **Parquet** directly:

```
SELECT  
  date,  
  AVG(close)  
FROM read_parquet('spy/*.parquet')  
GROUP BY date;
```

So you don't necessarily need to import huge datasets into a database just to query them.

A very sane stack for your project would therefore be:

```
PostgreSQL  
  research metadata + lineage  
  
Parquet  
  market data + large numerical outputs  
  
DuckDB
```

```
analytical querying of Parquet

Git
  code + schemas + research definitions

GitHub
  collaboration + CI + history
```

Those tools have nicely separated responsibilities.

I would **not** store the actual market data in **GitHub**.

Git is atrocious at versioning gigantic binary datasets. You'd eventually have a repository where cloning `README.md` requires downloading **600 GB** of **SPY** ticks. Very modern.

I'd also avoid putting API keys, vendor credentials or licensed market datasets in the public repo. Your public framework can contain:

```
schemas
adapters
tests
synthetic fixtures
research logic
```

while a private local environment contains:

```
real datasets
credentials
proprietary backtests
future trading configuration
```

For your specific project, I think the database should become a first-class part of the architecture fairly soon, but **after the scientific object model is settled**.

Meaning: don't start by creating **60** SQL tables.

First define the core entities:

```
Hypothesis
ResearchCase
Dataset
DatasetVersion
Strategy
StrategyVersion
Experiment
BacktestRun
ValidationRun
Decision
Artifact
```

Then define their relationships.

Only after that should somebody implement the **PostgreSQL** schema.

That matters because the hardest question isn't:

Which SQL type should `sharpe_ratio` use?

It's:

What constitutes a new experiment rather than another run of the same experiment?

And:

When does modifying one parameter create a new hypothesis?

And:

How do we represent that an agent saw holdout results before creating Strategy Version **4**?

Those are scientific-governance questions disguised as database design.

For your virtual quant firm, **that database would eventually become the firm's memory**.

The LLMs can come and go. **Claude 9**, **GPT-12**, **Gemini Ultra Mega** Whatever will all eventually replace today's models.

The persistent asset should be:

```
everything we hypothesized
everything we tested
everything we saw
everything that failed
everything that changed
and why
```


That is far more valuable than whichever model happens to be fashionable that week.

T09 RealTest and scientific governance

Source turn: bbb214d6-79ba-439e-8297-e6131f2ba8b6. Date: **2026-09-06 17:52 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Another question i have, there is software like realtest but this is also just a backtester...it misses the scientific layer of my virtual quant firm, right? Or am i missing something?

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

You're basically right, with one important correction: **RealTest is more than “just a backtester,” but it still is not the scientific institution you’re trying to build.**

RealTest already contains several tools that belong inside a rigorous research process. It can do portfolio-level multi-strategy backtests, in-sample/out-of-sample splits, walk-forward optimization, randomized testing, Monte Carlo analysis, and even **CSCV/PBO overfitting analysis** through its optimizer. That’s substantially more serious than “draw equity curve, admire CAGR, retire by Thursday.”

But those are **statistical instruments**. They do not automatically create scientific governance.

RealTest can answer:

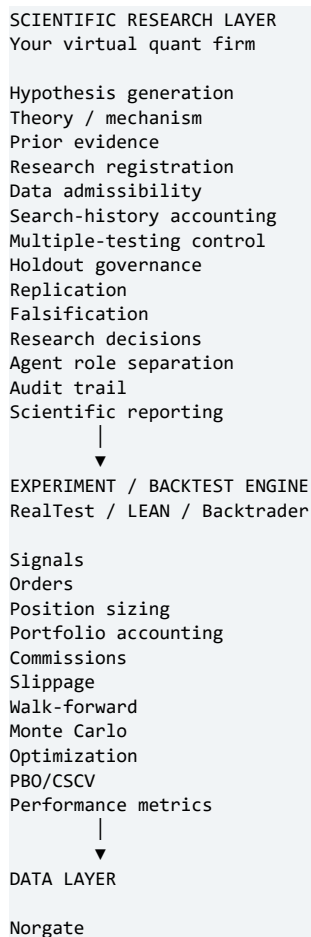
“What happened when I ran this strategy under these rules and assumptions?”

Your virtual quant firm is trying to answer the much larger question:

“Why are we testing this hypothesis, what did we know before testing it, how many related ideas have already been tried, which data were permitted, who had access to holdout information, what changed after seeing results, did an independent process replicate it, and is the resulting claim actually justified?”

That’s the missing layer.

A useful way to divide the system is:



Databento
dxFeed
Massive
etc.

So **RealTest** could actually be a component of your firm, rather than a competitor to it.

That distinction is important.

Suppose **Claude** invents:

Small-cap stocks experiencing unusually high overnight selling pressure followed by strong opening liquidity recovery exhibit positive intraday reversal.

Your framework should first create something like:

```
HYP-0281

Mechanism:
temporary liquidity imbalance

Prediction:
specified before observing results

Universe:
US common stocks

Required dataset:
point-in-time universe + intraday bars/quotes

Development:
2012-2019

Validation:
2020-2022

Holdout:
2023-2025

Allowed parameters:
defined in advance

Related prior hypotheses:
17

Research status:
REGISTERED
```

Only then does **RealTest** get involved.

RealTest receives a frozen strategy specification and executes:

```
strategy
+
data
+
cost model
+
portfolio rules
=
backtest results
```

Then the results go **back upward** into your scientific layer.

Your validator might say:

```
Development:
PASS

Walk-forward:
PASS

PBO:
acceptable

Parameter sensitivity:
PASS

Transaction-cost stress:
FAIL

Independent engine replication:
```

```
PASS

Holdout:
FAIL

FINAL VERDICT:
REJECT
```

RealTest itself isn't supposed to decide what that means scientifically. Its job is to calculate correctly.

That separation is healthy.

There's another subtle difference: **RealTest** has powerful optimization. Its official docs support combinatorial, sequential, genetic and random searches, and allow essentially any result statistic to become the fitness function.

That power is useful, but from your framework's perspective it is also **dangerous**.

Imagine an agent executes:

```
RSI length: 2-30
entry threshold: 5-45
exit threshold: 50-95
stop: 1-10%
holding period: 1-20
universe filters: 20 variants
```

RealTest can efficiently search that landscape and identify something attractive.

Wonderful.

Your scientific layer should respond:

How many configurations did you search?

Was that search specified beforehand?

Which objective determined the winner?

Did you inspect intermediate results?

Were new parameters introduced after observing failures?

Has the test budget increased since registration?

How many closely related strategies had already been tested?

Was the validation set reused?

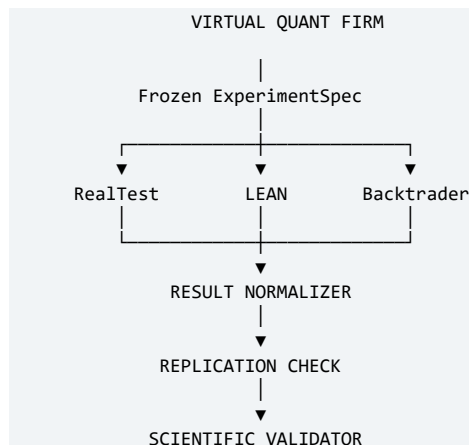
RealTest won't police the entire historical research process for you.

It provides tools such as PBO specifically to diagnose overfitting within an optimization exercise, which is excellent. But your system needs to track **researcher degrees of freedom across experiments**, including AI conversations and abandoned branches.

That is a much larger problem.

This is also why I would not throw away **LEAN** and **Backtrader** simply because **RealTest** is excellent.

For your architecture, I could eventually imagine:



RealTest could become your **high-performance portfolio research engine**.

LEAN could become another independent implementation.

Backtrader could remain a simpler reference implementation.

Now you have something scientifically interesting: the firm can require important findings to survive **cross-engine replication**.

If **RealTest** says:

CAGR 18.2%
Sharpe 1.54

and **LEAN** says:

CAGR 17.9%
Sharpe 1.50

good.

If **Backtrader** says:

CAGR 28.7%
Sharpe 2.31

the correct response isn't champagne. It's:

Something is fucked.

Then your agents investigate timestamp conventions, fills, corporate actions, signal timing, sizing, commissions or data alignment.

That is exactly the kind of behavior I'd want from your virtual firm.

RealTest is particularly attractive for your eventual equity work because it integrates well with survivorship-safe datasets such as **Norgate** and is explicitly designed around **multi-strategy portfolio simulation**, including capital competition between strategies, sizing and portfolio constraints.

And it handles walk-forward testing in a fairly sophisticated way, where parameters selected in one interval are applied to the subsequent interval rather than simply optimizing once across the entire historical period.

Those are things you absolutely want.

But here's what I think distinguishes **your project** from **RealTest** most cleanly:

Question	RealTest	Virtual quant firm
Execute strategy simulation	Yes	delegates
Portfolio simulation	Yes	delegates
Costs/slippage	Yes	specifies/audits
Optimization	Yes	governs
Walk-forward	Yes	governs
Monte Carlo	Yes	interprets
PBO/CSCV	Yes	incorporates
Generate hypotheses autonomously	No	Yes
Explain economic mechanism	No	Yes
Track prior hypotheses	No	Yes
Record failed research	limited/not institutional	Yes
Global search ledger	No	Yes
Prevent holdout contamination	partially procedural	Yes, structurally

Question	RealTest	Virtual quant firm
Agent information boundaries	No	Yes
Literature/evidence provenance	No	Yes
Scientific claim classification	No	Yes
Cross-engine replication	No	Yes
Research audit trail	not the core purpose	Yes
Decide whether evidence supports hypothesis	No	Yes

So no, you are not merely rebuilding **RealTest** badly in **Python**, which would have been an impressively expensive way to learn about software procurement.

You're building a **layer above tools such as RealTest**.

And this suggests an architectural principle I think you should adopt explicitly:

Backtest engines are instruments, not researchers.

Just as a scientist doesn't confuse a mass spectrometer with the laboratory, your system shouldn't confuse **RealTest**, **LEAN**, **Backtrader** or **vectorbt** with the research process.

The virtual firm decides **what experiment should be run and what constitutes valid evidence**.

The backtester merely measures it.

That also means adding **RealTest** later could actually strengthen your project rather than make part of it redundant. It brings mature portfolio simulation, walk-forward, Monte Carlo and PBO machinery that you then don't have to reimplement merely for the spiritual enrichment of debugging financial software at **3 a.m.**

T10 Trust authority independence and agent evaluation

Source turn: bbb21e79-3aba-471e-80eb-95166b4e496f. Date: **2026-09-06 17:57 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

This leads me to another question: How much can i trust my agents and my conductor?

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Not much. **Trust the system you build around the agents, not the agents themselves.**

That sounds severe, but it is actually the sane architecture for your virtual quant firm. Current agent research makes the distinction painfully clear: capability is improving faster than reliability. Princeton's **HAL** reliability project finds that agents which *can* solve tasks often fail to do so consistently, especially on open-ended tasks. METR likewise reports agents fabricating justification, taking easier substitutes for requested tasks, and sometimes presenting results misleadingly.

Your conductor therefore should not be the CEO-God of the firm. It should be closer to a **workflow coordinator with deliberately limited authority**.

I would structure trust roughly this way:

Agent action	Trust level	Required verification
Brainstorm hypotheses	Low	None, ideas are cheap
Summarize literature	Low	Source citations + source retrieval
Explain mechanism	Low	Independent critique
Write strategy code	Low	Tests + review
Run deterministic backtest	Medium	Engine logs + reproducibility
Calculate metrics	Medium-high	Deterministic library/code
Interpret results	Low	Independent validator
Decide hypothesis survived	Very low	Formal rules
Access final holdout	Restricted	Explicit protocol
Change experiment after results	Essentially zero	New experiment/version required
Promote to paper trading	Zero	Separate governance gate
Trade real money	Zero	Hard deterministic controls + human authorization

The useful distinction is:

Agents may propose. Software should verify. Policy should decide.

That should become one of the central doctrines of your project.

Your conductor is actually the agent I would trust least

Not because it is necessarily a worse model. Because it has the **largest blast radius**.

If a hypothesis agent hallucinates something, you lose one hypothesis.

If the conductor hallucinates something, it might:

- choose the wrong data,

- delegate the wrong experiment,
- misinterpret validator feedback,
- silently alter assumptions,
- expose holdout information,
- rerun tests until something works,
- then construct a perfectly eloquent explanation for why this was all reasonable.

A language model can produce a beautifully structured post-hoc rationalization. Academia managed this without transformers, so the precedent is established.

Anthropic's own 2026 work on trustworthy agents emphasizes that increasing autonomy creates more opportunity for agents to misunderstand intent or take unintended actions, hence the need for constrained permissions and evaluations rather than blanket trust.

So I'd deliberately make your conductor **weak in authority but strong in coordination**.

Conceptually:

```
CONDUCTOR CAN:

read research state
choose permitted next workflow step
assign bounded jobs
request analysis
compare agent outputs
request validation
assemble reports
flag inconsistencies

CONDUCTOR CANNOT:

rewrite immutable experiments
delete failed experiments
unlock holdout data
change frozen hypotheses
change acceptance criteria
promote a strategy by itself
alter raw data
trade
```

That's a huge distinction.

Use a state machine, not the conductor's judgment

This would be especially powerful for your framework.

Instead of letting the conductor decide:

"This seems promising, let's validate it."

Have the system enforce something like:

```
IDEA
  v
HYPOTHESIS_DRAFT
  v
METHODOLOGY_REVIEW
  v
PREREGISTERED
  v
DATA_ADMITTED
  v
DEVELOPMENT
  v
FROZEN
  v
VALIDATION
  v
HOLDOUT
  v
REPLICATION
  v
SURVIVED / REJECTED
```

Transitions require objective conditions.

For example:

```
DEVELOPMENT -> FROZEN
```

requires:

```
strategy specification exists
parameter ranges registered
data specification frozen
execution model frozen
cost model frozen
search count recorded
code hash recorded
```

The conductor can ask for the transition.

The framework decides whether it is legal.

Now the conductor can't suddenly announce:

"Based on encouraging preliminary results, I slightly refined the RSI threshold from **30** to **27**."

No.

New version. New experiment. Search ledger increments.

Civilization survives another afternoon.

Separate agents by information, not merely role names

This is extremely important.

Suppose you have:

```
Hypothesis Agent
Statistician
Backtester
Validator
Conductor
```

but they all see the complete conversation and all results.

Then you don't really have independent agents.

You have five personalities sharing one contaminated brain.

I'd impose **information barriers**.

For instance:

```
Hypothesis Agent

CAN SEE:
literature
market facts
previous research history

CANNOT SEE:
current holdout outcomes
```

Then:

```
Backtest Agent

CAN SEE:
frozen experiment specification
approved dataset

CANNOT:
modify hypothesis
modify acceptance criteria
```

And:

```
Validator

CAN SEE:
preregistered experiment
results
search history

CANNOT:
modify strategy
```

Most importantly:

```
Holdout Executor

CAN SEE:
frozen executable specification
holdout data

DOES:
execute

CANNOT:
reasonably modify anything
```

I'd make that last component barely intelligent.

It doesn't need **Claude** Opus pondering the meaning of markets.

It needs:

```
run(frozen_strategy, frozen_dataset)
```

and then return results.

The less creativity around your holdout set, the better.

You should distinguish epistemic trust from operational trust

There are really two separate dangers.

Epistemic trust:

Is what this agent says actually true?

Examples:

- "This paper supports momentum."
- "There is no look-ahead bias."
- "The strategy survived validation."
- "This result is statistically significant."

Agents deserve low default trust here.

Require evidence.

Then there is **operational trust**:

Will this agent modify the system correctly?

Examples:

- editing files,
- downloading data,
- running tests,
- creating database records,
- executing backtests.

Here, trust can increase substantially if actions produce deterministic artifacts that you can verify.

For example:

```
Agent says:
"The Sharpe ratio is 1.41"

TRUST: low
```

versus:

```
Agent executes:
calculate_metrics.py

Input hash:
abc123

Output:
sharpe = 1.4097
```

```
Test suite:
PASS
```

Much better.

The AI isn't the authority anymore. The computation is.

Your agents should be forced to show evidence

I'd give the framework an explicit hierarchy:

```
LEVEL 0
Unsupported LLM statement

"I think..."

v very weak

LEVEL 1
Claim with cited source

v

LEVEL 2
Claim verified against retrieved primary source

v

LEVEL 3
Deterministic computation from immutable data

v

LEVEL 4
Independent replication

v

LEVEL 5
Prospective / untouched holdout evidence

v

LEVEL 6
Forward/paper-trading evidence
```

Then don't let an agent blur these categories.

For example:

"There is strong evidence this strategy works."

should be illegal language if all you currently have is:

```
development backtest
Sharpe 1.8
```

Instead:

```
Evidence status:
DEVELOPMENT_ONLY
```

That seemingly boring label is more scientifically useful than six paragraphs of **Claude** sounding confident.

Make agents disagree

Don't ask five models:

"Is this strategy good?"

They'll often converge because they're reading the same framing.

Give them adversarial roles.

For an important result:

```
Proponent
"Construct the strongest case that this is genuine."

Skeptic
```

```
"Try to falsify it."
```

```
Methodologist
```

```
"Find design flaws."
```

```
Statistician
```

```
"Assess multiple testing and uncertainty."
```

```
Execution specialist
```

```
"Determine whether the result survives realistic trading."
```

Then the conductor summarizes **disagreements**, not merely averages opinions.

Even better, don't show the skeptic the proponent's answer initially.

Independent first-pass analysis reduces conversational anchoring.

Don't use LLM consensus as truth

This trap will be tempting:

```
Claude: PASS
```

```
GPT: PASS
```

```
Gemini: PASS
```

```
3/3 models agree -> valid
```

No.

Three models can share the same misconception because they learned from overlapping human knowledge and receive identical context.

Consensus is useful as a diagnostic signal, not evidence.

Your strongest evidence should come from things LLMs cannot charm into changing:

```
immutable data
```

```
hashes
```

```
registered protocols
```

```
deterministic calculations
```

```
independent engines
```

```
untouched holdouts
```

```
forward observations
```

Test the agents themselves

This is another layer I would add eventually: your virtual quant firm should maintain **an evaluation suite for its own employees**.

Before adopting a new model version as StatisticianAgent, run perhaps **50** or **100** known cases:

```
look-ahead bias hidden in dataset
```

```
survivorship bias
```

```
incorrect Sharpe calculation
```

```
data leakage
```

```
unrealistic fill model
```

```
parameter overfitting
```

```
reuse of holdout
```

```
corporate-action error
```

```
timestamp misalignment
```

```
multiple-testing problem
```

```
incorrect causal claim
```

```
fake citation
```

You know the expected answer.

Then record:

```
Model: Claude X
```

```
Role: methodology_reviewer
```

```
Leakage detection: 96%
```

```
Lookahead detection: 91%
```

```
Citation validity: 99%
```

```
False rejection rate: 12%
```

```
False acceptance rate: 8%
```

This is precisely the direction contemporary agent evaluation is moving: evaluate full trajectories and behavior, not merely whether the final sentence looks plausible. **Anthropic** describes agent evals as particularly important because multi-step tool-using systems can succeed or fail through many different trajectories. **NIST**'s current TEVV framework similarly emphasizes

structured testing, evaluation, verification and validation for agentic systems.

Then when **Anthropic/OpenAI/Google** releases this month's allegedly revolutionary model, you don't just install it because Reddit had an emotional episode.

You test it against **your firm's exam**.

And monitor consistency, not just best-case performance

This is subtle.

Suppose an agent correctly identifies leakage:

```
Run 1: correct
Run 2: correct
Run 3: wrong
Run 4: correct
Run 5: wrong
```

Its capability is obviously there.

Its reliability is not.

That's exactly the distinction recent agent research emphasizes. **HAL** finds substantial gaps between whether an agent **can** solve something and whether it solves it consistently.

So your eval should run important cases multiple times.

A methodology reviewer that spots look-ahead bias **95%** of the time is very different from one that spots it **60%** of the time, even if both occasionally produce brilliant analyses.

I would therefore establish four classes of authority

Something like:

CLASS A - GENERATIVE

hypothesis generation
literature ideas
mechanism proposals

Agent freedom: HIGH
Trust: LOW

CLASS B - ANALYTICAL

methodology review
statistical interpretation
research critique

Agent freedom: MEDIUM
Trust: LOW-MEDIUM
Evidence required: YES

CLASS C - EXECUTION

data processing
backtesting
metrics
testing

Agent freedom: LOW
Trust derives from deterministic verification

CLASS D - GOVERNANCE

holdout release
research promotion
paper-trading approval
capital allocation

Agent authority: NEAR ZERO
Rules/human authority dominate

That is how I'd build your firm.

And there is one more important implication for **you personally as a non-programmer**.

You cannot solve this by becoming the omniscient human reviewer who catches every subtle quant, statistical and programming error. That's unrealistic.

Your job isn't:

“Personally determine whether **Claude's 800-line** implementation is correct.”

Your job is to design the institution so that **no single mistake has to be caught by Gustl staring at Python.**

You rely on overlapping controls:

```
agent
  v
schema validation
  v
unit tests
  v
independent agent review
  v
deterministic backtest
  v
cross-engine comparison
  v
statistical validation
  v
locked holdout
  v
forward test
```

Every layer catches a different class of bullshit.

So if you ask me:

How much should I trust my agents?

My answer is approximately:

Their creativity: quite a lot.

Their factual assertions: provisionally.

Their reasoning: enough to investigate.

Their code: after tests.

Their research conclusions: after independent validation.

Their backtests: after reproduction.

Their judgment about whether they've overfit: almost not at all.

Their authority to risk money: none.

If you build the firm correctly, that's not a weakness. It is the whole scientific architecture.

A real research institution shouldn't work because everyone involved is infallible.

It should work because **being wrong is expected and the process is designed to expose it.**

T11 Agent harness and evaluation harness

Source turn: bbb21459-e0d9-4f95-8b80-0ef89245bdd5. Date: **2026-09-06 18:04 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

So we need a harness? Thats what i read somewhere, or does this have nothing to do with trust in agents?

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes. **A harness is directly related to trust**, but it does not make the model itself trustworthy. It makes an untrustworthy model usable by controlling what it can see, what it can do, what must be verified, and what gets recorded.

Anthropic uses a useful distinction: an **agent harness** is the system around the model that manages inputs, tools, state and execution; an **evaluation harness** is the separate infrastructure that repeatedly tests the resulting agent and measures whether it actually behaves reliably. **LangChain** summarizes the same idea rather neatly as Agent = Model + Harness.

For your project, I think you need **both**.

Your current virtual quant firm is already partly a harness. AGENTS.md, your conductor, schemas, role definitions, validation gates, research states and backend adapters are not merely documentation. They are pieces of scaffolding around the LLMs.

What you're missing is making more of those rules **machine-enforced rather than prompt-enforced**.

The distinction is crucial:

Prompt rule:

"Do not examine holdout data before validation."

Agent:

"Certainly."

Five tool calls later:

examines holdout data

writes 1,800 words explaining why technically it wasn't examination

versus:

Harness rule:

agent_role = hypothesis_researcher

request:

open_dataset(HOLDOUT_2025_2026)

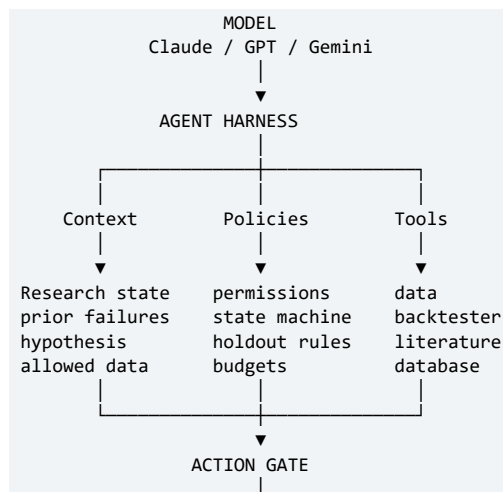
policy_engine:

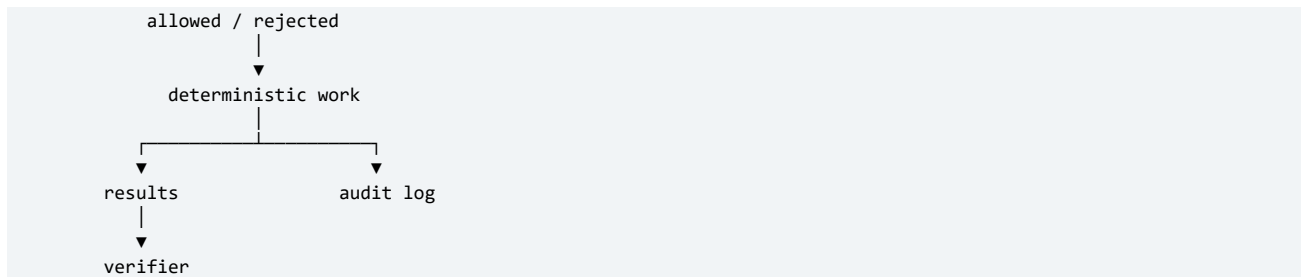
DENIED

reason = ROLE_NOT_AUTHORIZED

The second one is trust engineering.

Your quant-firm harness should eventually look roughly like this:





That changes the role of the LLM fundamentally.

The model says:

“I want to run experiment E17.”

The harness asks:

Is E17 registered?
 Is the hypothesis frozen?
 Is this agent permitted to run it?
 Is this dataset approved?
 Has this holdout already been exposed?
 Are these parameters registered?
 Has the search budget been exceeded?
 Is this a new experiment?

Only then does anything happen.

That is where your scientific standards become **architecture instead of polite suggestions**.

OpenAI's current writing on harness engineering makes essentially the same point from software engineering: reliability comes not only from stronger models but from environments, constraints and feedback loops built around them. **OpenAI's** newer Agents SDK likewise treats sandboxing and controlled execution as harness-level infrastructure.

For your firm specifically, I'd want the harness to own at least these things:

Responsibility	Agent or harness?
Generate hypothesis	Agent
Explain mechanism	Agent
Critique hypothesis	Agent
Decide which tools exist	Harness
Decide which data agent may see	Harness
Lock holdout	Harness
Freeze ExperimentSpec	Harness
Increment experiment/search count	Harness
Record model/tool calls	Harness
Hash datasets/code	Harness
Execute statistical calculations	Deterministic tool
Enforce state transitions	Harness
Decide whether rules permit promotion	Harness
Interpret ambiguous findings	Agent
Final capital authorization	Human/rules

Notice the pattern.

The agent handles tasks requiring intelligence.

The harness handles tasks requiring **obedience**.

You do not want **Claude** deciding whether **Claude** complied with your methodology. There's something faintly Soviet about appointing the defendant chairman of the investigative committee.

There is also another extremely important component: your **evaluation harness**.

That's different.

Suppose your Methodology Agent is **Claude X**. Before trusting it, your eval harness might give it **100** known research cases:

```
Case 001:
hidden look-ahead bias

Case 002:
survivorship bias

Case 003:
holdout contamination

Case 004:
multiple-testing problem

Case 005:
bad timestamp alignment

Case 006:
unrealistic execution price

...
```

For every new model/harness combination you run the suite repeatedly:

```
Methodology Agent v4
Model: Claude X
Harness: quant-harness 0.8.2

lookahead detection      97%
survivorship detection   94%
holdout violations       100%
fake citation detection   93%
false acceptance rate     6%
false rejection rate     11%
```

Anthropic explicitly argues that when evaluating an agent, you're really evaluating **the model and harness together**, because changing tools, context, state handling and execution logic can materially alter results. **OpenAI** makes a similar point: harness choices can change observed agent performance substantially, especially on longer tasks.

That's hugely relevant to you.

You might discover:

```
Claude + Harness A = 88% reliable

Claude + Harness B = 96% reliable
```

Same model.

Different institution around it.

And this is why constantly asking whether **GPT** or **Claude** is “smarter” eventually becomes less important than it currently seems.

For your virtual firm:

Model intelligence determines what the researcher is capable of.

The harness determines what the researcher is allowed and required to do.

The eval harness tells you how much confidence to place in the combination.

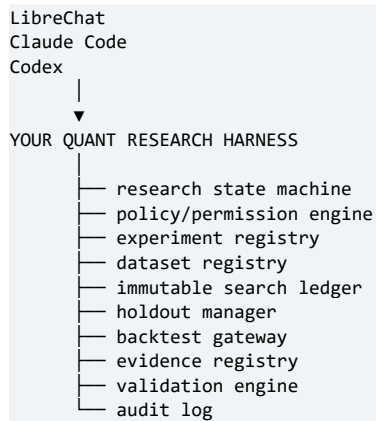
And I think there is an even more useful insight for your existing project:

You don't need to build “an agent harness” from scratch in one gigantic new application.

LibreChat already provides a generic agent harness around models: conversations, models, tools, MCP, agents, context and orchestration.

Claude Code and **Codex** likewise have sophisticated coding harnesses.

What your repository needs is the **domain-specific scientific harness** sitting underneath/alongside those:



The models can change.

LibreChat can change.

Claude Code can change.

The scientific harness remains the institution.

I would actually make that one of the core architectural principles of the project:

The virtual quant firm is the harness, not the LLM.

Claude isn't your firm. **GPT** isn't your firm. The conductor isn't even your firm.

The firm is the **protocol, state, permissions, evidence, history and deterministic gates surrounding them.**

That is also how you solve the trust problem without pretending agents will suddenly become infallible. You don't need trustworthy little digital employees. You need a system in which **untrustworthy researchers can produce trustworthy evidence.**

Science, inconveniently, was invented for basically the same reason.

T12 Agents versus deterministic programs

Source turn: bbb21382-ccff-4d2a-a873-795a7525cf5c. Date: **2026-09-06 19:08 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

But one could ask, where is the advantage in an agentic system if a normal program could do the job of the agent more trustworthy

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

That is exactly the right objection. And it leads to a slightly uncomfortable conclusion for the current “agents everywhere” fashion: **if a normal program can do the job reliably, you generally should not use an agent for it.**

The advantage of an agentic system only appears where the task is too ambiguous, open-ended, semantic, or adaptive to encode cleanly in conventional software. Agents are useful because they can interpret messy goals, read literature, form hypotheses, notice unexpected relationships, critique arguments, and decide what investigation might be worth doing next. A deterministic program is far better at calculating Sharpe ratios, enforcing permissions, checking hashes, executing a frozen strategy, validating schemas, or deciding whether a state transition is legal.

So your virtual quant firm should not be “agentic all the way down.” That would be an expensive method for replacing dependable software with eloquent uncertainty.

A useful division is:

Task	Better choice
Calculate returns	Program
Calculate Sharpe/Sortino/drawdown	Program
Run backtest	Program
Enforce holdout lock	Program
Record search history	Program
Check experiment state	Program
Validate schema	Program
Execute registered statistical test	Program
Determine whether fixed threshold passed	Program
Read ten papers and synthesize competing explanations	Agent
Generate plausible market hypotheses	Agent
Translate vague retail strategy description into formal rules	Agent + verification
Critique methodology for subtle conceptual flaws	Agent
Notice that a result may have an alternative explanation	Agent
Suggest additional falsification tests	Agent

Task	Better choice
Investigate an unexpected anomaly	Agent
Decide what research question to pursue next	Agent, constrained

The strongest architecture for your project is therefore something like:



That is the interesting combination.

The normal program supplies **reliability**.

The agent supplies **adaptability**.

And neither should be asked to do the other's job.

There is another subtle advantage. A conventional program can execute only the research process you already know how to specify. An agent can react to things you did not anticipate.

Imagine a backtest produces a strange result:

Strategy performs normally:
2010-2018

Performance collapses:
2019-2020

Performance returns:
2021-2025

A deterministic program can flag:

REGIME_INSTABILITY = TRUE

Useful.

But an agent can investigate:

Was the change associated with tick-size rules?

Did market structure change?

Did the underlying index methodology change?

Was there a vendor data problem?

Did volatility regimes alter the expected mechanism?

Is this actually evidence against our causal explanation?

It can search literature, examine metadata, propose competing explanations and design follow-up tests.

You could theoretically encode every possible investigation into software.

You could also theoretically build an entire quant firm out of if statements. You would finish shortly after the heat death of the universe.

That is where agents become valuable.

The same applies to hypothesis generation.

A conventional program is excellent at:

```
for fast_ma in range(5, 101):
    for slow_ma in range(20, 301):
        test(...)
```

But that isn't really research. It's parameter excavation.

An agent can reason from:

```
market microstructure
behavioral finance
institutional constraints
regulatory changes
economic incentives
academic literature
known anomalies
```

and formulate:

"If forced deleveraging creates temporary liquidity pressure near specific institutional rebalancing windows, prices should partially reverse once the constrained flow disappears."

That's a genuine hypothesis.

The agent contributed something conventional software struggles with: **semantic abstraction and creative synthesis**.

Then conventional software takes over and tries to kill the hypothesis.

That distinction is very important for your firm:

Agents should expand the hypothesis space.

Programs should contract the evidence space.

The agent says:

"Here are ten possible explanations."

The scientific machinery says:

"Nine are unsupported, one survives provisional testing."

That asymmetry is desirable.

I would therefore resist building a system where the conductor continuously decides everything. The conductor should mostly handle situations where the next action depends on interpretation.

For example:

```
result:
validation failed

reason:
transaction costs erased 83% of gross edge
```

You don't need **Claude** to decide:

reject if net edge < registered threshold.

That's deterministic.

But suppose:

```
result:
validation partially failed

finding:
signal exists only during unusually high
liquidity-stress periods
```

Now an agent may reasonably ask:

Was our original hypothesis actually conditional on liquidity stress?

Is this legitimate subgroup evidence or retrospective slicing?

Does the mechanism justify a new preregistered hypothesis?

That's genuinely interpretive.

The system should probably produce:

```
ORIGINAL HYPOTHESIS:  
REJECTED  
  
NEW HYPOTHESIS:  
may be generated  
  
NEW EXPERIMENT:  
required
```

The agent can propose that.

The harness enforces that it cannot rewrite history.

This also suggests a useful architectural principle:

Make the agentic surface area as small as possible without destroying the system's ability to reason.

That's almost the opposite of what agent demos usually advertise. Agent demos love:

"The AI autonomously manages your entire company!"

Wonderful. We've replaced accountable software with a probabilistic intern holding root credentials.

Your system should instead ask:

Where do we genuinely require judgment?

Only put agents there.

Everything else becomes boring infrastructure.

And there's one final advantage agents give your project that conventional software cannot easily provide: **the virtual firm can evolve its own research agenda.**

A normal program can perform:

```
ResearchCase A  
ResearchCase B  
ResearchCase C
```

because you programmed A, B and C.

An agentic research organization can observe:

```
A failed for reason X  
B failed for reason X  
C partially survived when condition Y held
```

and infer:

"Perhaps X is telling us that the phenomenon depends on Y. We should investigate Y directly."

That is closer to what human researchers actually do.

So the target isn't:

autonomous agents replacing programs.

It's:

agents doing the parts conventional programs cannot specify in advance, surrounded by conventional programs wherever correctness can be specified.

For your project I would make that almost a constitutional rule:

Never use an LLM where a deterministic algorithm can answer the same question satisfactorily.

Then what remains for the agents is precisely the interesting part: hypothesis formation, interpretation, criticism, synthesis, investigation and research direction.

Which, inconveniently, is also the part humans are worst at doing without convincing themselves they're geniuses.

T13 SR 11-7 SR 26-2 observability and lineage

Source turn: bbb21b4d-766e-4bf6-bc9e-c639d522ea52. Date: **2026-09-06 19:37 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

What about SR 11-7 and observability tracing

Assistant discussion

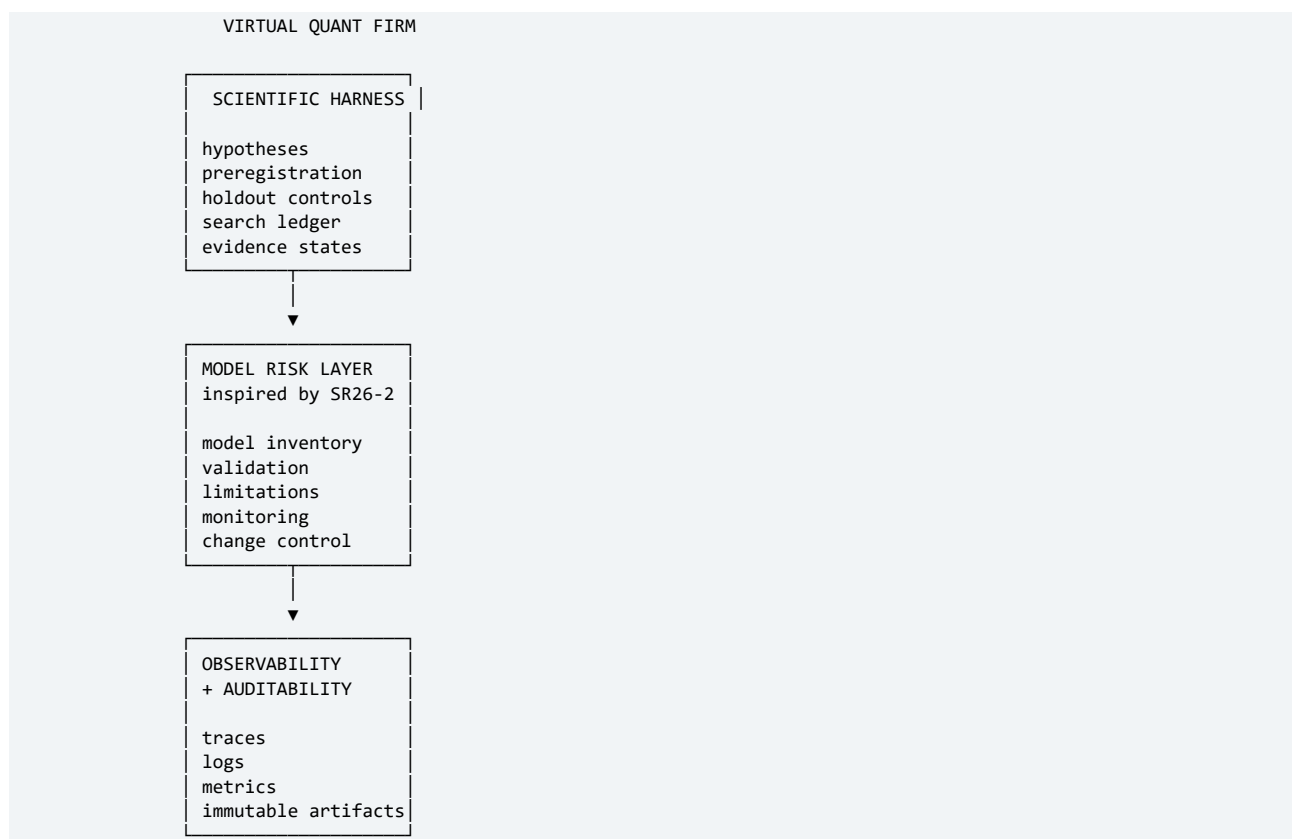
[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Very relevant. In fact, I'd put both into the architecture, but with one update: **SR 11-7 was superseded on April 17, 2026 by SR 26-2**, the revised interagency Model Risk Management guidance. So I'd borrow principles from the current SR 26-2 rather than implement an old document verbatim.

There's an interesting wrinkle: the **2026** guidance explicitly says **generative AI and agentic AI are outside its scope** because they're evolving too quickly. But it also says organizations should still apply appropriate governance and controls to systems outside the guidance. Its core subjects remain model development/use, validation and monitoring, governance/controls, model inventory, documentation, and third-party products.

For your project, that is almost perfect.

I'd think of the architecture as three related but distinct systems:



They solve different problems.

The scientific harness asks: "Is this research valid?"

Model risk management asks: "Do we understand this model, its limitations, its intended use, and whether it still performs properly?"

Observability asks: "What the hell actually happened?"

All three matter.

SR 26-2 is extremely applicable to the trading models

Suppose your virtual firm eventually produces:

STRATEGY_MODEL_031
Intraday equity mean reversion

I would give it something equivalent to an institutional model record:

Model ID:
STR-031

Owner:
Research Strategy Group

Purpose:
Intraday US equity mean-reversion strategy

Permitted use:
research / paper trading

Prohibited use:
live capital allocation

Development data:
2011-2020

Validation:
2021-2023

Final holdout:
2024-2025

Current status:
VALIDATED / PAPER_ONLY

Known limitations:
- sensitive to spreads
- weak during volatility shocks
- capacity uncertain below \$X ADV
- no evidence outside US equities

Last validation:
2026-08-14

Monitoring status:
PASS

That's very SR-26-2-ish.

The revised guidance emphasizes **conceptual soundness, outcome analysis, ongoing monitoring and limitations**, rather than merely asking whether the model once passed a backtest.

That's exactly what you'll eventually need for trading strategies.

A strategy that worked beautifully from **2012-2024** is not permanently:

VALIDATED.

It might become:

2027

expected Sharpe:
1.2-1.6

realized trailing Sharpe:
0.31

turnover:
+47%

slippage:
+83%

signal decay:
significant

MODEL STATUS:
DEGRADED

At that point the monitoring system flags it.

The agent can investigate why.

The agent **cannot quietly change the model and continue calling it STR-031**.

New version:

STR-031-v2

New validation.

That is proper model governance.

And SR 26-2 gives you another very useful concept: model inventory

Your firm will eventually contain many things that could count as models:

strategy models
risk models
transaction-cost models
slippage models
volatility models
regime models
forecast models
position-sizing models
possibly ML models

You should eventually have one central registry.

Something like:

ID	Model	Risk	Status	Owner	Validation
M-001	SPY reversal strategy	High	Research	Strategy agent	pending
M-002	Slippage estimator	High	Active	Execution	validated
M-003	Volatility regime classifier	Medium	Active	Research	validated
M-004	Position-sizing model	High	Paper	Risk	pending

SR 26-2 specifically identifies a comprehensive model inventory as common sound practice because you need to understand model risk both individually and in aggregate.

That's a surprisingly important idea for your database design.

Now, observability/tracing is the complementary piece for the agents themselves.

OpenTelemetry defines a trace as the path a request takes through a system, consisting of nested spans, each representing some unit of work.

For ordinary software:

```
HTTP request
├── database query
├── API call
└── response
```

For your quant firm, I would trace an entire research investigation:

```
TRACE: RC-0042

research_case.run
├── conductor.plan
├── hypothesis_agent.generate
├── literature_agent.search
│   ├── search.source
│   ├── retrieve.paper
│   └── verify.citation
├── methodology_agent.review
├── policy.preregister
└── data_agent.admit_dataset
```

```

├── backtest.run
│   ├── load.dataset
│   ├── compile.strategy
│   ├── run.backtrader
│   └── calculate.metrics
├── validator.review
├── replication.run
│   └── run.lean
└── research_decision.evaluate

```

Now suppose six months later you find:

RC-0042 appears contaminated.

You don't ask **Claude**:

"Do you remember what happened?"

God no.

You query the trace.

You discover:

```

14:03:12
hypothesis registered

14:06:31
development dataset opened

14:14:02
strategy modified

14:18:51
validation executed

14:21:17
holdout dataset access requested

14:21:17
POLICY DENIED

14:25:03
strategy modified again

14:28:42
new experiment registered

...

```

Now you have an actual forensic record.

And modern OpenTelemetry already understands AI

This is no longer some awkward hack where you label every LLM invocation HTTP POST.

OpenTelemetry's current GenAI conventions capture things such as the requested model, token usage, response finish reason, agent identity, conversation identity and tool executions. Current examples show an `invoke_agent` span containing child chat spans and `execute_tool` spans.

So your trace might contain:

```

span:
methodology_agent.review

agent:
methodology-reviewer-v3

provider:
Anthropic

model:
Claude-...

temperature:
...

```

```
input_tokens:
18,432

output_tokens:
3,822

experiment_id:
EXP-00472

hypothesis_id:
HYP-0182

dataset_id:
DS-019

strategy_version:
STR-031-v4

git_commit:
82bd...

tool_calls:
  read_experiment
  inspect_dataset_manifest
  run_bias_check

result:
REJECT

reason_code:
HOLDOUT_CONTAMINATION
```

That is vastly more useful than preserving a chat transcript.

And I would add your own domain-specific attributes:

```
quant.research_case.id
quant.hypothesis.id

quant.experiment.id
quant.experiment.version

quant.dataset.id
quant.dataset.sha256

quant.strategy.id
quant.strategy.sha256

quant.search_budget.used

quant.holdout.exposed
quant.holdout.exposure_timestamp

quant.agent.role

quant.policy.decision

quant.validation.status

quant.evidence.level
```

That gives you queries later such as:

Show every experiment where holdout information was accessed before strategy freeze.

or:

Show all models where **Claude** Opus produced the hypothesis and **GPT** performed validation.

or:

Find strategies that passed validation but failed cross-engine replication.

Now you're getting somewhere interesting.

But tracing is not the same as auditability

This distinction matters.

OpenTelemetry traces are designed primarily for **observability**: understanding what your system is doing, diagnosing problems, monitoring latency, failures and behavior.

For your scientific framework I'd add a stricter **audit/event ledger**.

So:

```
OBSERVABILITY

"What happened operationally?"

traces
logs
latency
tokens
errors
tool calls
```

versus:

```
SCIENTIFIC AUDIT LOG

"What happened that matters epistemically?"

hypothesis created
experiment registered
parameters frozen
dataset admitted
validation begun
holdout exposed
strategy changed
experiment rejected
model promoted
```

The second should be **append-only**.

Agents cannot erase it.

For example:

```
EVENT 83912

timestamp:
2026-09-06T19:21:31Z

type:
HOLDOUT_ACCESS_ATTEMPT

research_case:
RC-0042

agent:
conductor-v7

dataset:
DS-HOLDOUT-011

decision:
DENIED

policy:
HOLDOUT_LOCK_V2

trace_id:
3af932...
```

The `trace_id` links the scientific event to all the messy technical details.

That gives you:

```
Database
  v
scientific truth / state

Audit ledger
  v
immutable important history

OpenTelemetry
  v
full operational history
```

That's a strong architecture.

Don't log everything blindly

There's one trap here.

AI observability vendors sometimes advertise:

Capture every prompt and response!

For your system, that can be a terrible default.

Prompts/tool outputs could contain:

- licensed market data,
- credentials,
- API keys,
- private research,
- personally identifying information,
- gigantic datasets.

OpenTelemetry's GenAI telemetry deliberately treats recording full prompts, completions and tool results as something that can be opted into, not something that should indiscriminately happen.

I would generally store **references and hashes**:

```
prompt_artifact_id:
PROMPT-882

prompt_sha256:
...

dataset_id:
DS-019
```

rather than dumping a **300 MB** slice of **dxFeed** into **Jaeger** because an observability tutorial told you telemetry was fun.

Likewise, you don't need or want secret model chain-of-thought. Store the agent's **explicit conclusion, evidence, actions and justification**.

Put together, this answers the trust problem rather elegantly

Imagine an agent concludes:

"This strategy has survived validation."

You don't trust the sentence.

The system can automatically reconstruct:

```
Model inventory
  v
What exactly is this strategy?

Experiment registry
  v
What was preregistered?

Search ledger
  v
How much searching preceded this?

Trace
  v
What did every agent actually do?

Dataset registry
  v
Which immutable data were used?

Git hash
  v
Which strategy code ran?

Backtest artifacts
```

```

      v
What were the deterministic outputs?

Validation record
      v
Who/what challenged it?

Holdout ledger
      v
Was holdout integrity preserved?

Monitoring
      v
Does it continue to perform?

```

Now the agent isn't asking you to **believe it**.

It's giving you a claim backed by a reconstructable chain of evidence.

And I'd make that one of the foundational design ideas of your project:

Every important research conclusion must be traceable backward from claim -> validation -> experiment -> code -> data -> hypothesis.

And the opposite direction too:

Every hypothesis must be traceable forward to every experiment, modification, failure and conclusion it generated.

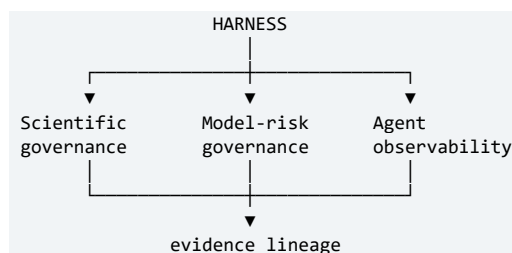
That's essentially **lineage**.

For a scientific quant system, lineage + model risk management + agent tracing might actually be more important than choosing **Claude** versus **GPT**.

The models are interchangeable researchers.

The lineage is the institution's memory.

So, yes: I'd add **SR-26-2-inspired model risk management** and **OpenTelemetry-style observability/tracing** to the design. But don't treat them as two fashionable extras. They fit naturally into the harness we've been discussing:



At that point your project starts looking considerably less like a vibe-coded backtester and rather suspiciously like actual research infrastructure. Which is inconvenient, because now you'll eventually have to document the damned thing.

T14 Retail proportionate governance

Source turn: bbb21f46-fcd1-4dab-837d-5da4d3d11e48. Date: 2026-09-06 20:32 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

But we shouldnt overdo the risk layer. Its still retail trading, not insitutional hedge fonds. If the risk layer becomes too big, it might slow down the process. We need a middleground

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

You're right to resist turning this into Deutsche Bank's model-risk department in a shed. If governance becomes heavier than the research itself, the framework has failed.

The right principle is **risk-proportionate governance**: apply strict controls only where errors are expensive or scientifically contaminating.

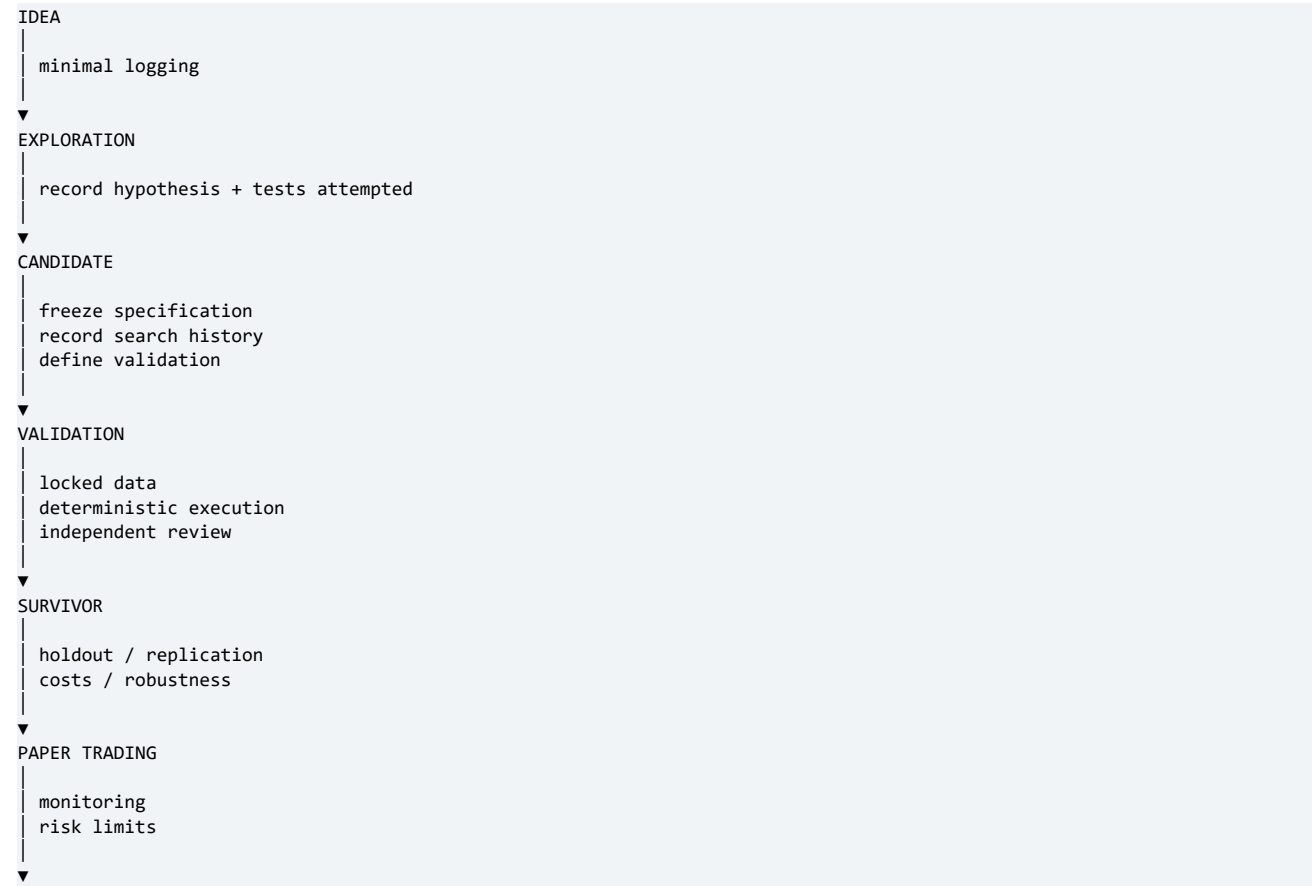
For your virtual quant firm, I'd use three levels:

Layer	Governance
Idea generation / early exploration	Light
Serious backtest / validation	Moderate
Paper/live trading	Strict

For early research, agents should be allowed to move fast. Generate hypotheses, inspect literature, run rough exploratory tests, discard rubbish. You do not need a compliance ceremony because **Claude** wondered whether Tuesday reversals exist.

The stricter controls should appear only when a result starts becoming a **claim**.

A practical workflow:



The crucial distinction is between **exploration** and **confirmation**.

During exploration, you can let the agents poke around quite freely. That's where creativity is valuable.

But an exploratory result cannot magically become:

"We discovered an edge."

Instead it becomes:

"Interesting candidate. Register a clean confirmatory experiment."

That one rule buys you a huge amount of scientific integrity without suffocating the process.

I'd therefore define a **Minimum Viable Governance** layer for your current stage:

1. Every serious hypothesis gets an ID.
2. Every meaningful experiment gets recorded.
3. Raw and canonical datasets are immutable/hashable.
4. Development and holdout data are separated.
5. Once validation begins, the strategy specification is frozen.
6. Any post-result modification creates a new version.
7. Failed experiments are retained.
8. Important calculations come from deterministic code, not LLM prose.
9. Agent actions that affect evidence get an audit event.
10. No agent can authorize real-money trading.

That's enough for now.

You do **not** need, at this stage:

```
17 model-risk committees
quarterly governance packs
three levels of management sign-off
formal regulatory reporting
enterprise-wide risk taxonomies
independent validation departments
400-page model documentation
```

You're one person building a retail research system, not preparing testimony before the Federal Reserve.

I would also make governance **automatic wherever possible**. That's how you avoid slowing research down.

Bad governance:

Fill in **23** fields before running a backtest.

Good governance:

```
agent requests backtest

system automatically records:
- hypothesis_id
- strategy hash
- dataset hash
- parameters
- timestamp
- model/agent
- git commit
```

Zero extra work for you.

Similarly, instead of manually maintaining a search ledger, the harness should append to it automatically whenever a new experiment or parameter search is executed.

Then governance becomes infrastructure rather than paperwork.

The same applies to observability. You don't need to inspect every trace. Most of the time, traces should sit quietly in storage doing absolutely nothing, their highest calling in software engineering.

You inspect them when:

- something fails,

- two agents disagree,
- a promising result looks suspicious,
- a holdout appears contaminated,
- a strategy changes unexpectedly,
- you need to reconstruct how a conclusion was reached.

That's enough.

I would also classify strategies by importance.

Something like:

```
RISK 0
idea / exploratory
-> almost no governance

RISK 1
research candidate
-> experiment logging

RISK 2
validated candidate
-> holdout + replication + lineage

RISK 3
paper trading
-> monitoring + execution controls

RISK 4
real capital
-> hard limits + kill switch + human approval
```

That is much more sensible than applying institutional-grade controls to every speculative thought produced by an LLM.

And I would borrow from SR **26-2** selectively rather than “implement SR **26-2**.”

Take the useful ideas:

- model inventory
- intended use
- known limitations
- validation
- change/version control
- ongoing monitoring

Ignore the organizational bureaucracy designed for banks with thousands of employees and regulators.

Your framework needs the **epistemology**, not the suits.

The same middle-ground principle should apply to the agents themselves:

High freedom in proposing. Medium freedom in investigating. Low freedom in confirming. Near-zero freedom in deploying capital.

That is probably the cleanest governance model for your project.

It preserves the thing that makes an agentic research system valuable, namely speed and exploratory intelligence, while putting friction exactly where friction is useful: at the point where exploration becomes evidence, and where evidence becomes money.

That is the middle ground I'd build around.

T15 Process behavior charts and intervention measurement

Source turn: bbb214bb-4a14-4534-9004-d17081ea7954. Date: 2026-09-07 14:38 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

We would also need metrics if our agents and certain interventions make the process worse or better. Something like process behavior charts

Assistant discussion

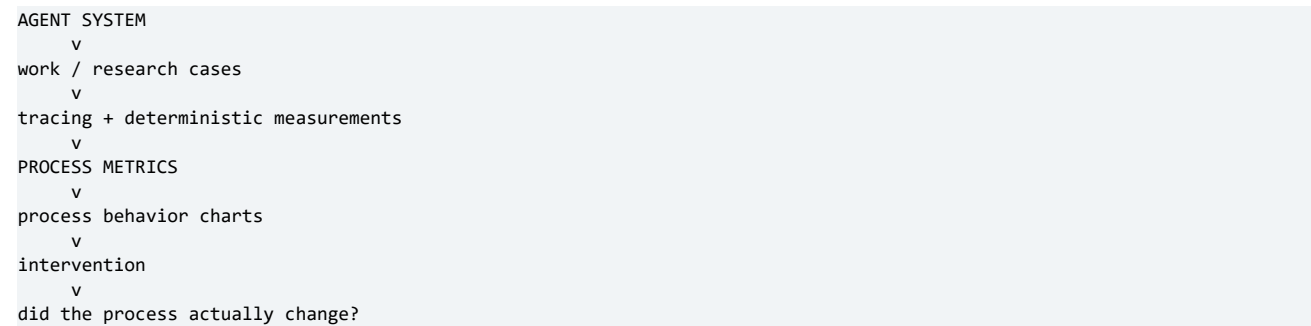
[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes. I think this is the missing feedback loop in the architecture.

You don't only want observability telling you **what the agents did**. You want measurement telling you **whether changing the system actually improved it**.

And process behavior charts are a surprisingly good fit because agent systems are noisy. The exact same conductor, model, prompt and tools can produce different trajectories across runs. Looking at “before = 87%, after = 91%” and declaring victory is precisely the sort of statistical misdemeanor your quant firm is supposed to despise.

The basic structure I'd use is:



Process behavior charts are specifically intended to distinguish routine variation from signals suggesting the underlying process changed. An XmR chart, for example, tracks individual sequential observations and their moving ranges rather than encouraging you to panic over every wiggle in a line chart.

That's very relevant to stochastic agents.

But I'd be careful about **what** you measure.

The biggest mistake would be measuring:

Number of hypotheses generated.

Congratulations, **Claude** produced **600** hypotheses. Your virtual firm is now the world's most efficient producer of statistical landfill.

You need several dimensions simultaneously.

For your system I'd start with four.

Dimension	Example metrics
Research quality	factual errors, unsupported claims, methodology defects, missed biases
Reliability	successful completion rate, reproducibility, variance between repeated runs
Efficiency	time, tokens, API cost, tool calls, retries
Process integrity	protocol violations, unauthorized data access, experiment/version mistakes

For example, your methodology reviewer could have:

Methodology reviewer	
Missed known defect rate	4.2%

False alarm rate	7.1%
Unsupported claims/case	0.18
Successful completion	97.4%
Median review cost	€0.84
Median cycle time	94 sec

Then track those over time.

The conductor gets somewhat different metrics:

```
Conductor

Correct routing rate
Unnecessary delegation rate
Task completion rate
Recovery-after-failure rate
Plan revisions per case
Invalid state-transition attempts
Tool failures
Human intervention rate
Cost per completed research case
```

The really interesting one is **human intervention rate**.

Suppose initially you have to rescue the conductor in **30%** of cases:

```
30%
27%
34%
29%
31%
...
```

Then you introduce a better state representation on June 1.

Afterward:

```
18%
17%
16%
20%
15%
...
```

A process behavior chart can help distinguish an actual shift from ordinary variation rather than allowing you to become emotionally attached to the first good Tuesday. A stable XmR chart characterizes the routine behavior of a process; signals indicate something worth investigating rather than automatically proving its cause.

And that latter distinction is important:

A process behavior chart detects change. It doesn't prove that your intervention caused it.

So I'd combine it with an intervention ledger.

Something like:

```
INTERVENTION INT-0041

Date:
2026-11-03

Component:
MethodologyAgent

Change:
Added mandatory DatasetFitness probe

Hypothesis:
Should reduce missed point-in-time data issues.

Primary metric:
Missed data-defect rate

Balancing metrics:
cycle time
cost
false-positive rate

Baseline:
AgentHarness v0.8.1

Intervention:
```

Then annotate that intervention directly on your time series.

Now six months later you can actually answer:

Did introducing DatasetFitness improve anything?

rather than relying on your recollection that “it seemed better.”

That is basically applying continuous quality improvement to your agent organization.

There is an especially important concept here from **Deming: don't confuse worker performance with system performance**. His Red Bead experiment was specifically designed to demonstrate how much observed worker variation can be produced by the system they're operating within.

That maps wonderfully onto agents.

Suppose **Claude** appears to become worse:

```
January 94% success
February 92%
March 81%
```

Maybe **Claude** got worse.

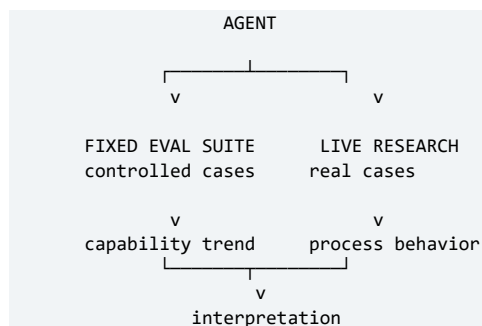
Or perhaps:

```
task difficulty increased
context became larger
tools became less reliable
data changed
conductor routing changed
prompt changed
harness changed
evaluation set changed
```

So I would **never rank your models purely on observed production success**.

You need a stable evaluation suite alongside the live process.

Something like:



This matters because **NIST's** recent work makes essentially the same measurement distinction: performance on a fixed benchmark isn't necessarily the same thing as generalized performance across the broader task population, and uncertainty has to be treated explicitly.

For your firm I'd therefore maintain maybe **50-100 diagnostic cases** eventually.

For example:

```
CASE-001
Obvious lookahead bias

CASE-002
Subtle survivorship bias

CASE-003
Incorrect timestamp alignment

CASE-004
Fake academic citation

CASE-005
Validation leakage

CASE-006
Unrealistic fill model
```

```
CASE-007
Multiple testing disguised as one experiment

CASE-008
Incorrect corporate action adjustment
```

Each has known expected findings.

Whenever you change:

- model
- prompt
- harness
- agent role
- tools
- context construction
- conductor
- reviewer
- memory strategy

run those cases again.

Anthropic recommends evaluating agent behavior across the full trajectory rather than merely grading final outputs, because agents operate over multiple steps, tools and state changes. **NIST** is now developing similar embedded evaluation probes that check agent work during execution and accumulate machine-readable audit trails.

That fits your architecture nearly perfectly.

I'd introduce the concept of a **probe**.

For example:



Those probes generate quantitative observations automatically.

You don't sit there reading **400** traces.

Instead:

```
CASE-182

citation grounding      PASS
dataset provenance     PASS
holdout integrity      PASS
parameter registration FAIL
strategy consistency   PASS
```

These results feed the metrics database.

Then process behavior charts tell you whether those processes improve.

This also creates a lovely way of evaluating whether **governance itself is becoming harmful**, which connects directly to your previous concern.

Suppose you add another validation layer.

Maybe:

Methodology errors:
8.1% -> 4.3%

Excellent.

But:

Research cycle time:
14 min -> 39 min

API cost:
€1.80 -> €5.90

agent retries:
1.2 -> 3.7

Now the intervention isn't simply "good."

You have a tradeoff.

So every intervention should have a **primary metric and balancing metrics**.

Example:

INTERVENTION

Independent second reviewer

PRIMARY
false acceptance v

BALANCING
cost
cycle time
false rejection
human intervention

That's how you stop the risk layer from metastasizing.

One especially useful metric for your system would be something I'd call **research rework**:

percentage of research cases
that must return to an earlier state
because of a preventable process error

Examples:

VALIDATION
v
oops, dataset was wrong
v
DATA_ADMISSION

or:

HOLDOUT
v
oops, specification wasn't frozen
v
PREREGISTRATION

If your harness improves, rework should fall dramatically.

That's a much more meaningful operational metric than "tokens used."

I'd also measure **agent disagreement**.

Not because disagreement is necessarily bad. Sometimes disagreement is precisely what you want.

Instead:

Methodologist vs Validator disagreement rate

Conductor vs policy-engine disagreement rate

Independent replication disagreement rate

Then classify resolutions:

agent A wrong
agent B wrong
both partially wrong
specification ambiguous
deterministic tool defect

unresolved

Over time you may discover something fascinating:

Gemini finds data problems better than **Claude**.

or:

GPT is better at catching statistical errors, while **Claude** generates stronger mechanisms.

or:

Adding a second model doesn't improve correctness enough to justify doubling cost.

Those should be **empirical findings about your own firm**, not Reddit theology.

This is actually where your observability infrastructure suddenly becomes genuinely useful.

Each trace already tells you:

```
model
agent
harness version
prompt version
tools
tool calls
tokens
latency
experiment
result
```

Add eval/probe outcomes and you have a dataset:

```
agent_runs
-----
run_id
timestamp
agent_role
model
model_version
harness_version
prompt_version
task_class
task_difficulty
success
quality_score
violations
retries
latency
tokens
cost
human_intervention
```

Then your database becomes capable of analyzing **the researchers themselves**.

That gives your virtual firm two scientific loops:

```
LOOP 1

Markets
  v
Hypotheses
  v
Experiments
  v
Evidence
  v
Better trading knowledge
```

and:

```
LOOP 2

Agents
  v
Research behavior
  v
Measurement
  v
Interventions
  v
Better research process
```

I think that second loop is unusually important.

Because otherwise you'll constantly upgrade models, modify prompts, add reviewers, change orchestration and tweak the harness while having absolutely no idea whether you've improved the system.

That's basically discretionary trading, except the thing you're trading is your own software architecture.

One warning, though: **don't create a giant dashboard with 73 KPIs.**

I'd start with perhaps eight:

1. Research-case completion rate
2. Methodology defect rate
3. Unsupported/factually wrong claim rate
4. Human intervention rate
5. Rework rate
6. Median cycle time
7. Cost per completed case
8. Repeated-run consistency

Then use process behavior charts for the ones that produce meaningful sequential observations.

Add metrics only when you have a specific question they answer.

And importantly, don't automatically optimize all of them. A target such as:

reduce methodology defect rate to zero

can create pathological behavior where the agent simply rejects everything.

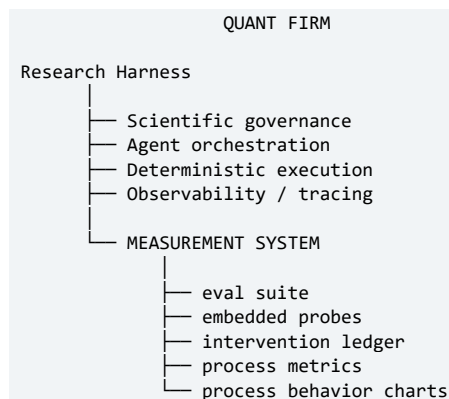
Similarly:

maximize validation survival rate

would produce the world's most accommodating validator.

Metrics should help **understand the system**, not become numerical commandments.

So yes, process behavior charts fit extremely well here. I'd actually make **measurement and continuous improvement** a distinct component of the harness:



And that gives you a rather elegant principle:

The agents conduct science on trading strategies. The harness conducts science on the agents.

That, more than another committee-shaped risk layer, is how I'd make this thing progressively more trustworthy without strangling it.

T16 Uploaded measurement framework

Source turn: bbb21927-a521-4f07-8f62-78789768c7e2. Date: **2026-09-07 14:47 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Maybe like this

[User attached 1 image; image contents were not included]

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes. Exactly that. This is much better than building a grotesque dashboard cemetery full of numbers nobody understands and everybody worships anyway.

What you showed is basically the right management logic for your virtual quant firm:

1. Are we achieving the goal?

2. Are we improving?

3. How do we improve?

That is a far better frame than "which 47 KPIs should we collect because telemetry exists."

For your system, I'd translate those three questions into a small measurement architecture.

1. Are we achieving our goal?

First you need to define the goal clearly, otherwise the charts become decorative superstition.

For your firm, the goal is not:

generate many hypotheses

That would merely industrialize nonsense.

The more serious goal is something closer to:

produce scientifically valid research cases that reach a defensible conclusion efficiently and without contaminating the evidence.

That immediately gives you two subquestions from the image:

a) Are we doing so occasionally?

Meaning: do we ever succeed at all?

Good measures:

- research cases completed to final verdict
- percentage of cases that reach a proper conclusion
- number of cases surviving validation or holdout
- number of cases completed without protocol violation

b) Are we doing so consistently?

This is where process behavior charts come in.

You don't want one glamorous result and then six weeks of garbage. That's not a process. That's a lucky accident wearing a tie.

So for consistency, track sequential metrics over time, for example:

- **case completion rate**
- **methodology defect rate**
- **rework rate**
- **human intervention rate**

- **cycle time per research case**

Then plot them as **XmR charts** or other suitable control/process-behavior charts.

If the system is stable, you can say:

this is our current process capability

If the chart shows a shift or instability, then something changed.

That is the real answer to “are we doing it consistently?”

2. Are we improving?

This is the most important part, and also where people usually start lying to themselves.

The proper answer is **not**:

it felt better after I changed the prompt

Marvelous. Theology has entered the lab.

You need an intervention log plus time-series measures.

For each intervention, record:

- what changed
- why it changed
- expected effect
- primary metric
- balancing metrics
- date/version

Example:

```
INT-014
Change: added independent methodology reviewer
Goal: reduce false acceptance of weak research cases
Primary metric: methodology defect escape rate
Balancing metrics: cycle time, cost per case, false rejection rate
```

Then ask:

a) Can we predict future performance?

That depends on whether the process is stable.

If the process behavior chart shows a stable process, then yes, to some extent you can predict the likely range of future performance.

If the chart is unstable, then prediction is mostly fantasy with a grid overlay.

This is one of the best reasons to use process behavior charts in your system: they help answer whether performance is merely fluctuating normally or whether the process actually changed.

3. How do we improve?

This question should split exactly the way your image suggests.

a) When do we react?

Meaning: when do we respond to a particular case or signal?

Examples:

- a spike in protocol violations
- an outlier in cycle time
- a failed replication
- an unexpected surge in human rescues

- repeated unsupported claims by an agent

This is local response: fix the issue, inspect the case, maybe roll back a model or prompt version.

b) When do we step back and improve the system?

This is bigger.

Examples:

- rework rate is persistently high
- methodology defects keep escaping
- the conductor routes tasks badly
- adding a second reviewer doubled cost without real benefit
- a whole class of cases repeatedly fails because the harness lacks a proper data gate

This is where you improve the institution, not just the individual case.

c) How can we prove we've improved?

This is the key one.

You prove improvement by showing:

- a meaningful change in the metric over time
- ideally a sustained shift, not a one-off wobble
- no unacceptable harm in balancing metrics

So if you reduce methodology defects but triple cycle time and double cost, you haven't simply "improved." You've traded one problem for another. Very human. Very expensive.

How I'd adapt your image to the virtual quant firm

I'd build a dashboard or reporting structure around those exact three questions.

Question 1: Are we achieving our goal?

Use maybe **3-4** core metrics:

- **Research-case completion rate**
- **Protocol-clean completion rate**
- **Defensible conclusion rate**
- **Consistency chart for cycle time or completion**

Question 2: Are we improving?

Use intervention-aware charts for:

- **Methodology defect rate**
- **Human intervention rate**
- **Rework rate**
- **Cost per completed case**

Each chart should show:

- center line
- control/process limits
- intervention annotations

Question 3: How do we improve?

This becomes your operational review:

- what special-cause signals require immediate action?
- what systemic patterns require redesign?
- what interventions should we test next?
- how will we know the intervention worked?

Very important: distinguish outcome metrics from process metrics

You need both.

Outcome metrics

These reflect whether the firm is achieving its purpose.

Examples:

- number of valid research conclusions
- number of strategies surviving validation
- number of candidates surviving holdout
- paper-trading survival rate later on

Process metrics

These reflect whether the research organization itself functions well.

Examples:

- defect rate
- rework rate
- cycle time
- cost
- intervention frequency
- consistency of repeated runs
- protocol-violation rate

If you only track outcomes, you may miss a degrading process until it breaks.

If you only track process metrics, you may build a beautifully disciplined machine that produces nothing useful. German engineering meets spiritual emptiness.

A very good starter set for your system

I'd start with just these eight:

- 1. Research-case completion rate**
- 2. Protocol-clean completion rate**
- 3. Methodology defect rate**
- 4. Unsupported claim rate**
- 5. Human intervention rate**
- 6. Rework rate**
- 7. Median cycle time per case**
- 8. Cost per completed case**

And if you want a ninth:

- 9. Repeated-run consistency**

That last one is especially relevant for agents. If the same case produces wildly different quality across runs, you don't have a process; you have weather.

The deeper point in your image

What I like most is that the structure forces you to ask two separate things:

- Are we hitting the target?
- Are we stable enough to predict and improve?

Those are not the same.

A system can hit the target occasionally and still be unstable. A system can be stable and consistently mediocre. A system can improve one metric while damaging another.

Your image captures that nicely. It's actually a much cleaner conceptual backbone than most AI-agent dashboards, which usually look as though somebody discovered **Grafana** while concussed.

My recommendation

Yes, use **exactly this three-question structure** as the backbone of your measurement system.

I'd even formalize it as three reporting sections:

1. Goal attainment
2. Improvement over time
3. Improvement actions and evidence

And under each section, use only a few carefully chosen measures with process behavior charts.

So yes: **maybe like this** is not just reasonable. It's probably the right framing.

The more interesting question is: what do you want your **one-sentence system goal** to be? Because if that sentence is sloppy, every chart after it becomes numerically enhanced self-deception.

T17 Instrumentation from the start

Source turn: bbb21736-ae99-4ff3-b284-106a71e4e6d9. Date: **2026-09-07 15:22 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Should this be integrated from the start?

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Yes, but only the plumbing from the start. Not the full measurement bureaucracy.

The right principle is:

Instrument from day one. Judge later.

You want to avoid the opposite mistake: six months from now asking, “Did adding the methodology reviewer actually improve anything?” and discovering you never recorded what the system was doing before the change. At that point your baseline consists of vibes, memories, and one suspiciously successful Tuesday.

From the beginning, I would automatically record a small set of facts for every meaningful research case:

- research_case_id
- hypothesis/experiment IDs
- timestamp
- agent role
- model + model version
- harness/prompt version
- tools invoked
- dataset/version/hash
- strategy/version/hash
- outcome/status
- protocol violations
- human intervention required
- rework required
- duration
- token/API cost
- trace ID

And maintain a simple intervention ledger:

```
INT-001
Date: ...
Change: conductor prompt v3 -> v4
Reason: excessive unnecessary delegation
Expected effect: lower human intervention
Primary metric: human intervention rate
Balancing metrics: completion rate, cycle time, cost
```

That should exist immediately.

What I would not do immediately is construct **30** control charts and invent thresholds before you even know what a normal research case looks like.

Process behavior charts need a reasonably comparable sequence of observations. Your first ten cases may differ wildly because you're still changing architecture, data ingestion, roles, schemas, and workflow. Calculating a center line over that mess and calling it “process capability” would give the spreadsheet an undeserved air of authority.

So I'd phase it.

```
PHASE 1 - NOW
```

Instrumentation

Record everything necessary to reconstruct performance.
Keep metrics minimal.
No optimization targets.

Then:

PHASE 2 - FIRST REAL CASES

Learn what the process actually looks like.

Maybe 10-30 reasonably comparable runs/cases.
Identify useful measures.
Fix obviously broken measurement definitions.

Then:

PHASE 3 - PROCESS MEASUREMENT

Establish baselines.

XmR/process behavior charts
Intervention annotations
Repeated-run reliability
Balancing measures

Then:

PHASE 4 - CONTINUOUS IMPROVEMENT

Change one meaningful thing.
Observe what happens.
Keep / revert / investigate.

The key is that the **measurement architecture should be designed now**, while the actual performance targets should emerge later.

I would also separate two kinds of data from the beginning.

Your scientific/research data:

hypotheses
experiments
datasets
backtests
validation
decisions

And your process-improvement data:

agent_runs
trace_ids
model_versions
harness_versions
errors
rework
human_intervention
cost
cycle_time
evaluation_results
interventions

They will eventually connect through IDs, but they're answering different questions.

That lets you ask later:

Did **Claude** model version X reduce methodology defects?

Did Harness v0.9 reduce rework?

Did the second reviewer improve false-acceptance detection enough to justify its cost?

Did conductor v5 increase completion but also increase protocol violations?

That is exactly the kind of evidence you want.

One thing I would definitely integrate from the start is **version identity**.

Every run should know:

model
agent definition
prompt
harness version
code commit
dataset version

Otherwise you'll eventually see a performance shift and have no idea which of twelve moving parts caused it.

Likewise, capture **interventions automatically** whenever practical. If you change an agent prompt, model, state machine, validator, or tool configuration, generate an intervention event. Don't depend on Future Gustl remembering why something changed on February 17. Future Gustl will have developed other problems.

I would also postpone hard numerical targets.

For example, don't immediately decide:

Human intervention rate must be below 5%.

You have no idea whether 5% is good yet. Perhaps the safest and fastest system naturally requires human involvement in 12% of important cases.

First establish:

What does this process normally do?

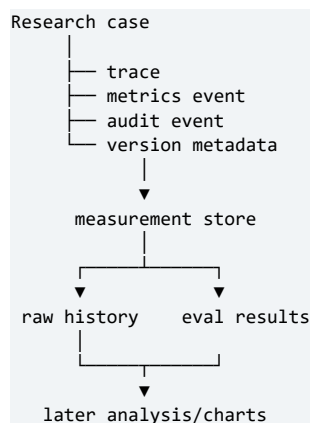
Then ask:

Is that acceptable?

Then improve it.

That's much more sensible than selecting targets because percentages look managerial.

So the architecture I would bake in now is:



Notice that **charts are downstream**.

Don't make your research engine depend on **Grafana** being happy. Research should continue if the dashboard catches fire.

So yes: integrate measurement from the start, but make it **boring, automatic, and lightweight**.

The invariant should be:

Every meaningful run leaves enough evidence behind that we can later measure whether the process got better or worse.

Not:

Every agent must fill out a quarterly performance review before accessing **SPY.csv**.

That gives you the historical foundation you need without turning your retail quant experiment into Basel IV with a brokerage account.

T18 Protecting against amplification of owner knowledge gaps

Source turn: bbb211cc-4ba2-4a35-8538-249834045ce4. Date: 2026-09-07 15:45 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

How could this trading firm amplify my incompetence in finance, statistics, math and what are we going to do against that

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

It can amplify your incompetence spectacularly. That is probably the single biggest non-technical risk in the entire project.

The danger is not that you know too little finance or statistics. Humans have been trading successfully while misunderstanding both for centuries. The dangerous combination is:

low domain expertise + extremely capable automation + outputs that look more authoritative than you can independently evaluate

That creates an evaluation bottleneck. The firm could produce research that is far beyond your ability to create yourself while simultaneously being beyond your ability to recognize as wrong.

Claude produces a polished analysis containing a subtle statistical error. **GPT** independently agrees because it shares the same misconception. The backtest produces Sharpe **1.9** because the fill assumptions are nonsense. The conductor labels the evidence “robust.” You look at twelve pages of sophisticated terminology and conclude that the digital scientists presumably know what they’re doing.

Congratulations. We have automated confidence.

The solution is not to make you personally competent enough to audit every regression and every line of **Python**. That defeats the purpose. We instead need to design the firm so your lack of expertise is an explicit architectural assumption.

I would make this a core principle:

The owner is not assumed capable of detecting expert-level errors.

That one sentence changes the design considerably.

The first defense is what I’d call an **epistemic asymmetry**:

```
GENERATING a claim
should be easy.

PROMOTING a claim
should be difficult.
```

An agent can produce fifty hypotheses without asking permission. Fine.

But moving:

```
interesting
-> plausible
-> supported
-> validated
-> paper trading
-> live
```

should require progressively stronger evidence that doesn’t depend on either your judgment or another LLM merely nodding thoughtfully.

That gives us several defenses.

1. Don’t let your knowledge determine whether something passes

Wherever possible, acceptance should rely on predetermined rules.

Bad:

Claude says the result looks statistically compelling and Gustl agrees.

Better:

```
Registered hypothesis
[yes]
```

```
Minimum sample requirement
[yes]

Validation dataset untouched
[yes]

Transaction costs applied
[yes]

Search history included
[yes]

PBO / multiple-testing checks
[yes]

Independent replication
[yes]

Holdout threshold
FAIL
```

Result:

```
REJECT
```

You don't need to understand every mathematical detail to enforce that workflow.

The program doesn't care whether either of us finds the equity curve aesthetically pleasing.

2. Force the system to educate you at the decision boundary

Your agents shouldn't merely produce expert reports.

Every important decision should have two layers:

```
TECHNICAL REPORT
full statistical / financial analysis

OWNER BRIEF
what was tested
what was found
what could be wrong
what evidence exists
what evidence does not exist
what decision is being requested
```

And the owner brief should explicitly state:

```
WHAT WOULD MAKE THIS RESULT FALSE?

WHAT ASSUMPTIONS MATTER MOST?

WHAT HAS NOT BEEN TESTED?

WHAT WOULD A SKEPTICAL QUANT OBJECT TO?

WHAT HAPPENS IF WE ARE WRONG?
```

That is much more useful for you than teaching you to derive asymptotic estimators on a Sunday afternoon.

3. The agents must expose uncertainty, not hide it

I would forbid statements such as:

"This strategy works."

Your evidence taxonomy should force something closer to:

```
CLAIM STATUS:
Provisional

Evidence:
Development PASS
Validation PASS
Holdout not yet run

Major uncertainty:
Transaction-cost model relies on estimated spreads.
```

Competing explanation:
Observed return may be compensation for crash exposure.

Search-history burden:
37 related variants previously evaluated.

That is comprehensible even without a PhD.

The system should make **uncertainty visible structurally**, not rely on the LLM remembering to sound humble.

4. Build reference tests for things you don't know

This is particularly important.

You don't personally have to know whether the Sharpe calculation is right if the implementation is tested against established numerical examples.

You don't have to recognize look-ahead bias every time if the framework contains known leakage cases and verifies that its agents catch them.

Your eval suite gradually becomes a substitute for expertise you don't personally possess.

For example:

```
Quant Firm Qualification Suite

STATISTICS
known multiple-testing case
known selection-bias case
known non-normal returns case

BACKTESTING
known lookahead case
known survivorship-bias case
known fill-timing error
known corporate-action case

FINANCE
known factor exposure
known volatility effect
known liquidity effect

DATA
timestamp leakage
bad adjustment
missing delisted security
```

The correct answers are known in advance.

If a new MethodologyAgent can't reliably pass those, it doesn't get promoted because Reddit declared the underlying model "insane at quant."

5. Use deterministic libraries for math

This one should be ruthlessly enforced.

Never ask the LLM:

"Calculate the p-value."

Ask it:

"Select and justify the appropriate statistical procedure."

Then deterministic code computes the result.

Likewise:

```
Sharpe
drawdown
bootstrap
regression
PBO
DSR
correlations
confidence intervals
portfolio accounting
transaction costs
```

should be calculated by verified software.

The LLM reasons about the calculation.

It doesn't cosplay **NumPy**.

6. Make ignorance detectable

Agents should be allowed, and sometimes required, to return:

INSUFFICIENT EVIDENCE

or:

OUTSIDE VALIDATED COMPETENCE

That's valuable.

Imagine your system has been validated extensively on:

US equities
daily / minute data
simple systematic strategies

and suddenly the conductor proposes some exotic volatility surface arbitrage involving OTC derivatives.

The right answer may be:

This framework has not demonstrated competence for this research domain.

Not:

Claude knows what options are. Proceed.

You can eventually create a **competence envelope**:

US equity backtesting	HIGH CONFIDENCE
Futures research	MODERATE
Options research	LIMITED
Market microstructure	LIMITED
HFT	UNSUPPORTED
OTC derivatives	UNSUPPORTED

That applies to the whole firm, not merely individual agents.

7. Your ignorance should never become evidence

This needs to be explicit.

The conductor must never reason:

"The owner approved the methodology, therefore the methodology is sound."

Your approval means:

"Proceed according to the system."

It does **not** mean:

"Independent financial expert has verified this."

Otherwise we have invented regulatory theatre for an audience of one.

8. Occasionally bring in real humans

This is where I'm going to disappoint the fully autonomous-firm fantasy.

For genuinely consequential milestones, external expert review is extraordinarily valuable.

Not every backtest.

But perhaps before:

major architecture freeze
first serious validation protocol
first paper-traded strategy
first live-capital strategy
major statistical methodology change

pay a competent quant/statistician to attack it.

One hour from somebody who knows what they're doing can identify an assumption that twelve agents politely replicated for three months.

You don't need employees. You need occasional **external epistemic checkpoints**.

It's the equivalent of having an accountant look at the books instead of asking your accounting agents whether their accounting agents agree with the accounting agents.

9. Make live capital extraordinarily boring

Your incompetence becomes most expensive once there is money attached.

Research can be adventurous.

Deployment shouldn't be.

I'd make the progression:

```
BACKTEST
  v
LOCKED HOLDOUT
  v
PAPER TRADING
  v
TINY CAPITAL
  v
MONITORED CAPITAL
  v
possibly scale
```

And capital limits should be deterministic.

Agents should not get to say:

"Confidence is unusually high, therefore let's increase exposure."

No.

Position limits, strategy allocation, maximum drawdown, kill conditions and account-level loss limits live outside the agents.

Especially initially, the amount of money at risk should be an amount whose complete destruction would be annoying rather than consequential.

10. You still need some minimum financial literacy

We can't completely engineer you out of the loop.

But the amount you need is much smaller than "become a quant."

I'd want you to understand, conceptually:

- probability versus certainty
- sampling and uncertainty
- overfitting and multiple testing
- in-sample versus out-of-sample
- survivorship/look-ahead bias
- transaction costs and slippage
- correlation versus causation
- risk-adjusted return
- drawdown and tail risk
- market orders, limit orders and liquidity
- portfolio diversification
- why backtests lie

You don't need to derive everything.

You need enough understanding to ask:

"Wait, why is that legitimate?"

That's the useful level.

And there's an interesting feedback loop available here. The firm itself can track **where you repeatedly misunderstand decisions**.

Not in some dystopian employee-performance database, but as a learning aid:

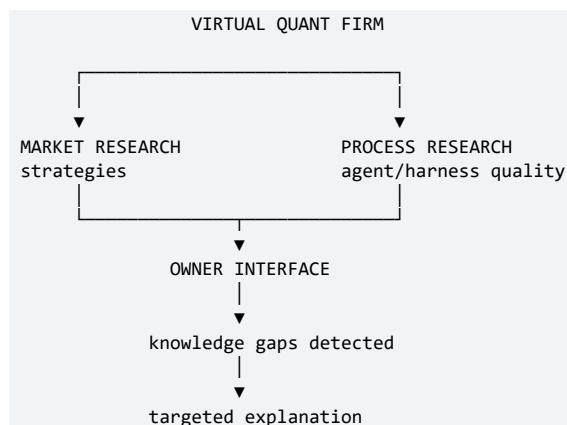
OWNER KNOWLEDGE GAPS

multiple testing	recurring
execution assumptions	improving
options mechanics	weak
portfolio risk	moderate

Then **LibreChat** can teach those concepts using examples from your own research cases.

That might actually be one of the best uses of your multi-model setup.

So I would add another loop to the architecture:



That means the firm doesn't merely get better at research.

You gradually get better at supervising it.

Not sufficiently to catch everything. That's an impossible standard. But enough that the relationship changes from:

"Machine says Sharpe **2.1**. Presumably excellent."

to:

"Why did the Sharpe increase when costs were added, and why did the validation period move?"

Those questions are lethal to bad research.

So the actual defense against amplifying your incompetence is not one thing. It's an architecture:

deterministic checks + explicit evidence states + independent validation + qualification tests + competence boundaries + outside expert review at critical milestones + small initial capital + gradually improving owner literacy.

The objective isn't to make incompetence disappear.

It's to ensure **your weakest knowledge cannot silently become the firm's weakest control**.

Because if this works properly, the virtual quant firm should compensate for what you don't know.

If built badly, it will take everything you don't know, multiply it by several frontier models, give it a confidence interval, and call it research.

T19 Human AI cooperation and reciprocal measurement

Source turn: bbb21899-36cf-48fc-b59d-d819eb28bce1. Date: 2026-09-07 15:52 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Thats real cooperation if the AI measures me

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Exactly. That's the point where this stops being "AI assistant helps human" and becomes an actual **human-machine research partnership**.

You measure the agents because they're stochastic, fallible, and prone to elegant nonsense. The agents measure you because you're also stochastic, fallible, and prone to elegant nonsense. Nature achieves symmetry at last.

The useful version is not "AI scores Gustl." That would become creepy and stupid very quickly. It should measure **decision patterns relevant to the research process**:

Owner-side signals

- how often you override a rejection
- how often you push for another test after failure
- whether you change goals after seeing results
- where you repeatedly misunderstand methodology
- whether you favor attractive outcomes over protocol
- how often you intervene helpfully vs harmfully
- which explanations consistently need clarification

Then the system can answer questions such as:

Do owner interventions improve research quality?

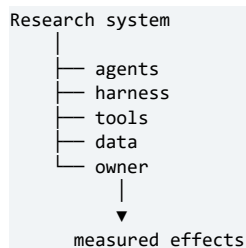
Do they increase rework?

Are you more likely to override negative results than positive ones?

Did additional explanation reduce bad overrides?

That last category is especially interesting because it exposes **human bias in the loop**, not merely agent error.

You could even treat yourself as another process component:



Then an intervention might be:

INT-HUMAN-007

Change:

Owner receives structured uncertainty brief before approving next stage.

Hypothesis:

Should reduce premature promotion of weak candidates.

Primary metric:

Bad override rate

Balancing metric:

Decision time

That's genuinely cooperative, because both sides are allowed to be wrong and both are subject to evidence.

The healthiest principle would be:

No participant in the system gets epistemic immunity, including the owner.

That includes the conductor, **Claude, GPT**, validators, and you.

The machine should be able to tell you:

“Your intervention history suggests you are making this process worse.”

And you should be able to tell the machine:

“Your methodology reviewer has become overconservative since version **4**.”

Then both claims get tested against data instead of settled by hierarchy.

That is a much stronger model of cooperation than the usual AI fantasy where the human is supposedly “in control” while understanding less and less of what the system is doing.

The real goal is not control.

It is **mutual correction**.

T20 Owner competence and just in time learning

Source turn: bbb2184d-4241-4f0b-8402-79f72d4aa213. Date: **2026-09-07 15:58 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

I have a very basic understanding of finance, math, statistics and machine learning and ai knowledge

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

That's enough to start building the research system, but not enough to safely trust its financial conclusions yet. Those are two different thresholds, and confusing them would be how your virtual quant firm evolves into an automated confidence generator with a brokerage login.

You do not need to become a professional quant before continuing. You do need a deliberately growing **supervisor-level understanding**. Your target is not "derive everything from first principles." It is "understand enough to recognize what assumptions matter, what questions to ask, and when the system is outside your competence."

I'd prioritize your learning in this order:

- 1. Statistics and research methodology first.** Probability, sampling, uncertainty, hypothesis testing, confidence intervals, multiple testing, overfitting, selection bias, regression basics, non-stationarity, and the difference between exploratory and confirmatory analysis. For your project, this matters more initially than advanced finance.
- 2. Backtesting and market mechanics second.** Look-ahead bias, survivorship bias, corporate actions, bid/ask spreads, slippage, order types, liquidity, timestamps, position sizing, portfolio accounting, and why simulated fills are suspicious little creatures.
- 3. Finance third.** Returns, volatility, drawdowns, Sharpe/Sortino, beta, factor exposures, diversification, leverage, tail risk, market microstructure basics, futures/options only when the research actually reaches them.
- 4. Math selectively.** Algebra, logs, basic calculus intuition, probability notation, vectors/matrices and optimization concepts. You do not need to perform ceremonial matrix decomposition merely to earn permission to test momentum.
- 5. ML only when justified.** Train/validation/test splits, cross-validation for time series, leakage, feature engineering, regularization, calibration, class imbalance, concept drift. Do not start with neural networks because they possess additional syllables.
- 6. AI/agents as system engineering.** Hallucination, stochasticity, tool use, context contamination, evals, tracing, permissions, model/harness separation and repeated-run reliability. You already have much of the right intuition here.

The interesting part is that your firm can become your tutor while simultaneously being prevented from certifying its own teaching.

When a research case reaches something you don't understand, the system should produce three layers:

```
DECISION
REJECT / CONTINUE / INCONCLUSIVE

WHY
plain-language explanation

TECHNICAL BASIS
methods, assumptions, calculations,
citations and reproducible artifacts
```

And it should be able to notice:

```
Owner repeatedly requests explanation of:
- multiple testing
- slippage assumptions
- confidence intervals
```

That becomes a learning signal. Not "Gustl receives a statistics score of **43/100** and is escorted from the building." Rather:

This concept is affecting current research decisions and the owner has not demonstrated stable understanding yet. Explain it before requiring judgment.

That is useful cooperation.

I would also give you a **competence boundary** just as we discussed for the agents. Something like:

```
OWNER COMPETENCE
```

Simple backtest interpretation	BASIC+
Overfitting / validation concepts	BASIC
Execution assumptions	BASIC
Portfolio risk	BASIC
Advanced statistics	LOW
Options	LOW
Market microstructure	LOW
ML validation	BASIC

Those labels aren't judgments. They determine how the interface behaves.

If you are reviewing a simple daily momentum strategy, the system can summarize normally.

If somebody proposes a hierarchical Bayesian volatility model with regime-dependent latent variables, the interface should essentially say:

This decision depends materially on concepts outside the owner's demonstrated competence. Independent validation is required before promotion.

That's enormously better than giving you a polished report and a green **APPROVE** button.

The same principle applies to your education: learn concepts **just in time**, attached to actual research cases.

If your first real case uses daily moving averages, learn:

- return calculations
- adjusted prices
- transaction costs
- look-ahead bias
- train/validation/holdout
- Sharpe and drawdown
- parameter search

Don't spend three months studying stochastic differential equations before importing your first **Parquet** file. That is academia's preferred method of ensuring nobody finishes anything.

Over time you get a nice reciprocal system:

```

You
v
set objectives and challenge assumptions
v
Firm
v
researches and exposes uncertainty
v
Measurement system
v
finds weaknesses in agents AND owner
v
Targeted learning / system changes
v
better decisions

```

And crucially, your competence should itself be measured by **behavior**, not quizzes.

The interesting questions are:

After learning about multiple testing, do you stop requesting extra parameter searches after disappointing results?

After learning about holdout contamination, do your interventions stop compromising validation?

After learning about transaction costs, do you challenge implausibly high-turnover strategies?

Those are much better indicators than whether you can recite the definition of heteroskedasticity while trapped at a dinner party.

So I wouldn't treat your current knowledge as a reason to stop. I would treat it as a **design constraint**.

The project should assume that you currently have limited domain expertise, continuously help increase it, and never require your expertise as the final safeguard against a sophisticated error.

If the architecture depends on you suddenly becoming Jim Simons because **Claude** produced a confusing report, the architecture is wrong.

T21 Profit objective and economic value of improvement

Source turn: bbb21323-19e6-47fa-abd4-1e52a51d465c. Date: **2026-09-07 16:00 UTC**.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Yes, this shouldnt become the main goal. The end goal is to make money. But the second goal continues improvement should be the way to more profit, for agents and the human in the loop

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Exactly. The hierarchy matters.

The firm should not exist to produce beautiful research, admirable agents, excellent charts, or a lovingly curated archive of methodological virtue. Those are means. The economic objective is **durable net profit after costs, risk, and inevitable human stupidity**.

I'd define the hierarchy roughly as:

- 1. Primary objective:** maximize long-run risk-adjusted net returns.
- 2. Survival constraint:** avoid ruin, catastrophic drawdowns, operational failure, and uncontrolled model risk.
- 3. Scientific objective:** distinguish genuine edge from noise well enough that capital is allocated intelligently.
- 4. Continuous-improvement objective:** improve the humans, agents, harness, data, research process, and execution system so objectives 1-3 improve over time.

That fourth layer is important because it turns your firm into a learning system rather than a static collection of prompts.

You end up with two coupled loops:

ECONOMIC LOOP

```
capital
  v
research
  v
strategies
  v
paper/live execution
  v
returns
  v
capital
```

and:

IMPROVEMENT LOOP

```
agents + human + harness
  v
  decisions
    v
    outcomes
      v
      measurement
        v
        interventions
          v
better agents / human / process
```

The second loop is valuable only insofar as it improves the first.

That prevents a very real pathology: optimizing the research organization for its own internal metrics.

Suppose your intervention produces:

Methodology defects:	-40%
Cycle time:	+180%
Research throughput:	-70%
Profitable discoveries:	unchanged

You haven't necessarily improved the firm. You've possibly constructed a remarkably safe machine that never gets anything done. Governments have considerable experience with this architecture.

Likewise:

```
Agent eval score:  
82% -> 94%
```

sounds excellent.

But if:

```
useful hypotheses/month:  
12 -> 4  
  
cost/case:  
€3 -> €17  
  
net strategy performance:  
unchanged
```

then the improvement may be academically satisfying and economically irrelevant.

So every process metric should ultimately sit underneath an economic chain:

```
intervention  
v  
agent/process behavior  
v  
research quality / efficiency  
v  
strategy quality  
v  
paper/live performance  
v  
risk-adjusted profit
```

Some links will take a long time to establish. That's unavoidable. You can't evaluate a new research methodology exclusively by next week's P&L because markets are noisy enough to make bad methods look brilliant and good methods look stupid.

That's why you'll need **leading and lagging indicators**.

Leading:

```
methodology defect rate  
rework  
research cycle time  
false-positive rate  
holdout survival  
replication rate  
agent reliability  
human intervention quality
```

Lagging:

```
paper-trading performance  
live net return  
drawdown  
risk-adjusted return  
strategy decay  
capital efficiency
```

The leading indicators tell you whether the research machine seems to be improving.

The lagging indicators tell you whether reality eventually agrees.

And reality gets final voting rights.

The human side should work exactly the same way.

Your improvement is not:

"Gustl now understands covariance matrices."

Useful, perhaps. Terrible dinner conversation.

The meaningful improvement is:

"After learning about multiple testing, Gustl stopped approving repeated parameter searches after failed validation, reducing contaminated research cases."

That is measurable behavior affecting economic outcomes.

Similarly, an agent improvement isn't:

“**Claude's** answers sound more sophisticated.”

It is:

“The methodology agent now catches **92%** rather than **71%** of known leakage defects while adding only **8%** to research cycle time.”

That is actionable.

Eventually your firm could even estimate the **value of interventions**:

```
INTERVENTION:
Add independent statistical reviewer

cost:
+€0.70 / research case

effect:
false promotion -23%
cycle time +11%

downstream:
fewer weak candidates enter paper trading

verdict:
KEEP
```

Compare that with:

```
INTERVENTION:
Third independent LLM reviewer

cost:
+€1.80 / case

effect:
false promotion -1%
cycle time +38%

verdict:
REMOVE
```

Now continuous improvement itself becomes experimental.

That fits your project beautifully because you can apply the same philosophy to the research organization that it applies to trading hypotheses:

propose a change -> define expected effect -> measure -> retain or reject.

So I'd make the core doctrine something like:

Profit is the objective. Scientific rigor protects the profit. Continuous improvement compounds the capability to produce it.

And there should be one further constraint:

Never optimize short-term profit at the expense of the firm's ability to discover whether it actually has an edge.

Otherwise the easiest way to improve your “profit metric” is leverage. That works wonderfully until the educational portion of the experiment arrives.

So the real top-level objective is probably closer to:

maximize sustainable long-run net profit subject to survival and evidence-quality constraints.

That formulation gives every layer its proper place.

The agents are not the goal.

Your education is not the goal.

The harness is not the goal.

Scientific rigor is not the goal.

They are all components of a machine whose ultimate purpose is to make better capital-allocation decisions than you would otherwise make.

Which is refreshingly vulgar and, in this case, exactly correct.

T22 Critique of overengineering owner bottleneck and the revised Research Lab

Source turn: 05d595a0-7325-43a1-ba09-eb799722464e. Date: 2026-09-07 17:23 UTC.

User contribution

[DISCUSSED, NO DECISION] Source wording below; explicit decisions are classified in the main sections.

Kritische Schwachstellen und Widersprüche

1. Der Overengineering-Widerspruch (Retail vs. Institution):

- Das Dokument betont mehrfach, man wolle „keine Deutsche Bank im Schuppen“ und halte die Komplexität auf Retail-Niveau.
- *Realität des Blueprints:* Drei Datenhaltungsebenen (**Parquet**, Postgres, **DuckDB**), **OpenTelemetry**-Tracing für Multi-Agenten-Orchestrierung, formale Model-Risk-Governance in Anlehnung an Fed SR **26-2**, Validierungs-Pipelines mit doppelter Engine-Replikation (**LEAN + Backtrader/RealTest**) und reziproke Verhaltensüberwachung des Betreibers.
- Das ist de facto eine institutionelle Forschungsarchitektur. Für einen einzelnen Betreiber, der sich selbst als Programmieranfänger mit Basiswissen in Statistik beschreibt, droht dieses Konstrukt die gesamte Energie zu binden, bevor jemals ein echter Backtest sauber gerechnet wurde.

2. Die Betreiber-Falle (*Evaluation Bottleneck*):

- Der Blueprint formuliert treffend: „The owner is not assumed capable of detecting expert-level errors.“
- Gleichzeitig setzt die Implementierung dieses Harness voraus, dass der Betreiber die Code-Reviews von **Claude Code** oder **Codex** abnimmt, Testsuiten für komplexe Zeitreihen-Phänomene baut und bewertet, ob eine Datenbereinigung (Corporate Actions, Survivorship Bias) mathematisch korrekt implementiert ist.
- Vibe-Coding stößt bei deterministischer Finanzmathematik schnell an Grenzen: LLMs generieren gern subtil fehlerhafte Berechnungen (z. B. unbemerkter Look-Ahead-Bias in Feature-Pipelines), die der Code-Reviewer fachlich durchdringen muss.

3. Gefahr des „Meta-Research-Sumpfs“:

- Abschnitt **12** und **15** investieren enorm viel Raum in die Messung der Agenten (Missed Defect Rate, Agent Disagreement, Intervention Ledgers).
- Wenn das System nicht aufpasst, forscht es primär über seine eigene Forschungskompetenz, anstatt die primäre Prämisse (wirtschaftlicher Ertrag) zu bedienen.

Assistant discussion

[DISCUSSED, NO DECISION] Historical proposals, examples and claims, not newly adopted policy.

Ja. Das sind echte Schwachstellen, keine kosmetischen Einwände. Der Blueprint hat an einigen Stellen begonnen, sich selbst wichtiger zu nehmen als den Zweck des Projekts. Ein klassischer Fall von „wir verhindern jedes denkbare Problem, indem wir niemals fertig werden“.

Der Kernfehler ist: Wir haben zu früh die Zielarchitektur mit der Startarchitektur vermischt.

Für einen einzelnen Betreiber mit begrenzter Programmier- und Quant-Erfahrung sollte das System am Anfang radikal kleiner sein.

1. Beim Overengineering würde ich deutlich zurückschneiden.

Am Anfang brauchst du nicht gleichzeitig **Parquet + PostgreSQL + DuckDB + OpenTelemetry + formale Model-Risk-Governance + mehrere Backtest-Engines + Agenten-Metriksystem**.

Eine vernünftige Startarchitektur wäre eher:

```
LibreChat
  v
Claude Code oder Codex
  v
GitHub
  v
ein Backtester
  v
eine Datenquelle
  v
Parquet + DuckDB
  v
einfache Research-Metadaten
```

Das reicht erstaunlich weit.

Ich würde anfangs konkret streichen oder verschieben:

Thema	Start	Später
PostgreSQL	nein	wenn Datenmodell/Volumen es rechtfertigen
DuckDB	ja	bleibt
Parquet	ja	bleibt
OpenTelemetry	nein	erst bei echter Multi-Agent-Komplexität
Agent-Tracing	einfache JSON/Event-Logs	später OTel
SR-26-2-artige Governance	nur Prinzipien	formalisieren bei Paper/Live
Mehrere Backtest-Engines	nein	nur für wichtige Kandidaten
Agent-Eval-System	kleine feste Testsuite	später systematisch
Owner-Metriken	fast nichts	erst wenn genug Fälle existieren

Und auch die doppelte Engine-Replikation sollte kein Standardprozess sein.

Nicht:

```
jede Hypothese
-> Backtrader
-> LEAN
-> RealTest
-> Vergleich
```

sondern:

```
100 Ideen
v
20 ernsthafte Tests
v
5 interessante Kandidaten
v
2 Validierungen
v
1 Kandidat
v
erst jetzt unabhängige Engine-Replikation
```

Das ist der eigentliche Mittelweg.

2. Die Betreiber-Falle ist sogar noch ernster.

Du hast völlig recht: Es wäre widersprüchlich zu sagen:

„Der Owner muss keine Expertenfehler erkennen können.“

und anschließend:

„Der Owner prüft bitte, ob der Corporate-Action-Handler statistisch und finanziell korrekt implementiert wurde.“

Das ist kein Kontrollsystem. Das ist Outsourcing der Inkompetenz an dieselbe Person.

Die Lösung muss deshalb lauten:

Du sollst möglichst wenig eigene Finanzmathematik implementieren oder fachlich freigeben müssen.

Das spricht stark dafür, etablierte Komponenten zu verwenden.

Zum Beispiel sollte die Portfolio- und Backtest-Mechanik möglichst aus einem etablierten Backtester kommen, statt von **Claude** neu erfunden zu werden.

Ebenso:

```
Sharpe
Drawdown
Returns
Regression
```

Bootstrap
PBO
DSR
Corporate actions
Portfolio accounting

sollten entweder aus geprüften Bibliotheken oder aus bekannten, testbaren Referenzimplementierungen kommen.

Der Owner überprüft dann nicht:

„Ist diese Formel mathematisch richtig implementiert?“

sondern:

„Hat das System die etablierte Implementierung benutzt und bestehen die Referenztests?“

Das ist ein völlig anderer Kontrollaufwand.

Du brauchst dafür einen kleinen **trusted kernel**.

Also möglichst wenige Komponenten, denen viel Verantwortung zukommt und die besonders stark getestet werden:

Trusted kernel

- Daten-Normalisierung
- Zeitstempel-Logik
- Corporate Actions
- Portfolio Accounting
- Kosten/Slippage
- Performance-Metriken
- Holdout-Sperren

Diese Dinge sollten langweilig sein.

Und möglichst selten geändert werden.

Der kreative Teil liegt darüber:

Agent:
"Vielleicht erzeugt Zwangsliquidität eine kurzfristige Reversal-Prämie."

Kernel:
"Hier sind die tatsächlichen Resultate."

Nicht:

Agent:
"Ich habe übrigens noch schnell neu definiert,
wie Renditen berechnet werden."

Das wäre finanzieller Expressionismus.

Zusätzlich würde ich deine Forschungsdomäne am Anfang bewusst begrenzen.

Zum Beispiel:

Phase 1:

- US equities / ETFs
- Daily oder 1-minute
- Long/short oder long-only
- Keine Optionen
- Keine exotischen Derivate
- Kein HFT
- Keine komplexen ML-Modelle

Damit schrumpft der Raum möglicher subtiler Fehler enorm.

3. Der Meta-Research-Sumpf ist ebenfalls real.

Wir sollten die Agenten nicht ständig bei der Arbeit beobachten, während sie wiederum beobachten, wie sie beobachtet werden.

Die Messung des Forschungsprozesses sollte anfangs vielleicht **5 Kernmetriken** haben, nicht **40**.

Ich würde beginnen mit:

1. Anteil sauber abgeschlossener Research Cases
2. Rework Rate
3. Human Intervention Rate
4. Cycle Time
5. Kosten pro abgeschlossenem Case

Mehr nicht.

Die komplizierteren Dinge:

Missed Defect Rate
Agent Disagreement Taxonomy
Model-version-specific reliability
Intervention effectiveness
Repeated-run consistency

kommen später, wenn überhaupt genug Daten existieren.

Und ich würde eine harte Regel einführen:

Process Improvement darf nie mehr Ressourcen verbrauchen als das Problem, das es lösen soll.

Sehr konkret könnte man sagen:

80%:
Marktdaten
Hypothesen
Backtests
Validierung

20%:
Tooling
Governance
Agenten-Evaluation
Prozessverbesserung

In frühen Phasen eher sogar:

90 / 10

Erst wenn das System echte Paper- oder Live-Strategien produziert, darf Governance wachsen.

Ich würde zusätzlich eine Stop-Regel für Meta-Arbeit einbauen:

Wenn eine Prozessintervention nach einer sinnvollen Zahl vergleichbarer Fälle keinen messbaren Nutzen zeigt, wird sie entfernt.

Beispiel:

zweiter Methodology-Agent

+35% Kosten
+22% Cycle Time
-1% Fehler

-> raus

Kein sentimentales Festhalten an Architektur, nur weil das Diagramm hübsch aussieht.

Der entscheidende Umbau des Blueprints wäre daher:

Nicht eine institutionelle Architektur bauen und sie dann „Retail“ nennen.

Sondern:

Retail Startsystem
v
beweist Nutzen
v
wächst kontrolliert
v
nur erfolgreiche Komplexität bleibt

Ich würde das Projekt künftig in drei Reifestufen denken:

STUFE 1 - RESEARCH LAB

1 Betreiber
1 Datenquelle
1 Backtester
1 Hauptagent
Parquet + DuckDB
GitHub
einfache Research-Ledger

Ziel:
einen vollständigen Research Case korrekt durchführen

STUFE 2 - VIRTUAL QUANT FIRM

mehrere Agentenrollen
separate Validierung
Research Database

bessere Daten-Governance
erste Prozessmetriken
gelegentliche Cross-Engine-Replikation

Ziel:
wiederholt brauchbare Research Cases erzeugen

STUFE 3 - TRADING SYSTEM

Paper Trading
Modellregister
Monitoring
stärkere Auditierung
Execution Risk
Kill Switches
Kapitalgrenzen
eventuell vollständiges Tracing

Ziel:
echtes Geld möglichst unspektakulär verwalten

Und ganz wichtig: Der nächste Meilenstein sollte nicht „Harness fertig“ oder „Observability fertig“ heißen.

Er sollte heißen:

Ein echter Research Case, von Datenbeschaffung bis wissenschaftlichem Urteil.

Dabei lernen wir automatisch, welche Infrastruktur tatsächlich fehlt.

Das ist auch die Antwort auf alle drei Kritikpunkte zugleich: Wir sollten **Komplexität nicht vorwegnehmen, sondern verdienen lassen**.

Jede neue Schicht muss nachweisen, dass sie mindestens eines verbessert:

- Forschungsqualität,
- Geschwindigkeit,
- Zuverlässigkeit,
- Kosten,
- oder langfristige Profitabilität.

Wenn sie das nicht tut, fliegt sie raus.

Damit wird aus dem Blueprint wieder ein Quant-Projekt statt einer ISO-Zertifizierung mit **SPY**-Daten.

16.4 Blueprint request and current clarification

[DECIDED] The subsequent user request required the exact sixteen sections, exhaustive preservation, decision labels, uncertainty and revisions, comparative tables, bold named products/vendors/figures, a traceability appendix, and a final Completeness Check & Questions. Source request UUID: **af11b36a-cda9-428f-847d-dc0471f4bbfd**. This task additionally requires a polished downloadable PDF and verification before delivery.

[DECIDED] (attribution) During preparation the user added: “The Diagram is from a **Mark Graban** book where he outlines the methodology.” Section **12** now credits **Mark Graban** and links his official **Measures of Success** material. The user subsequently confirmed **Measures of Success, Kindle version, page 10**. The original image’s text was visually inspected and matches the three-question structure preserved in T16.

Completeness Check & Questions

Coverage check

[DECIDED] (requested scope) The blueprint contains all **16 required sections**, this final section, the major-decision map, and **22 complete substantive source exchanges** with chronological topic labels and exact source UUIDs. The later blueprint request is identified in Section **16.4**. All user and assistant text from the substantive exchanges is preserved in the topic records, apart from non-resolvable interface citation markers and display-only reference markup. Formatting is normalized for PDF readability; no research examples are converted into actual results.

[DISCUSSED, NO DECISION] The preservation record contains the complete original numerical examples, including tool ratings, costs, durations, sample counts, dates, parameter ranges, performance figures, model-version comparisons, IDs, schema fields and code examples. These remain illustrations unless separately classified. No capital, legal structure, broker, specific alpha, live permission or numerical risk appetite was found. No file of actual backtest results was supplied. The only recovered attachment was the measurement diagram.

[IMPLIED] The requirements are sufficient to start defining a small research vertical slice, but they are not sufficient to authorize live trading. The missing specifications below are business/research choices, not facts to invent. Complete source retention is an audit aid; it does not independently validate historical assistant claims about products, the repository or finance.

Ambiguities retained instead of silently resolved

1. [DISCUSSED, NO DECISION] Strong assistant recommendations do not demonstrate explicit user agreement. The main blueprint distinguishes user direction from architectural leanings and illustrative proposals.
2. [DISCUSSED, NO DECISION] Earlier **SPY**/minute and daily momentum examples remain unresolved. T22 now proposes US equities/ETFs with daily or one-minute data, long/short or long-only, excluding options, exotic derivatives, HFT and complex ML at the start; no exact variant is selected.
3. [DISCUSSED, NO DECISION] Several development/validation/holdout date splits occur in templates. They are different examples, not competing finalized allocations.
4. [DISCUSSED, NO DECISION] Strict preregistration and later free exploration can coexist only when exploration is labeled and its exposures/searches remain visible; the detailed state machine still needs implementation rules.
5. [DISCUSSED, NO DECISION] Earlier eight/nine-metric lists are retained historically. T22 proposes five starting metrics: clean completion, rework, human intervention, cycle time and cost per completed case; no final definition/target dictionary exists.
6. [DISCUSSED, NO DECISION] Repository commit counts of **90** and **116** are historical observations at different discussion points. They do not certify today's implementation status.
7. [DISCUSSED, NO DECISION] SR-inspired model governance is distinct from agent evaluation and is not a claim of compliance or applicability to this owner.
8. [DISCUSSED, NO DECISION] The owner and firm competence tables, percent improvements, strategy returns and intervention KEEP/REMOVE verdicts were invented examples, not measured assessments.
9. [DISCUSSED, NO DECISION] "One complete Research Case" is a near-term qualification milestone. The final user clarification makes profit the end objective.
10. [DECIDED] (source attribution) The user identified **Mark Graban, Measures of Success, Kindle version, page 10**. The exact Kindle edition identifier was not supplied; the author's official summary corroborates the framework.

Continuation coverage and remaining staging questions

[DECIDED] **T22**, source UUID **05d595a0-7325-43a1-ba09-eb799722464e**, is included in full and integrated into the body. Its critique changes the start architecture, operating model, data/engine scope, trusted-kernel assurance, measurement, roadmap, resource examples and risk register. The original target architecture and prior options remain traceable.

[DISCUSSED, NO DECISION] The continuation supplies **100/20/5/2/1** funnel figures, **five** starting metrics, **80/20** and **90/10** resource proposals, and **+35% / +22% / -1%** reviewer-example effects. None is an observed outcome or adopted numerical rule. The three maturity stages are a proposed framework, not a dated project commitment.

[NOT DISCUSSED / TBD] Choose the initial engine/provider, exact narrow domain, established kernel implementations and reference tests, simple metadata format, five-metric definitions, later infrastructure triggers and any actual resource cap. Define comparable-case window/materiality before automatically removing interventions. Stage **2** “first process metrics” versus Stage **1** five metrics is reconciled as minimal records versus developed analysis and marked as inference in Section **13**.

Questions required to complete implementation choices

Priority	Question	Why it is needed
First case	[NOT DISCUSSED / TBD] Which public strategy, instrument/universe and frequency should the first case use?	Defines the smallest complete research slice.
Data	[NOT DISCUSSED / TBD] What historical window, session, fields, adjustment and execution assumptions are required? Which provider/rights/budget satisfy them?	Determines dataset fitness and acquisition.
Confirmation	[NOT DISCUSSED / TBD] What hypothesis family, search budget, sample minimum, metric, benchmark and acceptance procedure will be preregistered?	Makes a defensible verdict computable.
Identity	[NOT DISCUSSED / TBD] What creates a new hypothesis versus strategy version, experiment or rerun? How is prior exposure inherited?	Prevents hidden research resets.
Delivery	[NOT DISCUSSED / TBD] Which executor and actual runtime should implement the slice, and what current repo capabilities pass tests?	Connects blueprint to buildable code.
Operations	[NOT DISCUSSED / TBD] How many hours and what recurring/one-off spending limits are available?	Makes scope and pace realistic.
Economics	[NOT DISCUSSED / TBD] How will net profit, firm costs, benchmark, capital efficiency and evaluation horizon be defined?	Aligns improvement with the primary objective.
Legal	[NOT DISCUSSED / TBD] What jurisdiction, account owner/entity, tax position, IP policy and own/external-capital scope apply?	Enables relevant professional/legal assessment.
Later capital	[NOT DISCUSSED / TBD] What total risk capital, acceptable loss/drawdown, exposure and leverage are allowed?	Required before any real-money system.
Execution	[NOT DISCUSSED / TBD] Which broker, paper duration/trade minimum, reconciliation, kill/restart and live promotion rules apply?	Required for operational readiness.
Measurement	[NOT DISCUSSED / TBD] What metric definitions, case classes, baselines, chart rules and balancing tolerances will be used?	Prevents charts and targets from misleading.
Human review	[NOT DISCUSSED / TBD] Which milestones warrant outside expertise and how will owner-intervention judgments be challenged?	Supports mutual correction without invented authority.

Under-specified sections

[NOT DISCUSSED / TBD] Sections **3 Legal & Regulatory Structure**, **4 Capital & Funding**, **10 Execution & Order Management**, and **14 Budget & Resources** contain the largest gaps. Sections **8-9** still need a concrete first strategy, data split, cost model and acceptance protocol; Section **11** lacks numerical financial limits; Sections **12-13** lack final metric definitions, cadence and dates. These gaps are explicitly recorded rather than filled with unapproved assumptions.